codex-windows / 019f2174-3b01-7d91-8951-acfb4159fda3

Metadata

- Source: `codex-windows`
- Kind: `codex`
- Source file: `C:\Users\avidu\.codex\archived_sessions\rollout-2026-07-02T11-41-33-019f2174-3b01-7d91-8951-acfb4159fda3.jsonl`
- SHA-256: `127bbf7ca9b3d5f0bfefb86cc91fe11d9759bb522a1875d1f01d70a8d529e0de`
- Source modified: `2026-07-02T06:11:33+00:00`
- Imported at: `2026-07-05T16:48:16+00:00`
- cli_version: `0.142.5`
- cwd: `C:\Users\avidu\Projects\badminton-highlight-indexer`
- model_provider: `openai`
- session_id: `019f2174-3b01-7d91-8951-acfb4159fda3`
- source: `vscode`

Transcript

1. user (2026-07-02T06:11:33.251Z)

In the badminton-highlight-indexer repo, `tests/test_ablation.py::test_litmus_reproduces_gate_a` and `::test_litmus_report_is_well_formed` FAIL locally with `KeyError: 'shuttle'` when the full n=11 golden corpus is present in `output/`.

Root cause (confirmed): `_golden_present()` (tests/test_ablation.py ~line 366) globs ALL `output/*_golden_features.json` (runs the litmus when ?6 exist), and the litmus path in `backend/eval/ablation.py` accesses `candidate['shuttle']` (also `['person']`, `['audio']`). But the 5 newer real-footage clips added during the R3/R4 corpus growth ? `mahadevpura_1`, `mahadevpura_2`, `GX010128`, `GX030094`, `Badminton_BXH_2` ? have a LEANER candidate schema produced by the velocity-windowing/distill path: keys `['active_frame_count','chunk_index','end','mean_velocity','peak_velocity','start','track_id_count']`, with NO `shuttle`/`person`/`audio` feature dicts. The original n=6 files (e.g. `adarsh_avi_singles`) have the full fusion schema `['audio','end','label','person','shuttle','start']`. The litmus crashes on the leaner files.

This is local-only: CI's `output/` is empty so these tests SKIP (green there). But on the owner's machine `python run_all_tests.py` shows 2 reds.

Pick the right fix (your call): (a) make the litmus loader FILTER to golden_features whose candidates carry the full fusion feature schema (skip windowing-only files) ? most robust; or (b) regenerate the 5 real clips' golden_features through `fusion_golden` so they carry person/audio/shuttle columns (heavier; needs the feature extraction); or (c) guard the litmus to a pinned subset. Prefer (a). Add a regression test for the mixed-schema corpus. Keep it offline/mocked. Verify: full suite + ruff + mypy clean. Branch from origin/master in an isolated worktree, re-verify HEAD before commit, stage explicit paths (shared checkout). Do NOT touch the nightly_regression.py work (separate PR).

2. assistant (2026-07-02T06:11:33.251Z)

I'll start by getting oriented: pull latest, read the relevant code, and check the current-state memory.

3. assistant (2026-07-02T06:11:33.251Z)

[external_agent_tool_call: Bash]
description: Fetch origin and show branch status
command: git fetch origin 2>&1 | head -20 && echo "---STATUS---" && git status -sb && echo "---HEAD---" && git log --oneline -1
[/external_agent_tool_call]

4. assistant (2026-07-02T06:11:33.251Z)

[external_agent_tool_call: Read]

file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_ablation.py
[/external_agent_tool_call]

5. assistant (2026-07-02T06:11:33.251Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

6. assistant (2026-07-02T06:11:33.251Z)

[external_agent_tool_result]
From https://github.com/avidullu/badminton-highlight-indexer
 * [new branch]   claude/indexer-doc-review-XVPTh -> origin/claude/indexer-doc-review-XVPTh
   8736e2d..be39a70 master   -> origin/master
---STATUS---
docs/golden-regression-fixtures-plan...origin/docs/golden-regression-fixtures-plan [ahead 18, behind 1]
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
---HEAD---
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
[/external_agent_tool_result]

7. assistant (2026-07-02T06:11:33.251Z)

[external_agent_tool_result: error]
File content (272.6KB) exceeds maximum allowed size (256KB). Use offset and limit parameters to read specific portions of the file, or search for specific content instead of reading the whole file.
[/external_agent_tool_result]

8. assistant (2026-07-02T06:11:33.251Z)

[external_agent_tool_result]
```
1       """GPU-free tests for the LOSO/LOGO ablation runner (``backend.eval.ablation``).
2
3       Covers, all on deterministic synthetic candidate rows (no GPU, no real video):
4
5         - LOSO / LOGO correctness ? a cleanly-separating signal *earns its keep*, a noise signal is
6           *inert*; LOGO collapses a group's columns; the ablated variant subtracts columns / zeroes
7           the gate knob without mutating ``full``.
8         - the structural-guard exemption ? the SAME signal marked ``structural=True`` is reported
9           (honest ?) but verdict is always ``structural_protected``, never auto-demoted on "no lift".
10        - the honest-stats contract ? every row ships a CI + sign test, and no verdict is stronger
11          than the stats support (the no-over-claim contract).
12        - a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
13        - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the runner
14          reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031). Skips
15          cleanly when the golden JSONs are absent (CI), runs when present (owner box).
16      """
17      from __future__ import annotations
18
19      import glob
20      import json
21      import os
22
```

```
    import pytest

    from backend.eval import ablation
    from backend.eval.ablation import (
        AblationConfig,
        AblationReport,
        Signal,
        ablate_one,
        run_ablation,
    )
    from backend.eval.experiment import Variant, VideoCandidates


    # ---------------------------------------------------------------------------- synthetic
fixtures


    def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
        """``n`` videos with a ``net_crossings`` gate signal.

        ``separating=True``: rallies have net_crossings=5, junk has 0 ? so a
``min_crossings=2`` gate
        perfectly removes the junk and keeping it is load-bearing. ``separating=False``:
every window
        has net_crossings=5, so the gate is a no-op (ablating it changes nothing ?
inert)."""
        videos: list[VideoCandidates] = []
        for k in range(n):
            intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
            junk_val = 5.0 if not separating else 0.0
            feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
            gts = [(0.0, 10.0), (20.0, 30.0)]
            videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
        return videos


    def _learned_videos(n: int = 6) -> list[VideoCandidates]:
        """``n`` videos where a PERSON feature (``players_in_court``) cleanly separates
rallies from
        junk while the shuttle/audio columns are constant (non-separating). So a learned
full-set
        ablation must show person as the load-bearing group and the others as washes ? a
deterministic
        LOGO/LOSO correctness fixture (mirrors ``test_fusion_compare``'s construction)."""
        videos: list[VideoCandidates] = []
        for k in range(n):
            intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
            feats: dict[str, list[float]] = {}
            for fn in ablation.SHUTTLE_COLUMNS:
                feats[fn] = [1.0, 1.0, 1.0, 1.0]                    # constant ? non-
separating
            for fn in ablation.PERSON_COLUMNS:
                feats[fn] = [0.0, 0.0, 0.0, 0.0]
            feats["players_in_court"] = [4.0, 4.0, 0.0, 0.0]         # the separating signal
            feats["in_court_ratio"] = [1.0, 1.0, 0.0, 0.0]
            for fn in ablation.AUDIO_COLUMNS:
                feats[fn] = [0.0, 0.0, 0.0, 0.0]
            gts = [(0.0, 10.0), (20.0, 30.0)]
            videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
        return videos


    # ---------------------------------------------------------------------------- LOSO
```

```
correctness
78
79
80      def test_separating_gate_earns_keep():
81          """A non-structural gate that cleanly separates rallies mus...
[/external_agent_tool_result]
```

9. assistant (2026-07-02T06:11:33.251Z)

Now let me read the ablation module and the current-state memory (with offset since it's large).

10. assistant (2026-07-02T06:11:33.251Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\ablation.py
[/external_agent_tool_call]
```

11. assistant (2026-07-02T06:11:33.251Z)

```
[external_agent_tool_result]
1       """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3       A *systematic, repeatable* version of the one-off ablation
(``scratch/gate_ab_ablation.py``):
4       run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
per
5       signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
bootstrap
6       95% CI + paired sign test + the C4 over-segmentation ratio. See
``scratch/ABLATION_FRAMEWORK_PLAN.md``
7       (the design) and ``docs/ABLATION_LAYER.md`` (the shipped note).
8
9       **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
10      ``experiment.compare`` + ``eval_stats`` engine, exactly as ``compare`` was a driver over
11      ``metrics``/``calibration``. For each signal ``s`` the marginal contribution is:
12
13          ?(s) = score(full set S) ? score(S \\ {s})            # leave-one-out drop, ?F1
primary
14
15      operationally ``experiment.compare(videos, variant_a=without_s, variant_b=full)`` ? A is
the
16      ablated (smaller) variant, B is full, so the reported B?A delta IS ?(s), a per-video-
paired ?F1
17      with bootstrap CI + exact sign test, LOVO-honest for learned variants.
18
19      Two ablation granularities fall out of one inventory:
20
21        - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
22        - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
owner's
23          "is this old SIGNAL still relevant?" question at cue granularity).
24
25      A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
26      a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT
by
27      dropping a column). The runner handles both uniformly: ``without_s`` subtracts the
columns AND/OR
28      zeroes the gate knob from ``cfg.full``.
29
30      Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; §6 of the
plan):
31        - never a bare mean ? every row ships a CI + the exact sign test;
32        - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the
```

```
sign test
33          agreeing** (the ``per_user_gate`` bar, in reverse);
34        - **structural signals are exempt from auto-demotion** ? a guard whose value is in
failure modes
35          the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
is NEVER
36          flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
? honestly
37          but tags ``structural_protected``;
38        - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39          MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41      Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42      writes one ``<name>_golden_features.json`` per video). CLI::
43
44          python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45      """
46      from __future__ import annotations
47
48      import argparse
49      import glob
50      import json
51      import logging
52      import os
53      from dataclasses import dataclass, field
54      from typing import Any, Dict, List, Optional, Sequence, Tuple
55
56      from backend.eval import experiment
57      from backend.eval.experiment import Variant, VideoCandidates
58
59      logger = logging.getLogger(__name__)
60
61      # --- The signal inventory (the things the runner ablates), grouped by producer.
---------
62      # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63      # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
64      # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.
65      SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
66                                          "active_ratio", "net_crossings")
67      PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
"mean_player_motion",
68                     ...
[/external_agent_tool_result]
```

12. assistant (2026-07-02T06:11:33.251Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md
[/external_agent_tool_call]
```

13. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_result]
1       ---
2       name: current-state
3       description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
only on request). What just landed, what's next, open decisions."
4       metadata:
5         node_type: memory
```

```
6         type: project
7         originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8       ---
9
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      **Checkpoint:** 2026-06-18 ~00:10 (? **REMAINING-3 4K REELS DONE ? ALL 6 BOXHILL-17
CLIPS NOW HAVE 4K** + cost-scaling doc + 10/10 visual QA ? all on G:). Owner asked for "just 4K
highlights of the remaining 3, all local no Gemini." Delivered:
13      - ? **GX010138/139/140 4K reels** (the larger 9/10/12.8-min clips): **45/47/68
rallies**, 1.77/1.89/2.72 GB, 3840×2160, watermarked. Published to G: as `? - Highlights
(4K).mp4`. Robust detached Scheduled Task `KhelSutraBoxhill4KRemaining` (keep-awake + self-
restart; unregistered on completion).
14      - ? **DOGFOODED `--detect-from` end-to-end on fresh videos** (the merged #205 feature):
`--video <4K original> --detect-from <1080p proxy>` ? detect on proxy, stitch 4K from original,
reel named after --video (NO `_1080p` naming bug this time). Log confirmed "detecting on proxy ?
stitching reel from ?". Validated beyond the unit tests.
15      - ? **Visual QA workflow** (`wf_2bd49aa1-361`, 10 vision agents): **10/10 delivered
reels clean** ? correct resolution, watermark VISIBLY burned in, real rally content.
(Adversarial verification of the shipped artifacts.)
16      - ? **`COST_SCALING.md` (all 6 clips)** on G:. **Cost model (RTX 2070 Super Max-Q):**
GPU detection (WASB) is the driver at **~3.4× real-time** (?3.4 GPU-min/footage-min, ~0.055
s/frame); NVENC proxy ~0.2× RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ~5× clip length;
peak VRAM ~5.7 GB (fits 8 GB). ? the per-video GPU-busy sampler OVER-counts (background desktop
GPU during the CPU stitch counted as busy) ? use the WASB wall-clock as the clean GPU number.
6-clip totals: 52.2 min footage ? 176 GPU-min WASB + 68 CPU-min stitch, 250 rallies, ~11.6 GB of
4K reels.
17      - ? **OWNER (2026-06-18): will GOLDEN-LABEL the Boxhill-17 clips TODAY.** ? this SETTLES
the open precision question (is local's +50% rally over-detection vs Gemini real rallies Gemini
missed, or false positives?) and graduates these clips into the golden corpus. **When labels
arrive:** at-par each (reuse `scratch/at_par.py` + the proxy video_ids in `output/boxhill17.db`
? local rallies cached: GX010141=22, GX020140=29, GX010137=39, GX010138=45, GX010139=47,
GX010140=68; Gemini for the first 3 in `pr-stitch/output/boxhill17_gemini.db` = 16/16/27) ?
score local & Gemini vs the golden truth (real P/R/F1, not just agreement) ? ingest as golden
videos + update `docs/GOLDEN_VIDEOS.md`. The clips are 1080p proxies in
`output/boxhill17_proxies/` (calibration resolution) ? same timeline as the 4K, so golden labels
apply 1:1. Until then, the local-vs-Gemini numbers are AGREEMENT not accuracy. [[golden-set-
vendoring]] [[eval-dual-set-regression]] [[eval-boundary-tolerance]]
18      - **(superseded by the above)** data-driven triage of the precision question
(`peak_velocity`/`active_frame_count` cut) is now moot once golden labels land. [[gotcha-long-
runs-gpu-wsl]] [[paper-coauthor-aim]]
19
20      **Checkpoint:** 2026-06-17 ~20:55 (? **WATERMARK BUG FIXED #205 (`8736e2d`) + 4K reels +
GEMINI COMPARISON ? all published to G:**; Boxhill spawned-task follow-ups DONE). Owner reviewed
the Boxhill reels and flagged 3 things; all addressed:
21      - ? **#205 MERGED (`8736e2d`, owner-authorized merge):** `fix(runner): honor stitching
watermark/padding + add --detect-from`. **ROOT CAUSE of missing watermarks:**
`tools/generate_highlights.py` built `VideoCompiler(local_work_dir)` BARE ? `watermark=None`
(off) + ctor `end_padding=1.0` instead of `config.json stitching.{watermark,
begin/end_padding=0.5}`. Every other compile site threaded config; the runner didn't. Fix reads
`cfg.get("stitching")` dict-compatibly + passes...
[/external_agent_tool_result]
```

14. assistant (2026-07-02T06:11:33.252Z)

I have a clear picture of the bug and the preferred fix. Let me verify there are no other consumers of this schema before I implement, and check what's actually in `output/` here.

15. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_call: Grep]
input: {"pattern":"load_videos|_golden_features|\\[.shuttle.\\]|\\[.person.\\]|c\\[.audio.\\]","
output_mode":"content","-n":true,"glob":"**/*.py"}
[/external_agent_tool_call]
```

16. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_call: Glob]
input: {"pattern":"output/*_golden_features.json"}
[/external_agent_tool_call]
```

17. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_result]
backend\eval\ablation.py:42:writes one ``<name>_golden_features.json`` per video). CLI::
backend\eval\ablation.py:386:def load_videos(features_dir: str) -> List[VideoCandidates]:
backend\eval\ablation.py:387:    """Load every ``*_golden_features.json`` into a
``VideoCandidates`` (intervals + shuttle +
backend\eval\ablation.py:392:    for path in sorted(glob.glob(os.path.join(features_dir,
"*_golden_features.json"))):
backend\eval\ablation.py:399:            features[fn] = [float(c["shuttle"][fn]) for c in cands]
backend\eval\ablation.py:401:            features[fn] = [float(c["person"][fn]) for c in cands]
backend\eval\ablation.py:574:                    help="dir of *_golden_features.json (the
persisted golden substrate)")
backend\eval\ablation.py:586:    videos = load_videos(args.features_dir)
backend\eval\ablation.py:588:        raise SystemExit(f"need >=2 *_golden_features.json in
{args.features_dir} "
backend\config\models.py:88:    shuttle net-crossing count (Gate A signal) and ? only when
``cue_flags['person']`` is
scripts\nightly_regression.py:24:outside the repo + ``output/<name>_golden_features.json``); the
corpus is NOT committed. So the
scripts\nightly_regression.py:498:            f"and do the
{features_dir}/<name>_golden_features.json files have 'gts'?")
scripts\nightly_regression.py:529:                  help="dir holding
<name>_golden_features.json")
tests\test_ablation.py:13:  - the golden litmus reproduction ? on the 6 real
``output/*_golden_features.json`` the runner
tests\test_ablation.py:156:    assert rows["person"].delta_f1 > 0.0
tests\test_ablation.py:344:def test_load_videos_tolerates_missing_audio(tmp_path):
tests\test_ablation.py:353:    with open(tmp_path / "noaudio_golden_features.json", "w",
encoding="utf-8") as f:
tests\test_ablation.py:355:    videos = ablation.load_videos(str(tmp_path))
tests\test_ablation.py:367:    return len(glob.glob(os.path.join(GOLDEN_DIR,
"*_golden_features.json"))) >= 6
tests\test_ablation.py:371:                    reason="golden output/*_golden_features.json
absent (run fusion_golden first)")
tests\test_ablation.py:377:    videos = ablation.load_videos(GOLDEN_DIR)
tests\test_ablation.py:399:                    reason="golden output/*_golden_features.json
absent")
tests\test_ablation.py:402:    report = run_ablation(ablation.load_videos(GOLDEN_DIR),
ablation.litmus_config())
tests\test_ablation.py:445:        with open(os.path.join(d, f"v{k}_golden_features.json"), "w",
encoding="utf-8") as f:
tests\test_audio_features.py:50:    with open(os.path.join(d, f"{name}_golden_features.json"),
"w", encoding="utf-8") as f:
tests\test_audio_features.py:58:    videos = fusion_audio.load_videos_with_audio(d)
tests\test_audio_features.py:66:def test_load_videos_requires_audio(tmp_path):
tests\test_audio_features.py:73:    with open(os.path.join(str(tmp_path),
"x_golden_features.json"), "w", encoding="utf-8") as f:
tests\test_audio_features.py:76:        fusion_audio.load_videos_with_audio(str(tmp_path))
backend\eval\fusion_audio.py:29:    """Add the 3 audio features to each existing
``<name>_golden_features.json`` window."""
backend\eval\fusion_audio.py:34:        jp = os.path.join(features_dir,
f"{spec['name']}_golden_features.json")
backend\eval\fusion_audio.py:44:            c["audio"] = af.compute_audio_features(hits,
c["start"], c["end"])
backend\eval\fusion_audio.py:52:def load_videos_with_audio(features_dir: str) ->
List[experiment.VideoCandidates]:
backend\eval\fusion_audio.py:56:    for path in sorted(glob.glob(os.path.join(features_dir,
"*_golden_features.json"))):
```

```
backend\eval\fusion_audio.py:65:            features[fn] = [float(c["shuttle"][fn]) for c in
cands]
backend\eval\fusion_audio.py:67:            features[fn] = [float(c["person"][fn]) for c in
cands]
backend\eval\fusion_audio.py:69:            features[fn] = [float(c["audio"][fn]) for c in
cands]
backend\eval\fusion_audio.py:121:        videos = load_videos_with_audio(args.features_dir)
backend\eval\fusion_...
[/external_agent_tool_result]
```

18. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_result]
output\adarsh_avi_singles_golden_features.json
output\kushagra_singles_golden_features.json
output\largetest_doubles_golden_features.json
output\gbaaddy_golden_features.json
output\testlarge_short_golden_features.json
output\mahadevpura_singles_golden_features.json
output\mahadevpura_2_golden_features.json
output\GX010128_golden_features.json
output\mahadevpura_1_golden_features.json
output\GX030094_golden_features.json
output\Badminton_BXH_2_golden_features.json
[/external_agent_tool_result]
```

19. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_call: Grep]
input: {"pattern":"def write_golden_features|_golden_features|candidates.*shuttle|\"shuttle\":|'
shuttle':","output_mode":"files_with_matches"}
[/external_agent_tool_call]
```

20. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_result]
Found 27 files
scripts\nightly_regression.py
eval_baselines\fixtures\occlusion_break\expected.json
eval_baselines\fixtures\multi_rally_with_deadtime\expected.json
eval_baselines\fixtures\clean_single_rally\expected.json
eval_baselines\fixtures\README.md
docs\QUALITY_ITERATIONS.md
docs\GOLDEN_REGRESSION_FIXTURES.md
docs\GOLDEN_VIDEOS.md
docs\GOLDEN_DATA_SHARING.md
docs\CODE_MAP.md
backend\eval\rally_seg_eval.py
backend\eval\golden_fixtures.py
docs\README.md
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\ABLATION_LAYER.md
docs\NEXT_STEPS.md
backend\eval\serve_contrast.py
eval_baselines\experiment_leaderboard.json
tests\test_ablation.py
backend\eval\ablation.py
tests\test_fusion_golden_telemetry.py
backend\eval\fusion_golden.py
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
tests\test_fusion_compare.py
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
tests\test_audio_features.py
```

[/external_agent_tool_result]

21. assistant (2026-07-02T06:11:33.252Z)

`output/` here is fully populated with all 11 files ? including the 5 leaner ones. So I can **reproduce and verify locally**. Let me confirm the two schemas and check whether the test suite runs in this container.

22. assistant (2026-07-02T06:11:33.252Z)

[external_agent_tool_call: Bash]
description: Inspect golden_features schemas across the corpus
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== adarsh_avi_singles (full fusion) ? first candidate keys ==="
python -c "import json;d=json.load(open('output/adarsh_avi_singles_golden_features.json'));print(sorted(d['candidates'][0].keys()))"
echo "=== mahadevpura_1 (leaner) ? first candidate keys ==="
python -c "import json;d=json.load(open('output/mahadevpura_1_golden_features.json'));print(sorted(d['candidates'][0].keys()))"
echo "=== schema per file (has shuttle?) ==="
python -c "
import json,glob,os
for p in sorted(glob.glob('output/*_golden_features.json')):
    d=json.load(open(p))
    c=d['candidates'][0] if d['candidates'] else {}
    print(('FUSION' if 'shuttle' in c and 'person' in c else 'LEANER'), os.path.basename(p), '| ncands=', len(d['candidates']))
"
[/external_agent_tool_call]

23. assistant (2026-07-02T06:11:33.252Z)

[external_agent_tool_result]
=== adarsh_avi_singles (full fusion) ? first candidate keys ===
['audio', 'end', 'label', 'person', 'shuttle', 'start']
=== mahadevpura_1 (leaner) ? first candidate keys ===
['active_frame_count', 'chunk_index', 'end', 'mean_velocity', 'peak_velocity', 'start', 'track_id_count']
=== schema per file (has shuttle?) ===
LEANER Badminton_BXH_2_golden_features.json | ncands= 4
LEANER GX010128_golden_features.json | ncands= 15
LEANER GX030094_golden_features.json | ncands= 30
FUSION adarsh_avi_singles_golden_features.json | ncands= 172
FUSION gbaaddy_golden_features.json | ncands= 82
FUSION kushagra_singles_golden_features.json | ncands= 20
FUSION largetest_doubles_golden_features.json | ncands= 86
LEANER mahadevpura_1_golden_features.json | ncands= 1
LEANER mahadevpura_2_golden_features.json | ncands= 5
FUSION mahadevpura_singles_golden_features.json | ncands= 57
FUSION testlarge_short_golden_features.json | ncands= 14
[/external_agent_tool_result]

24. assistant (2026-07-02T06:11:33.252Z)

[external_agent_tool_call: Bash]
description: Check test deps available in container
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
python -c "import numpy; print('numpy', numpy.__version__)" 2>&1 | head -1
python -c "import scipy; print('scipy', scipy.__version__)" 2>&1 | head -1
python -c "import pytest; print('pytest', pytest.__version__)" 2>&1 | head -1
python -c "from backend.eval import ablation; print('ablation import OK')" 2>&1 | tail -3
[/external_agent_tool_call]

25. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_result]
numpy 2.4.6
scipy 1.17.1
pytest 9.0.3
ablation import OK
[/external_agent_tool_result]
```

26. assistant (2026-07-02T06:11:33.252Z)

```
Confirmed: exactly 6 FUSION files + 5 LEANER files, and numpy/scipy/pytest are available ? so I
can fully reproduce and verify. Let me reproduce the failure first.
```

27. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_call: Bash]
description: Reproduce the two failing litmus tests
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
python -m pytest tests/test_ablation.py::test_litmus_reproduces_gate_a
tests/test_ablation.py::test_litmus_report_is_well_formed -x -q 2>&1 | tail -25
[/external_agent_tool_call]
```

28. assistant (2026-07-02T06:11:33.252Z)

```
[external_agent_tool_result]
features_dir = 'C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\output'

    def load_videos(features_dir: str) -> List[VideoCandidates]:
        """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle
+
        person + audio columns + golden gts). Audio columns are optional (older JSONs predate
the
        audio augment) ? a missing block defaults those columns to 0.0 so the loader never
crashes,
        mirroring ``experiment._feature_column``'s tolerance."""
        videos: List[VideoCandidates] = []
        for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
            with open(path, encoding="utf-8") as f:
                d = json.load(f)
            cands = d["candidates"]
            intervals = [(float(c["start"]), float(c["end"])) for c in cands]
            features: Dict[str, List[float]] = {}
            for fn in SHUTTLE_COLUMNS:
>               features[fn] = [float(c["shuttle"][fn]) for c in cands]
                                      ^^^^^^^^^^^^
E               KeyError: 'shuttle'

backend\eval\ablation.py:399: KeyError
========================== short test summary info ==========================
FAILED tests/test_ablation.py::test_litmus_reproduces_gate_a - KeyError: 'shu...
!!!!!!!!!!!!!!!!!!!!!!!!! stopping after 1 failures !!!!!!!!!!!!!!!!!!!!!!!!!
1 failed in 2.29s
[/external_agent_tool_result]
```

29. assistant (2026-07-02T06:11:33.252Z)

```
Reproduced exactly (`KeyError: 'shuttle'`). Now let me set up an isolated worktree from
`origin/master` and implement the fix there.
```

30. assistant (2026-07-02T06:11:33.253Z)

```
[external_agent_tool_call: Bash]
description: Create isolated worktree from origin/master
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== origin/master HEAD ==="
```

```
git log --oneline -1 origin/master
echo "=== creating worktree ==="
git worktree add -b fix/ablation-litmus-mixed-schema .claude/worktrees/ablation-schema-fix
origin/master 2>&1 | tail -5
echo "=== verify worktree HEAD ==="
git -C .claude/worktrees/ablation-schema-fix log --oneline -1
git -C .claude/worktrees/ablation-schema-fix branch --show-current
[/external_agent_tool_call]
```

31. assistant (2026-07-02T06:11:33.253Z)

```
[external_agent_tool_result]
=== origin/master HEAD ===
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
=== creating worktree ===
Preparing worktree (new branch 'fix/ablation-litmus-mixed-schema')
branch 'fix/ablation-litmus-mixed-schema' set up to track 'origin/master'.
HEAD is now at be39a70 docs: rally-detection quality report ? technical-paper checkpoint
(2026-06-17) (#206)
=== verify worktree HEAD ===
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
fix/ablation-litmus-mixed-schema
[/external_agent_tool_result]
```

32. assistant (2026-07-02T06:11:33.253Z)

```
Worktree is on `origin/master` (be39a70). The main checkout was on a different branch (ahead
18), so let me confirm the worktree's versions of these files match what I analyzed before
editing.
```

33. assistant (2026-07-02T06:11:33.253Z)

```
[external_agent_tool_call: Bash]
description: Confirm worktree files match analyzed versions
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== diff ablation.py (worktree-master vs my-checkout) ==="
git diff origin/master -- backend/eval/ablation.py | head -5
echo "exit: $?  (empty == identical)"
echo "=== diff test_ablation.py ==="
git diff origin/master -- tests/test_ablation.py | head -5
echo "=== confirm load_videos region in worktree ==="
sed -n '386,408p' .claude/worktrees/ablation-schema-fix/backend/eval/ablation.py
[/external_agent_tool_call]
```

34. assistant (2026-07-02T06:11:33.253Z)

```
[external_agent_tool_result]
=== diff ablation.py (worktree-master vs my-checkout) ===
exit: 0  (empty == identical)
=== diff test_ablation.py ===
=== confirm load_videos region in worktree ===
def load_videos(features_dir: str) -> List[VideoCandidates]:
    """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
    person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
    audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
    mirroring ``experiment._feature_column``'s tolerance."""
    videos: List[VideoCandidates] = []
    for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
        with open(path, encoding="utf-8") as f:
            d = json.load(f)
        cands = d["candidates"]
        intervals = [(float(c["start"]), float(c["end"])) for c in cands]
        features: Dict[str, List[float]] = {}
        for fn in SHUTTLE_COLUMNS:
```

```
            features[fn] = [float(c["shuttle"][fn]) for c in cands]
        for fn in PERSON_COLUMNS:
            features[fn] = [float(c["person"][fn]) for c in cands]
        for fn in AUDIO_COLUMNS:
            features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
        gts = [(float(a), float(b)) for a, b in d["gts"]]
        videos.append(VideoCandidates(name=d["name"], intervals=intervals,
                                      features=features, gts=gts))
    return videos
[/external_agent_tool_result]
```

35. assistant (2026-07-02T06:11:33.253Z)

Files match master exactly. Now I'll implement the fix in the worktree. Let me read the worktree copies (Edit requires an in-conversation Read of the exact path).

36. assistant (2026-07-02T06:11:33.253Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\backend\eval\ablation.py
[/external_agent_tool_call]
```

37. assistant (2026-07-02T06:11:33.253Z)

```
[external_agent_tool_result]
383     # ---------------------------------------------------------------------------- corpus I/O
384
385
386     def load_videos(features_dir: str) -> List[VideoCandidates]:
387         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
388         person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
389         audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
390         mirroring ``experiment._feature_column``'s tolerance."""
391         videos: List[VideoCandidates] = []
392         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
393             with open(path, encoding="utf-8") as f:
394                 d = json.load(f)
395             cands = d["candidates"]
396             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
397             features: Dict[str, List[float]] = {}
398             for fn in SHUTTLE_COLUMNS:
399                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
400             for fn in PERSON_COLUMNS:
401                 features[fn] = [float(c["person"][fn]) for c in cands]
402             for fn in AUDIO_COLUMNS:
403                 features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
404             gts = [(float(a), float(b)) for a, b in d["gts"]]
405             videos.append(VideoCandidates(name=d["name"], intervals=intervals,
406                                           features=features, gts=gts))
407         return videos
[/external_agent_tool_result]
```

38. assistant (2026-07-02T06:11:33.253Z)

Now I'll implement the fix. First, the loader in `ablation.py`:

39. assistant (2026-07-02T06:11:33.253Z)

```
[external_agent_tool_call: Edit]
```

file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\backend\eval\ablation.py
[/external_agent_tool_call]

40. assistant (2026-07-02T06:11:33.253Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\backend\eval\ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

41. assistant (2026-07-02T06:11:33.253Z)

Now the test file. Let me read the worktree copy's relevant regions, then update `_golden_present()` and add the regression tests + helper.

42. assistant (2026-07-02T06:11:33.253Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\tests\test_ablation.py
[/external_agent_tool_call]

43. assistant (2026-07-02T06:11:33.253Z)

[external_agent_tool_result]
```
340
341     # ------------------------------------------------------------------------ corpus loader (no GPU)
342
343
344     def test_load_videos_tolerates_missing_audio(tmp_path):
345         """An older feature JSON without an ``audio`` block still loads ? audio columns default to
346         0.0 (loader never crashes on a pre-audio JSON)."""
347         cand = {"start": 0.0, "end": 10.0,
348                 "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
349                 "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
350                 "label": 1}
351         data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
352                 "candidates": [cand], "gts": [[0.0, 10.0]]}
353         with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
354             json.dump(data, f)
355         videos = ablation.load_videos(str(tmp_path))
356         assert len(videos) == 1
357         assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not crashed
358
359
360     # ------------------------------------------------------------------------ the golden litmus
361
362     GOLDEN_DIR = os.path.join(
363         os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "output")
364
365
366     def _golden_present() -> bool:
367         return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
368
369
```
[/external_agent_tool_result]

44. assistant (2026-07-02T06:11:33.253Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\tests\test_ablation.py
[/external_agent_tool_call]

45. assistant (2026-07-02T06:11:33.253Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\tests\test_ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

46. assistant (2026-07-02T06:11:33.254Z)

Now update `_golden_present()` so the litmus skip-gate counts only ablatable fusion files (keeping the gate consistent with the loader):

47. assistant (2026-07-02T06:11:33.254Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\tests\test_ablation.py
[/external_agent_tool_call]

48. assistant (2026-07-02T06:11:33.254Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix\tests\test_ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

49. assistant (2026-07-02T06:11:33.254Z)

The worktree has no `output/` dir, so the litmus would skip there. To faithfully reproduce the owner's box (mixed corpus present), let me copy the 11 golden files into the worktree's `output/` and confirm `output/` is gitignored so they won't be staged.

50. assistant (2026-07-02T06:11:33.254Z)

[external_agent_tool_call: Bash]
description: Copy golden files into worktree output and confirm ignored
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== is output/ gitignored? ==="
git -C .claude/worktrees/ablation-schema-fix check-ignore output/ && echo "(output/ IS ignored)"
echo "=== copy the 11 golden_features.json into the worktree output/ ==="
mkdir -p .claude/worktrees/ablation-schema-fix/output
cp output/*_golden_features.json .claude/worktrees/ablation-schema-fix/output/
ls .claude/worktrees/ablation-schema-fix/output/ | wc -l
echo "=== worktree git status (should show NO output files staged/untracked) ==="
git -C .claude/worktrees/ablation-schema-fix status --porcelain
[/external_agent_tool_call]

51. assistant (2026-07-02T06:11:33.254Z)

[external_agent_tool_result]
=== is output/ gitignored? ===
output/
(output/ IS ignored)
=== copy the 11 golden_features.json into the worktree output/ ===
11
=== worktree git status (should show NO output files staged/untracked) ===
 M backend/eval/ablation.py

```
 M tests/test_ablation.py
[/external_agent_tool_result]
```

52. assistant (2026-07-02T06:11:33.254Z)

`output/` is ignored (only my 2 source edits show as modified). Now run the targeted tests in the worktree:

53. assistant (2026-07-02T06:11:33.254Z)

```
[external_agent_tool_call: Bash]
description: Run litmus + new regression tests in worktree
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
echo "=== the two formerly-failing litmus tests + the new regression tests ==="
python -m pytest tests/test_ablation.py::test_litmus_reproduces_gate_a
tests/test_ablation.py::test_litmus_report_is_well_formed
tests/test_ablation.py::test_is_fusion_schema_distinguishes_lean_and_full
tests/test_ablation.py::test_load_videos_skips_windowing_only_schema -v 2>&1 | tail -20
[/external_agent_tool_call]
```

54. assistant (2026-07-02T06:11:33.254Z)

```
[external_agent_tool_result]
=== the two formerly-failing litmus tests + the new regression tests ===
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-schema-fix
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 4 items

tests/test_ablation.py::test_litmus_reproduces_gate_a PASSED             [ 25%]
tests/test_ablation.py::test_litmus_report_is_well_formed PASSED         [ 50%]
tests/test_ablation.py::test_is_fusion_schema_distinguishes_lean_and_full PASSED [ 75%]
tests/test_ablation.py::test_load_videos_skips_windowing_only_schema PASSED [100%]

============================== 4 passed in 2.30s ==============================
[/external_agent_tool_result]
```

55. assistant (2026-07-02T06:11:33.254Z)

All 4 green ? and the litmus reproduces the exact Gate-A numbers (?F1?0.074, 6/6, p?0.031), proving the filter picks exactly the right 6 fusion files. Now the full ablation file, then the whole suite + ruff + mypy.

56. assistant (2026-07-02T06:11:33.254Z)

```
[external_agent_tool_call: Bash]
description: Run full test_ablation.py in worktree
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
python -m pytest tests/test_ablation.py -q 2>&1 | tail -8
[/external_agent_tool_call]
```

57. assistant (2026-07-02T06:11:33.254Z)

```
[external_agent_tool_call: Bash]
description: Run ruff + mypy on changed files
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
```

```
echo "=== ruff (changed files) ==="
python -m ruff check backend/eval/ablation.py tests/test_ablation.py 2>&1 | tail -15
echo "=== mypy ablation.py ==="
python -m mypy backend/eval/ablation.py 2>&1 | tail -15
[/external_agent_tool_call]
```

58. assistant (2026-07-02T06:11:33.254Z)

```
[external_agent_tool_result]
.............................                                            [100%]
29 passed in 18.72s
[/external_agent_tool_result]
```

59. assistant (2026-07-02T06:11:33.254Z)

```
[external_agent_tool_result]
=== ruff (changed files) ===
All checks passed!
=== mypy ablation.py ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 1 source file
[/external_agent_tool_result]
```

60. assistant (2026-07-02T06:11:33.254Z)

```
29/29 in the ablation file, ruff clean, mypy clean. Now the full suite (owner's pre-LGTM gate).
Let me check how `run_all_tests.py` runs and execute it ? the worktree's `output/` now mirrors
the owner's box, so the litmus runs for real here.
```

61. assistant (2026-07-02T06:11:33.254Z)

```
[external_agent_tool_call: Bash]
description: Inspect run_all_tests.py harness
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-
schema-fix
head -40 run_all_tests.py 2>&1 | tail -40
[/external_agent_tool_call]
```

62. assistant (2026-07-02T06:11:33.254Z)

```
[external_agent_tool_result]
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the
whole suite before opening a PR:

    python run_all_tests.py             # full suite, quiet
    python run_all_tests.py -v          # verbose (passthrough to pytest)
    python run_all_tests.py -k config   # filter by keyword (passthrough)
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage

Anything after the script name is passed straight through to pytest.

Notes
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
```

```
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""
from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
[/external_agent_tool_result]
```

63. assistant (2026-07-02T06:11:33.255Z)

```
[external_agent_tool_call: Bash]
description: Run full test suite in worktree
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-
schema-fix
python run_all_tests.py 2>&1 | tail -20
[/external_agent_tool_call]
```

64. assistant (2026-07-02T06:11:33.255Z)

```
[external_agent_tool_result]
======================================================================
 Full test suite:  C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
 obsreport (optional reporting dep): available
======================================================================
........................................................................ [  8%]
........................................................................ [ 16%]
........................................................................ [ 25%]
........................................................................ [ 33%]
........................................................................ [ 42%]
........................................................................ [ 50%]
........................................................................ [ 59%]
........................................................................ [ 67%]
........................................................................sss [ 75%]
ss...................................................................... [ 84%]
........................................................................ [ 92%]
........................................................               [100%]
848 passed, 5 skipped in 70.47s (0:01:10)
======================================================================
 RESULT: PASS - all tests green
======================================================================
[/external_agent_tool_result]
```

65. assistant (2026-07-02T06:11:33.255Z)

```
Full suite: **848 passed, 5 skipped** (the litmus ran for real here and passed). Now let me
match CI exactly for repo-wide ruff + mypy and the coverage floor. Let me find the CI
invocations.
```

66. assistant (2026-07-02T06:11:33.255Z)

```
[external_agent_tool_call: Bash]
description: Locate CI ruff/mypy/coverage commands
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-
schema-fix
echo "=== CI config files ==="
ls .github/workflows/ 2>/dev/null
echo "--- grep CI commands (ruff/mypy/cov-fail-under) ---"
grep -rEn "ruff check|mypy |cov-fail-under|--cov" .github/ pyproject.toml setup.cfg pytest.ini
tox.ini Makefile 2>/dev/null | head -30
[/external_agent_tool_call]
```

67. assistant (2026-07-02T06:11:33.255Z)

```
[external_agent_tool_result]
=== CI config files ===
ci.yml
--- grep CI commands (ruff/mypy/cov-fail-under) ---
.github/workflows/ci.yml:31:        run: ruff check . --output-format github
.github/workflows/ci.yml:35:  # up. Typed core deps are installed so mypy sees their real
signatures
.github/workflows/ci.yml:45:      - name: Install mypy + typed core deps
.github/workflows/ci.yml:46:        run: python -m pip install mypy pydantic fastapi uvicorn
python-dotenv google-genai numpy
.github/workflows/ci.yml:48:        run: mypy backend training
.github/workflows/ci.yml:97:        run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
[/external_agent_tool_result]
```

68. assistant (2026-07-02T06:11:33.255Z)

```
CI runs `ruff check .`, `mypy backend training`, and the suite with `--cov-fail-under=70`. Let
me run all three to match.
```

69. assistant (2026-07-02T06:11:33.255Z)

```
[external_agent_tool_call: Bash]
description: Repo-wide ruff and mypy as CI
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-
schema-fix
echo "=== ruff check . ==="
python -m ruff check . 2>&1 | tail -8
echo "=== mypy backend training ==="
python -m mypy backend training 2>&1 | tail -8
[/external_agent_tool_call]
```

70. assistant (2026-07-02T06:11:33.255Z)

```
[external_agent_tool_result]
=== ruff check . ===
All checks passed!
=== mypy backend training ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 104 source files
[/external_agent_tool_result]
```

71. assistant (2026-07-02T06:11:33.255Z)

ruff clean, mypy clean (104 files). Now the coverage gate (`--cov-fail-under=70`):

72. assistant (2026-07-02T06:11:33.255Z)

[external_agent_tool_call: Bash]
description: Run suite with coverage gate
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70 2>&1 | tail -12
[/external_agent_tool_call]

73. assistant (2026-07-02T06:11:33.255Z)

[external_agent_tool_result]
```
backend\utils\run_telemetry.py                       126     12    90%
backend\utils\video.py                                25      3    88%
training\__init__.py                                   0      0   100%
training\gen0\__init__.py                              0      0   100%
training\gen0\harness.py                             108     11    90%
------------------------------------------------------------------------
TOTAL                                               7985   1544    81%
Required test coverage of 70% reached. Total coverage: 80.66%
848 passed, 5 skipped, 7 warnings in 107.27s (0:01:47)
========================================================================
 RESULT: PASS - all tests green
========================================================================
```
[/external_agent_tool_result]

74. assistant (2026-07-02T06:11:33.255Z)

Coverage **80.66%** (floor 70, matches the ~80.6% baseline ? no regression). Let me confirm `ablation.py`'s own coverage didn't slip:

75. assistant (2026-07-02T06:11:33.255Z)

[external_agent_tool_call: Bash]
description: Check ablation.py line coverage
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
python -m pytest tests/test_ablation.py -q --cov=backend.eval.ablation --cov-report=term-missing 2>&1 | grep -E "ablation.py|TOTAL"
[/external_agent_tool_call]

76. assistant (2026-07-02T06:11:33.255Z)

[external_agent_tool_result]
```
backend\eval\ablation.py     257      1    99%   632
TOTAL                        257      1    99%
```
[/external_agent_tool_result]

77. assistant (2026-07-02T06:11:33.255Z)

ablation.py at **99%** (only the `__main__` guard line is uncovered) ? new code fully exercised. Let me review the complete diff before committing.

78. assistant (2026-07-02T06:11:33.255Z)

[external_agent_tool_call: Bash]
description: Review full diff
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix

```
git diff -- backend/eval/ablation.py tests/test_ablation.py
[/external_agent_tool_call]
```

79. assistant (2026-07-02T06:11:33.255Z)

```
[external_agent_tool_result]
diff --git a/backend/eval/ablation.py b/backend/eval/ablation.py
index fec3e7d..968eefd 100644
--- a/backend/eval/ablation.py
+++ b/backend/eval/ablation.py
@@ -383,16 +383,37 @@ def litmus_config() -> AblationConfig:
 # ---------------------------------------------------------------------------- corpus I/O


+def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
+    """True iff every candidate row carries the per-cue feature dicts the ablation scorer reads
+    (``shuttle`` + ``person``; ``audio`` stays optional ? older JSONs predate it).
+
+    The velocity-windowing / distill path persists a LEANER ``*_golden_features.json`` row
+    (``start``/``end`` + velocity stats, no cue dicts) for clips that never went through fusion
+    feature extraction. ``load_videos`` SKIPS those rather than crash on ``c["shuttle"]`` ? so
a
+    mixed corpus (some fusion files, some windowing-only) loads the ablatable subset cleanly.
+    An empty candidate list is not ablatable either."""
+    return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
+
+
 def load_videos(features_dir: str) -> List[VideoCandidates]:
-    """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
-    person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
-    audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
-    mirroring ``experiment._feature_column``'s tolerance."""
+    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
+    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
+    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
+    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
+
+    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
+    R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
+    ablatable subset cleanly (see ``_is_fusion_schema``)."""
     videos: List[VideoCandidates] = []
     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
         with open(path, encoding="utf-8") as f:
             d = json.load(f)
         cands = d["candidates"]
+        if not _is_fusion_schema(cands):
+            logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
+                        "(no shuttle/person cue dicts), not an ablatable fusion-feature file",
+                        os.path.basename(path))
+            continue
         intervals = [(float(c["start"]), float(c["end"])) for c in cands]
         features: Dict[str, List[float]] = {}
         for fn in SHUTTLE_COLUMNS:
diff --git a/tests/test_ablation.py b/tests/test_ablation.py
index b29ecbb..224d497 100644
--- a/tests/test_ablation.py
+++ b/tests/test_ablation.py
@@ -357,6 +357,48 @@ def test_load_videos_tolerates_missing_audio(tmp_path):
     assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not crashed


+def _write_windowing_only(d: str, name: str) -> None:
+    """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema the
+    R3/R4 corpus-growth path persists for clips that never went through fusion feature
+    extraction (``start``/``end`` + velocity stats, NO shuttle/person/audio cue dicts). The
```

```
+    ablation loader must SKIP these, never crash on ``c["shuttle"]``."""
+    cands = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
+              "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
+    data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
+            "candidates": cands, "gts": [[0.0, 5.0]]}
+    with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
+        json.dump(data, f)
+
+
+def test_is_fusion_schema_disti...
[/external_agent_tool_result]
```

80. assistant (2026-07-02T06:11:33.255Z)

```
Diff is clean and fully covered. Now I'll commit per the shared-checkout protocol: re-verify
HEAD, stage explicit paths only, commit.
```

81. assistant (2026-07-02T06:11:33.255Z)

```
[external_agent_tool_call: Bash]
description: Re-verify HEAD and stage explicit paths
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-
schema-fix
echo "=== re-verify branch + HEAD before staging ==="
git branch --show-current
git log --oneline -1
echo "=== stage explicit paths only ==="
git add backend/eval/ablation.py tests/test_ablation.py
echo "=== staged (should be exactly 2 source files, NO output/) ==="
git status --porcelain
[/external_agent_tool_call]
```

82. assistant (2026-07-02T06:11:33.255Z)

```
[external_agent_tool_result]
=== re-verify branch + HEAD before staging ===
fix/ablation-litmus-mixed-schema
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
=== stage explicit paths only ===
=== staged (should be exactly 2 source files, NO output/) ===
M  backend/eval/ablation.py
M  tests/test_ablation.py
[/external_agent_tool_result]
```

83. assistant (2026-07-02T06:11:33.256Z)

```
Exactly 2 files staged on my branch atop `origin/master`. Committing:
```

84. assistant (2026-07-02T06:11:33.256Z)

```
[external_agent_tool_call: Bash]
description: Commit the fix and verify clean branch
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-
schema-fix
git commit -F - <<'EOF'
fix(eval): ablation litmus tolerates a mixed-schema golden corpus

`ablation.load_videos()` read `c["shuttle"]`/`c["person"]` unconditionally, so
once the R3/R4 corpus growth added clips persisted via the velocity-windowing /
distill path (leaner rows: start/end + velocity stats, no per-cue dicts), the
two litmus tests crashed with `KeyError: 'shuttle'` whenever the full n=11
corpus was present in `output/`. CI was unaffected (empty `output/` ? skipped);
the owner box saw 2 reds.
```

```
Fix (most-robust, option a): filter the loader to FULL-FUSION-SCHEMA files only.
- `_is_fusion_schema()` ? a file is ablatable iff every candidate carries the
  `shuttle`+`person` cue dicts (`audio` stays optional, preserving the pre-audio
  tolerance). `load_videos()` skips leaner windowing-only files with an info log
  instead of crashing, so a mixed corpus loads the ablatable subset cleanly.
- `_golden_present()` now counts only fusion-schema files, so the litmus
  skip-gate matches the loader (won't fire on an un-ablatable corpus).

The litmus still reproduces the recorded Gate-A result (?F1?0.074, 6/6, p?0.031)
because the 6 fusion-schema files are exactly the original n=6 corpus.

Regression: `test_load_videos_skips_windowing_only_schema` (mixed corpus loads
only the fusion files + the litmus path runs end-to-end) and
`test_is_fusion_schema_distinguishes_lean_and_full`. Offline/mocked.

Verified locally (output/ mirroring the owner box): full suite 848 passed /
5 skipped, ruff clean, mypy clean (104 files), coverage 80.66% (floor 70).
Does not touch the nightly_regression.py work.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
echo "=== verify only my commit sits atop origin/master ==="
git log --oneline origin/master..HEAD
echo "=== diffstat ==="
git diff --stat origin/master..HEAD
[/external_agent_tool_call]
```

85. assistant (2026-07-02T06:11:33.256Z)

```
[external_agent_tool_result]
[fix/ablation-litmus-mixed-schema 59ae347] fix(eval): ablation litmus tolerates a mixed-schema
golden corpus
 2 files changed, 81 insertions(+), 5 deletions(-)
=== verify only my commit sits atop origin/master ===
59ae347 fix(eval): ablation litmus tolerates a mixed-schema golden corpus
=== diffstat ===
 backend/eval/ablation.py | 29 ++++++++++++++++++++----
 tests/test_ablation.py   | 57 ++++++++++++++++++++++++++++++++++++++++++++++++-
 2 files changed, 81 insertions(+), 5 deletions(-)
[/external_agent_tool_result]
```

86. assistant (2026-07-02T06:11:33.256Z)

```
Committed cleanly ? exactly one commit atop `origin/master`, 2 files.

While reading, I found the **identical latent bug** in two sibling loaders that aren't currently
failing any test (their tests use synthetic tmp data, not the real `output/`), so they're out of
scope for this focused PR but would crash the same way on the n=11 corpus. Let me flag that as a
follow-up, then update the live checkpoint.
```

87. assistant (2026-07-02T06:11:33.256Z)

```
[external_agent_tool_call: mcp__ccd_session__spawn_task]
input: {"title":"Harden fusion_compare/fusion_audio loaders for mixed schema","prompt":"In the
badminton-highlight-indexer repo, two golden-feature loaders have the SAME latent bug just fixed
in `backend/eval/ablation.py::load_videos` (commit on branch `fix/ablation-litmus-mixed-
schema`): they read `c[\"shuttle\"]`/`c[\"person\"]` unconditionally on every candidate, so they
crash with `KeyError: 'shuttle'` on a MIXED golden corpus where some
`output/*_golden_features.json` files carry the leaner velocity-windowing/distill schema (keys:
active_frame_count, chunk_index, end, mean_velocity, peak_velocity, start, track_id_count ? no
shuttle/person/audio cue dicts).\n\nAffected:\n- `backend/eval/fusion_compare.py::load_videos`
(~line 39-48) ? `python -m backend.eval.fusion_compare --features-dir output` would KeyError on
the n=11 corpus.\n- `backend/eval/fusion_audio.py::load_videos_with_audio` (~line 56-69) ? same,
plus it reads `c[\"audio\"]` unconditionally too.\n\nUnlike the ablation litmus, these are NOT
```

caught by any test today because `tests/test_fusion_compare.py` and
`tests/test_audio_features.py` only exercise synthetic tmp_path corpora, never the real
`output/`. So this is latent, not currently red ? but it bites the moment someone runs those
CLIs on the real mixed corpus.\n\nFix: mirror the ablation fix ? reuse/duplicate the
`_is_fusion_schema(candidates)` predicate (full-fusion iff every candidate has
`shuttle`+`person`; audio optional) and skip leaner windowing-only files with an info log
instead of crashing. Consider whether `_is_fusion_schema` should move to a shared spot (e.g. it
currently lives in `backend/eval/ablation.py`) vs. being duplicated ? keep it simple, DRY only
if it doesn't create an awkward import. Add a regression test per loader (mixed corpus loads
only the fusion files). Keep it offline/mocked.\n\nVerify: full suite (`python
run_all_tests.py`), `ruff check .`, `mypy backend training` all clean; coverage stays ?70
(currently ~80.7%). Branch f...
[/external_agent_tool_call]

88. assistant (2026-07-02T06:11:33.256Z)

[external_agent_tool_result]
Noted (position 1, task_id: task_8dd4efde). A chip is showing for the user ? they can start it
in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or
superseded, call dismiss_task with this task_id. Currently pending: task_8dd4efde "Harden
fusion_compare/fusion_audio loaders for mixed schema". Continue your current work.
[/external_agent_tool_result]

89. assistant (2026-07-02T06:11:33.256Z)

Now let me update the live checkpoint to record that this previously out-of-scope item is fixed.

90. assistant (2026-07-02T06:11:33.256Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md
[/external_agent_tool_call]

91. assistant (2026-07-02T06:11:33.256Z)

[external_agent_tool_result]
10   # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12   **Checkpoint:** 2026-06-18 ~00:10 (? **REMAINING-3 4K REELS DONE ? ALL 6 BOXHILL-17
CLIPS NOW HAVE 4K** + cost-scaling doc + 10/10 visual QA ? all on G:). Owner asked for "just 4K
highlights of the remaining 3, all local no Gemini." Delivered:
[/external_agent_tool_result]

92. assistant (2026-07-02T06:11:33.256Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md
[/external_agent_tool_call]

93. assistant (2026-07-02T06:11:33.256Z)

[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
[/external_agent_tool_result]

94. assistant (2026-07-02T06:11:33.256Z)

Done. The two failing litmus tests are fixed, committed, and fully verified locally.

What was wrong

`ablation.load_videos()` read `c["shuttle"]`/`c["person"]` on every candidate. The 5 newer R3/R4 clips (`Badminton_BXH_2`, `GX010128`, `GX030094`, `mahadevpura_1`, `mahadevpura_2`) are persisted via the velocity-windowing/distill path with a leaner schema (no cue dicts), so the loader threw `KeyError: 'shuttle'` whenever the full n=11 corpus was in `output/`. I reproduced it here exactly (this box had all 11 files).

The fix (option a ? most robust)

backend/eval/ablation.py:
- **`_is_fusion_schema()`** ? a file is ablatable iff every candidate carries `shuttle`+`person` (audio stays optional, preserving the existing pre-audio tolerance).
- **`load_videos()`** skips leaner windowing-only files with an info log instead of crashing ? a mixed corpus loads only the ablatable subset.
- **`_golden_present()`** (test gate) now counts only fusion-schema files, so the litmus skip-gate matches the loader (it won't fire the litmus on an un-ablatable corpus).

The 6 fusion-schema files are exactly the original n=6, so the litmus still reproduces the recorded Gate-A result (?F1?0.074, 6/6, p?0.031) ? those assertions pass against real data, not just "no crash."

Regression coverage

`test_load_videos_skips_windowing_only_schema` (mixed corpus ? only fusion files load + litmus runs end-to-end) and `test_is_fusion_schema_distinguishes_lean_and_full`. Offline/mocked.

Verification (with `output/` mirroring the owner box, so the litmus ran for real)

- Full suite: **848 passed, 5 skipped**
- `ruff check .`: clean · `mypy backend training`: clean (104 files)
- Coverage **80.66%** (floor 70, no regression); `ablation.py` itself at 99%

State

- Branch `fix/ablation-litmus-mixed-schema` @ `59ae347`, **one commit** atop `origin/master` (`be39a70`), in worktree `.claude/worktrees/ablation-schema-fix`. Staged explicit paths only; the golden JSONs I copied into the worktree's `output/` are gitignored and not committed.
- **Not pushed / no PR** ? per CLAUDE.md "don't open a PR unless asked." Say the word and I'll push, open the PR, and merge it (you've durably authorized merging good-engineering fixes on green CI).
- Did **not** touch `nightly_regression.py`.

One follow-up flagged (chip `task_8dd4efde`)

`fusion_compare.load_videos` and `fusion_audio.load_videos_with_audio` have the **identical** unconditional `c["shuttle"]` access. They aren't red today (their tests only use synthetic tmp corpora), but the same CLIs would crash on the real mixed corpus. Kept out of this focused PR per the one-PR-per-item convention.

95. user (2026-07-02T06:11:33.256Z)

please report this as a github issue, attach the PR and merge it ensuring the due diligence in the repo.

96. assistant (2026-07-02T06:11:33.256Z)

The user has now explicitly authorized opening a PR and merging ? with due diligence. I'll: push the branch, file a GitHub issue, open a PR linking it, run an adversarial review + let CI run, and merge only on green. Let me start with environment checks.

97. assistant (2026-07-02T06:11:33.256Z)

[external_agent_tool_call: Bash]
description: Check gh auth, remote, CI triggers
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
echo "=== gh auth ==="

```
gh auth status 2>&1 | head -6
echo "=== remote ==="
git remote -v | head -2
echo "=== ci.yml triggers ==="
sed -n '1,20p' .github/workflows/ci.yml
[/external_agent_tool_call]
```

98. assistant (2026-07-02T06:11:33.256Z)

```
[external_agent_tool_result]
=== gh auth ===
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_**********************************
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
=== remote ===
origin  https://github.com/avidullu/badminton-highlight-indexer.git (fetch)
origin  https://github.com/avidullu/badminton-highlight-indexer.git (push)
=== ci.yml triggers ===
name: CI

on:
  push:
    branches: [master]
  pull_request:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
  # tracked .py, including scripts/ and training/, which the backend-only
  # py_compile smoke sweep does not reach.
  lint:
[/external_agent_tool_result]
```

99. assistant (2026-07-02T06:11:33.256Z)

```
All set ? `gh` authed with `repo` scope, CI runs on `pull_request`. Let me push the branch.
```

100. assistant (2026-07-02T06:11:33.257Z)

```
[external_agent_tool_call: Bash]
description: Push the fix branch to origin
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-
schema-fix
git push -u origin fix/ablation-litmus-mixed-schema 2>&1 | tail -8
[/external_agent_tool_call]
```

101. assistant (2026-07-02T06:11:33.257Z)

```
[external_agent_tool_result]
remote:
remote: Create a pull request for 'fix/ablation-litmus-mixed-schema' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/fix/ablation-
litmus-mixed-schema
remote:
```

```
branch 'fix/ablation-litmus-mixed-schema' set up to track 'origin/fix/ablation-litmus-mixed-
schema'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      fix/ablation-litmus-mixed-schema -> fix/ablation-litmus-mixed-schema
[/external_agent_tool_result]
```

102. assistant (2026-07-02T06:11:33.257Z)

```
Branch pushed. Now I'll file the GitHub issue describing the bug.
```

103. assistant (2026-07-02T06:11:33.257Z)

```
[external_agent_tool_call: Bash]
description: Create the GitHub issue
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-
schema-fix
cat > /tmp/issue_body.md <<'EOF'
```

Summary

```
`tests/test_ablation.py::test_litmus_reproduces_gate_a` and
`::test_litmus_report_is_well_formed` **fail locally** with `KeyError: 'shuttle'` whenever the
full **n=11** golden corpus is present in `output/`.

CI is unaffected (CI's `output/` is empty, so the litmus tests **skip**), but on a machine
holding the full golden corpus, `python run_all_tests.py` shows **2 reds**.
```

Root cause

```
`backend/eval/ablation.py::load_videos()` reads `c["shuttle"]` / `c["person"]`
**unconditionally** on every candidate:

```python
for fn in SHUTTLE_COLUMNS:
    features[fn] = [float(c["shuttle"][fn]) for c in cands]   # KeyError here
```

The litmus skip-gate `_golden_present()` globs **all** `output/*_golden_features.json` and runs
once ?6 exist. But the 5 newer real-footage clips added during R3/R4 corpus growth ?
`mahadevpura_1`, `mahadevpura_2`, `GX010128`, `GX030094`, `Badminton_BXH_2` ? carry a **leaner**
candidate schema produced by the velocity-windowing/distill path:

```
['active_frame_count', 'chunk_index', 'end', 'mean_velocity', 'peak_velocity', 'start',
'track_id_count']
```

with **no** `shuttle`/`person`/`audio` feature dicts. The original n=6 files (e.g.
`adarsh_avi_singles`) carry the full fusion schema
`['audio','end','label','person','shuttle','start']`. The loader crashes on the leaner files.
```

Reproduction

```
With the full n=11 corpus in `output/`:

```
$ python -m pytest tests/test_ablation.py::test_litmus_reproduces_gate_a -x -q
...
>   features[fn] = [float(c["shuttle"][fn]) for c in cands]
E   KeyError: 'shuttle'
```
```

Fix

Filter the loader to the **full-fusion-schema** files (skip windowing-only files) ? the most robust of the considered options. See the linked PR.

Out of scope (separate follow-up)

`backend/eval/fusion_compare.py::load_videos` and `backend/eval/fusion_audio.py::load_videos_with_audio` carry the **identical** unconditional `c["shuttle"]` access. They are not caught by any test today (their tests use synthetic tmp corpora only) but would crash the same way on the mixed corpus. Tracked separately.
EOF
gh issue create \
  --title "Ablation litmus tests crash with KeyError: 'shuttle' on the full n=11 golden corpus (mixed schema)" \
  --body-file /tmp/issue_body.md \
  --label bug 2>&1 | tail -5
[/external_agent_tool_call]

104. assistant (2026-07-02T06:11:33.257Z)

[external_agent_tool_result]
https://github.com/avidullu/badminton-highlight-indexer/issues/208
[/external_agent_tool_result]

105. assistant (2026-07-02T06:11:33.257Z)

Issue [#208](https://github.com/avidullu/badminton-highlight-indexer/issues/208) filed. Now the PR, linking the issue with `Closes #208`.

106. assistant (2026-07-02T06:11:33.257Z)

[external_agent_tool_call: Bash]
description: Open the PR linking issue #208
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
cat > /tmp/pr_body.md <<'EOF'
Closes #208.

Problem

`tests/test_ablation.py::test_litmus_reproduces_gate_a` and `::test_litmus_report_is_well_formed` crash with `KeyError: 'shuttle'` whenever the full **n=11** golden corpus is present in `output/`. CI stays green (empty `output/` ? litmus skips), but the owner box sees 2 reds.

`ablation.load_videos()` read `c["shuttle"]`/`c["person"]` unconditionally, but the 5 newer R3/R4 clips (`Badminton_BXH_2`, `GX010128`, `GX030094`, `mahadevpura_1`, `mahadevpura_2`) are persisted via the velocity-windowing/distill path with a leaner schema (no per-cue dicts). The litmus skip-gate `_golden_present()` counted those too, so it ran the litmus on an un-ablatable corpus.

Fix (most-robust option)

- **`_is_fusion_schema()`** ? a file is ablatable iff every candidate carries `shuttle`+`person` (`audio` stays optional, preserving the existing pre-audio tolerance).
- **`load_videos()`** skips leaner windowing-only files with an info log instead of crashing ? a mixed corpus loads only the ablatable subset.
- **`_golden_present()`** (test gate) now counts only fusion-schema files, so the skip-gate matches the loader.

The 6 fusion-schema files are exactly the original n=6, so the litmus still reproduces the recorded Gate-A result (?F1?0.074, 6/6 wins, p?0.031) ? against real data, not just "no crash".

Tests

- `test_load_videos_skips_windowing_only_schema` ? mixed corpus loads only the fusion files; the litmus path runs end-to-end.
- `test_is_fusion_schema_distinguishes_lean_and_full` ? predicate keeps full/audio-optional rows, rejects lean rows + empty.

Offline/mocked.

Verification (local, with `output/` mirroring the owner box so the litmus ran for real)

- Full suite: **848 passed, 5 skipped**
- `ruff check .`: clean · `mypy backend training`: clean (104 files)
- Coverage **80.66%** (`--cov-fail-under=70`); `ablation.py` at 99%

Does **not** touch `scripts/nightly_regression.py`.

Out of scope (follow-up)

`fusion_compare.load_videos` and `fusion_audio.load_videos_with_audio` have the identical unconditional `c["shuttle"]` access ? latent (not caught by any test today), tracked separately.

? Generated with [Claude Code](https://claude.com/claude-code)
EOF
gh pr create --base master --head fix/ablation-litmus-mixed-schema \
  --title "fix(eval): ablation litmus tolerates a mixed-schema golden corpus" \
  --body-file /tmp/pr_body.md 2>&1 | tail -5
[/external_agent_tool_call]

107. assistant (2026-07-02T06:11:33.257Z)

[external_agent_tool_result]
https://github.com/avidullu/badminton-highlight-indexer/pull/209
[/external_agent_tool_result]

108. assistant (2026-07-02T06:11:33.257Z)

PR [#209](https://github.com/avidullu/badminton-highlight-indexer/pull/209) is open and linked to the issue. Now the due-diligence layer: I'll launch an adversarial multi-lens review of the committed diff (read-only, won't touch the repo) while CI runs on GitHub. Let me kick off both.

109. assistant (2026-07-02T06:11:33.257Z)

[external_agent_tool_call: Workflow]
description: Adversarial review of the ablation fix
[/external_agent_tool_call]

110. assistant (2026-07-02T06:11:33.257Z)

[external_agent_tool_result]
Workflow launched in background. Task ID: w35v6q4cl
Summary: Adversarial multi-lens review of the ablation mixed-schema fix before merge
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_34138e24-10c
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-ablation-schema-fix\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\ablation-fix-due-diligence-wf_34138e24-10c.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-ablation-schema-fix\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\ablation-fix-due-diligence-wf_34138e24-10c.js"} to iterate without resending the script.)
Run ID: wf_34138e24-10c
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-ablation-schema-

fix\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\ablation-fix-due-diligence-wf_34138e24-10c.js", resumeFromRunId: "wf_34138e24-10c"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.
[/external_agent_tool_result]

111. assistant (2026-07-02T06:11:33.257Z)

[external_agent_tool_call: Bash]
description: Check PR CI status
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
sleep 5
echo "=== PR #209 checks ==="
gh pr checks 209 2>&1 | head -20
echo "=== PR mergeable state ==="
gh pr view 209 --json mergeStateStatus,mergeable,state,reviewDecision 2>&1 | head -10
[/external_agent_tool_call]

112. assistant (2026-07-02T06:11:33.257Z)

[external_agent_tool_result]
=== PR #209 checks ===
Golden fixtures cross-OS (windows-latest) pass 54s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985388
Mypy (lenient baseline) pass 29s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985329
Ruff lint pass 9s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985319
Syntax sweep + import smoke (no GPU stack) pass 22s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985373
Full test suite (offline, mocked) pending 0 https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985339
Golden fixtures cross-OS (macos-latest) pass 21s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985351
Golden fixtures cross-OS (ubuntu-latest) pass 27s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985346
=== PR mergeable state ===
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE","reviewDecision":"","state":"OPEN"}
[/external_agent_tool_result]

113. assistant (2026-07-02T06:11:33.257Z)

CI is already mostly green (ruff, mypy, syntax, golden-fixtures on all 3 OSes pass); only the full offline suite is still running. The review workflow is running in the background. Let me watch CI to completion.

114. assistant (2026-07-02T06:11:33.257Z)

[external_agent_tool_call: Bash]
description: Watch PR CI checks until they settle
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
gh pr checks 209 --watch --interval 15 2>&1 | tail -15
echo "=== exit: $? (0 == all checks passed) ==="
[/external_agent_tool_call]

115. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_result]
Golden fixtures cross-OS (ubuntu-latest) pass 27s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985346
Full test suite (offline, mocked) pass 2m13s https://github.com/avidullu/badminton-

highlight-indexer/actions/runs/27730077676/job/82034985339
Golden fixtures cross-OS (macos-latest) pass 21s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985351
Golden fixtures cross-OS (ubuntu-latest) pass 27s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985346
Golden fixtures cross-OS (windows-latest) pass 54s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985388
Mypy (lenient baseline) pass 29s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985329
Ruff lint pass 9s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985319
Syntax sweep + import smoke (no GPU stack) pass 22s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985373
Full test suite (offline, mocked) pass 2m13s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985339
Golden fixtures cross-OS (macos-latest) pass 21s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985351
Golden fixtures cross-OS (ubuntu-latest) pass 27s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985346
Golden fixtures cross-OS (windows-latest) pass 54s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985388
Mypy (lenient baseline) pass 29s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985329
Ruff lint pass 9s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985319
Syntax sweep + import smoke (no GPU stack) pass 22s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27730077676/job/82034985373
=== exit: 0 (0 == all checks passed) ===
[/external_agent_tool_result]

116. assistant (2026-07-02T06:11:33.258Z)

All 8 CI checks pass. The last gate is the adversarial review workflow ? I won't merge until it returns a clean verdict. Let me check its status.

117. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_call: ToolSearch]
input: {"query":"select:TaskGet,TaskOutput","max_results":5}
[/external_agent_tool_call]

118. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_result]
[/external_agent_tool_result]

119. assistant (2026-07-02T06:11:33.258Z)

I'll wait for the review workflow to finish before merging (it's the last gate).

120. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_call: TaskOutput]
input: {"task_id":"w35v6q4cl","block":true,"timeout":300000}
[/external_agent_tool_call]

121. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_result]
<retrieval_status>timeout</retrieval_status>

<task_id>w35v6q4cl</task_id>

<task_type>local_workflow</task_type>

<status>running</status>
[/external_agent_tool_result]

122. assistant (2026-07-02T06:11:33.258Z)

Still running. Let me give it more time ? I committed to gating the merge on its verdict.

123. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_call: TaskOutput]
input: {"task_id":"w35v6q4cl","block":true,"timeout":300000}
[/external_agent_tool_call]

124. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_result]
<retrieval_status>success</retrieval_status>

<task_id>w35v6q4cl</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

{
  "summary": "Adversarial multi-lens review of the ablation mixed-schema fix before merge",
  "agentCount": 5,
  "logs": [
    "reviews: 4/4 returned · findings: 13 · must-fix: 0"
  ],
  "result": {
    "synthesis": {
      "verdict": "merge",
      "must_fixes": [],
      "should_fixes": [
        "Litmus count-growth fragility (pre-existing, not a blocker): _golden_present() gates on >=6 fusion files (tests/test_ablation.py:422) while test_litmus_reproduces_gate_a asserts len(videos) == 6 exactly (line 433). Today this is consistent ? I verified exactly 6 fusion + 5 lean files in output/, so load_videos returns exactly 6. But if a 7th FULL-FUSION golden clip is ever added, the gate stays True and the == 6 assertion turns the litmus red on more data. This PR actually fixes the opposite drift (lean files no longer inflate the count). A follow-up could assert len(videos) >= 6 and slice/use a recorded 6, or make the litmus corpus selection explicit. Out of scope for this bug-fix.",
        "Sibling loaders fusion_compare.load_videos (backend/eval/fusion_compare.py:46-48) and fusion_audio.load_videos_with_audio still read c['shuttle']/c['person'] unconditionally and would KeyError on the same mixed corpus (golden_fixtures.py:352 documents this hazard). Pre-existing and explicitly outside this commit's scope (ablation litmus only); not run against the mixed corpus today. A follow-up could factor _is_fusion_schema into a shared filter."
      ],
      "nits": [
        "_is_fusion_schema (ablation.py:395) only guards TOP-LEVEL presence of shuttle/person, not the inner cue columns load_videos reads (e.g. c['shuttle'][fn] at line 420). A file that passes the predicate but has a cue dict missing an individual column would still KeyError. Not a concern for the current corpus ? I confirmed all 6 fusion files carry every shuttle/person column. Theoretical only; could switch inner reads to .get(fn, 0.0) if malformed-but-present cue dicts ever appear.",
        "_golden_present() (tests/test_ablation.py:415-419) catches (OSError, ValueError) around json.load(...).get('candidates'); a valid-but-non-dict top-level JSON (e.g. a bare list) would raise AttributeError on .get(), which is uncaught. Test-only skip-gate code; realistic failure modes (missing file, malformed JSON ? JSONDecodeError is a ValueError subclass) are handled. Very low priority; could guard isinstance(obj, dict).",
        "No explicit test for the asymmetric partial case (a candidate with 'person' present but 'shuttle' absent, or vice versa). The all('shuttle' in c and 'person' in c ...) predicate

</output>

handles it correctly (returns False -> skipped), but only the symmetric lean case is pinned. Could add a one-line assert _is_fusion_schema([{'shuttle':{}}]) is False.",
        "Docstring wording imprecision (tests/test_ablation.py:387): says the synthetic mixed corpus contains 'the 5 newer real clips' but the test body writes only 3 windowing-only files. The '5' refers to the real owner-box corpus (which I confirmed has 5 lean files); the synthetic uses 3 as a representative sample. Test logic is correct; cosmetic only."
      ],
      "rationale": "Merge. I verified every load-bearing claim locally against the real golden substrate, not just the PR description. (1) The diff is correctly scoped to exactly 2 files (backend/eval/ablation.py +29, tests/test_ablation.py +57) and touches nothing else. (2) The real corpus output/ has 11 golden files; _is_fusion_schema partitions them into exactly 6 fusion (adarsh_avi_singles, gbaaddy, kushagra_singles, largetest_doubles, mahadevpura_singles, testlarge_short) + 5 lean (Badminton_BXH_2, GX010128, GX030094, mahadevpura_1, mahadevpura_2), and load_videos('output') returns exactly the 6 fusion videos ? matching all four reviews. (3) The bug is real: pre-fix load_videos (git show HEAD~1...
[/external_agent_tool_result]

125. assistant (2026-07-02T06:11:33.258Z)

Both gates are clear: **all 8 CI checks pass** and the **adversarial review verdict is `merge` with 0 must-fixes** (all 4 lenses independently confirmed the bug is real, the litmus reproduces the Gate-A numbers, and the diff is correctly scoped). The two "should-fixes" it raised are both pre-existing/out-of-scope ? and one is the sibling-loader bug I already filed as a follow-up chip.

Merging now (squash, matching the repo's `?(#NNN)` convention):

126. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_call: Bash]
description: Squash-merge PR #209 and delete remote branch
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
echo "=== final pre-merge state ==="
gh pr view 209 --json mergeStateStatus,mergeable,state | head -5
echo "=== squash-merge + delete remote branch ==="
gh pr merge 209 --squash --delete-branch 2>&1 | tail -10
[/external_agent_tool_call]

127. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_result]
=== final pre-merge state ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","state":"OPEN"}
=== squash-merge + delete remote branch ===
failed to run git: fatal: 'master' is already used by worktree at 'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'
[/external_agent_tool_result]

128. assistant (2026-07-02T06:11:33.258Z)

The error is from `gh`'s local post-merge step (it tried to switch to `master`, which is checked out in another worktree). The server-side merge itself likely went through. Let me verify the PR state and master.

129. assistant (2026-07-02T06:11:33.258Z)

[external_agent_tool_call: Bash]
description: Verify PR merged, issue closed, remote branch deleted
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
echo "=== PR #209 state ==="
gh pr view 209 --json state,merged,mergedAt,mergeCommit 2>&1 | head -8
echo "=== issue #208 state (should auto-close via 'Closes #208') ==="

```
gh issue view 208 --json state,closed 2>&1 | head -5
echo "=== remote branch still there? ==="
git ls-remote --heads origin fix/ablation-litmus-mixed-schema 2>&1 | head -3
echo "(empty above == remote branch deleted)"
[/external_agent_tool_call]
```

130. assistant (2026-07-02T06:11:33.258Z)

```
[external_agent_tool_result]
=== PR #209 state ===
Unknown JSON field: "merged"
Available fields:
  additions
  assignees
  author
  autoMergeRequest
  baseRefName
  baseRefOid
=== issue #208 state (should auto-close via 'Closes #208') ===
{"closed":true,"state":"CLOSED"}
=== remote branch still there? ===
(empty above == remote branch deleted)
[/external_agent_tool_result]
```

131. assistant (2026-07-02T06:11:33.258Z)

Issue #208 auto-closed and the remote branch is gone ? both confirm the merge landed. Let me confirm the merge commit and that `origin/master` advanced.

132. assistant (2026-07-02T06:11:33.258Z)

```
[external_agent_tool_call: Bash]
description: Confirm merge commit and origin/master advanced
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-
schema-fix
echo "=== PR state + merge commit ==="
gh pr view 209 --json state,mergeCommit,title 2>&1
echo "=== origin/master after fetch ==="
git fetch origin master 2>&1 | tail -2
git log --oneline -3 origin/master
[/external_agent_tool_call]
```

133. assistant (2026-07-02T06:11:33.258Z)

```
[external_agent_tool_result]
=== PR state + merge commit ===
{"mergeCommit":{"oid":"4bd9d2dbe540a4903a654fe69acc96077d1e31ec"},"state":"MERGED","title":"fix(
eval): ablation litmus tolerates a mixed-schema golden corpus"}
=== origin/master after fetch ===
 * branch            master     -> FETCH_HEAD
   be39a70..4bd9d2d  master     -> origin/master
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
4bbd987 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17)
[/external_agent_tool_result]
```

134. assistant (2026-07-02T06:11:33.258Z)

PR **MERGED** ? `origin/master` is now at `4bd9d2d ?(#209)`. Now cleanup: remove the worktree (work is merged; the gitignored `output/` copies are disposable) and delete the local feature branch that `gh` couldn't remove.

135. assistant (2026-07-02T06:11:33.258Z)

```
[external_agent_tool_call: Bash]
description: Remove merged worktree and delete local branch
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== remove the merged worktree (force: drops gitignored output/ copies) ==="
git worktree remove .claude/worktrees/ablation-schema-fix --force 2>&1 | tail -3
echo "=== delete the local feature branch ==="
git branch -D fix/ablation-litmus-mixed-schema 2>&1 | tail -2
echo "=== worktrees now ==="
git worktree list
echo "=== main checkout untouched? ==="
git branch --show-current
[/external_agent_tool_call]
```

136. assistant (2026-07-02T06:11:33.258Z)

```
[external_agent_tool_result]
=== remove the merged worktree (force: drops gitignored output/ copies) ===
error: failed to delete 'C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/ablation-schema-fix': Permission denied
=== delete the local feature branch ===
Deleted branch fix/ablation-litmus-mixed-schema (was 59ae347).
=== worktrees now ===
C:/Users/avidu/Projects/badminton-highlight-indexer
f5f5e52 [docs/golden-regression-fixtures-plan]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed
7e0881a [perf/wasb-fast-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy
5cb7bbf [bhi-execpolicy]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17
8736e2d [run/boxhill17-highlights]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-golden
c33cf1d [docs/golden-videos-tracker]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-rally-quality
877ab1f [docs/rally-quality-research]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-today
ce7eb47 [docs/quality-iterations-realfootage]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hardchop
614325d [feat/w1-hard-cap-fallback]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/local-noai
3ea7ad4 [feat/local-no-ai-segmenter]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/pr-stitch
a6adcc5 [fix/runner-stitch-watermark]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/r2-impl
a450c53 [feat/r2-tolerance-metric]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/r4-endrule
e248d2d [feat/r4-inactivity-end]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier0-eval
9e03e5a [feat/tier0-rally-seg-eval]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing
a3b209a [master]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-netcross
effd920 [feat/w2-netcross-gate]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard
5ba6955 [feat/w2-never-empty-safeguard]
=== main checkout untouched? ===
docs/golden-regression-fixtures-plan
[/external_agent_tool_result]
```

137. assistant (2026-07-02T06:11:33.259Z)

```
The local branch is deleted and git no longer lists the worktree (its metadata was removed).
Only the physical directory deletion hit a Windows "Permission denied" (a lingering file lock).
Let me try to clear the leftover directory.
```

138. assistant (2026-07-02T06:11:33.259Z)

```
[external_agent_tool_call: Bash]
description: Remove leftover worktree directory and prune
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
if [ -d .claude/worktrees/ablation-schema-fix ]; then
  rm -rf .claude/worktrees/ablation-schema-fix 2>&1 | tail -3
fi
git worktree prune 2>&1 | tail -2
if [ -d .claude/worktrees/ablation-schema-fix ]; then
  echo "LEFTOVER: directory still present (locked file); harmless gitignored copies ? owner can
delete later"
else
  echo "CLEAN: worktree directory removed"
fi
echo "=== confirm not in worktree list ==="
git worktree list | grep -c ablation-schema-fix || echo "0 (absent from worktree list ? good)"
[/external_agent_tool_call]
```

139. assistant (2026-07-02T06:11:33.259Z)

```
[external_agent_tool_result]
CLEAN: worktree directory removed
=== confirm not in worktree list ===
0
0 (absent from worktree list ? good)
[/external_agent_tool_result]
```

140. assistant (2026-07-02T06:11:33.259Z)

Clean. Let me update the live checkpoint to reflect the merge.

141. assistant (2026-07-02T06:11:33.259Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md
[/external_agent_tool_call]
```

142. assistant (2026-07-02T06:11:33.259Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
[/external_agent_tool_result]
```

143. assistant (2026-07-02T06:11:33.259Z)

All done. The fix is reported, reviewed, and merged with full due diligence.

What landed

```
| Step | Result |
|---|---|
| **Issue** | [#208](https://github.com/avidullu/badminton-highlight-indexer/issues/208) ? bug
report (root cause, repro, scope) ? **auto-closed** by the merge |
| **PR** | [#209](https://github.com/avidullu/badminton-highlight-indexer/pull/209) `fix(eval):
ablation litmus tolerates a mixed-schema golden corpus` ? **MERGED** (squash) |
| **master** | advanced `be39a70 ? 4bd9d2d ?(#209)` |
```

Due diligence (three independent gates, all green)

1. **Local, mirroring the owner box** (golden corpus in `output/` so the litmus ran for real): full suite **848 passed / 5 skipped**, `ruff check .` clean, `mypy backend training` clean (104 files), coverage **80.66%** (floor 70), `ablation.py` at 99%.
2. **GitHub CI** ? all **8 checks pass**: Ruff, Mypy, full offline suite, syntax/import smoke, and golden-fixtures cross-OS (ubuntu/macos/windows).
3. **Adversarial 4-lens review workflow** (correctness · edge-cases · test-quality · convention/scope ? gatekeeper synthesis): **verdict = merge, 0 must-fixes**. The reviewers independently re-verified the bug against the pre-fix code, confirmed the litmus reproduces the recorded Gate-A numbers (?F1?0.074, 6/0, p?0.031), and confirmed the diff is scoped to exactly 2 files.

Cleanup

Remote and local `fix/ablation-litmus-mixed-schema` branches deleted; worktree removed; the live checkpoint memory is updated.

Two follow-ups the review surfaced (both correctly left out of this focused PR)

- **Sibling loaders** `fusion_compare.load_videos` / `fusion_audio.load_videos_with_audio` have the identical unconditional `c["shuttle"]` access ? latent (no test exercises the real corpus) but would crash the same way. Already tracked as a follow-up chip (`task_8dd4efde`).
- **Litmus count fragility** (pre-existing, non-blocking): `_golden_present()` gates on `>=6` fusion files while the litmus asserts `len == 6`. Fine today (exactly 6 fusion files) ? and this PR fixed the *opposite* drift ? but a 7th fusion clip would turn it red. Worth a one-line `>= 6` relaxation when the golden corpus next grows.

Want me to spin up the sibling-loader fix now, or leave it on the chip for later?

144. user (2026-07-02T06:11:33.259Z)

I approved a chip but if there are *any* other fixes then plesae go ahead and send and merge their PRs

145. assistant (2026-07-02T06:11:33.259Z)

The chip is being handled by its own spawned session ? so I'll steer clear of the sibling loaders to avoid colliding with it, and instead hunt for *other* real fixes to ship. Let me first sync to latest master (the chip session may have merged) and map the code surface.

146. assistant (2026-07-02T06:11:33.259Z)

```
[external_agent_tool_call: Bash]
description: Sync master, list open PRs, map code dirs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git fetch origin 2>&1 | tail -3
echo "=== origin/master HEAD now ==="
git log --oneline -3 origin/master
echo "=== any open PRs (chip session may have one in flight)? ==="
gh pr list --state open --limit 20 2>&1 | head -20
echo "=== backend tree ==="
find backend scripts training -name '*.py' -not -path '*/__pycache__/*' | sed 's#/[^/]*$##' |
sort -u
[/external_agent_tool_call]
```

147. assistant (2026-07-02T06:11:33.259Z)

```
[external_agent_tool_result]
=== origin/master HEAD now ===
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
4bbd987 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17)
=== any open PRs (chip session may have one in flight)? ===
210     fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus
fix/fusion-loaders-mixed-schema OPEN    2026-06-18T01:15:28Z
207     docs: capture external review inputs (research, pick-up-later)  claude/indexer-doc-
```

```
review-XVPTh OPEN    2026-06-17T18:10:06Z
204     [DRAFT · product-go needed] R1 milestone ? flip windowing defaults to cap=6 + R4
feat/r1-milestone-config-flip   DRAFT   2026-06-17T12:04:01Z
=== backend tree ===
backend
backend/annotations
backend/config
backend/eval
backend/eval/audio
backend/features
backend/infrastructure
backend/pipeline/audio
backend/pipeline/detectors
backend/pipeline/personalization
backend/pipeline/segmenters
backend/pipeline/stitcher
backend/pipeline/validators
backend/providers
backend/reporting
backend/sports
backend/tools
backend/utils
scripts
training
training/gen0
[/external_agent_tool_result]
```

148. assistant (2026-07-02T06:11:33.259Z)

PR **#210** is already open ? that's the chip session's sibling-loader fix, actively in flight. I'll leave it to that session and **not** touch the fusion loaders, to avoid racing on the shared checkout. The other open PRs (#207 docs, #204 product-gated draft) aren't mine to merge.

Now to hunt for *other* real fixes. Let me set up a clean `origin/master` worktree as the read base so any finding maps cleanly to a master-based PR.

149. assistant (2026-07-02T06:11:33.259Z)

```
[external_agent_tool_call: Bash]
description: Create read-only origin/master worktree for hunting
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git worktree add --detach .claude/worktrees/hunt-base origin/master 2>&1 | tail -3
git -C .claude/worktrees/hunt-base log --oneline -1
echo "=== source file count on master ==="
find .claude/worktrees/hunt-base/backend .claude/worktrees/hunt-base/scripts
.claude/worktrees/hunt-base/training -name '*.py' -not -path '*/__pycache__/*' | wc -l
[/external_agent_tool_call]
```

150. assistant (2026-07-02T06:11:33.259Z)

```
[external_agent_tool_result]
Preparing worktree (detached HEAD 4bd9d2d)
HEAD is now at 4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
=== source file count on master ===
108
[/external_agent_tool_result]
```

151. assistant (2026-07-02T06:11:33.259Z)

Clean `origin/master` base ready (108 source files). Now I'll run a high-precision adversarial bug-hunt: parallel finders across the codebase (excluding the just-fixed `ablation.py` and the in-flight fusion loaders) ? diverse refutation of each candidate ? synthesis into shippable vs owner-gated. Token cost isn't a constraint here (ultracode), but I'm tuning for precision ? only

real, reachable defects.

152. assistant (2026-07-02T06:11:33.259Z)

[external_agent_tool_call: Workflow]
description: Adversarial bug-hunt for shippable fixes
[/external_agent_tool_call]

153. assistant (2026-07-02T06:11:33.259Z)

[external_agent_tool_result]
Workflow launched in background. Task ID: wip5pkgi0
Summary: Adversarial hunt for real, reachable defects to ship as individual PRs
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_b7f012f8-5c3
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-shippable-fixes-wf_b7f012f8-5c3.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-shippable-fixes-wf_b7f012f8-5c3.js"} to iterate without resending the script.)
Run ID: wf_b7f012f8-5c3
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-shippable-fixes-wf_b7f012f8-5c3.js", resumeFromRunId: "wf_b7f012f8-5c3"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.
[/external_agent_tool_result]

154. assistant (2026-07-02T06:11:33.260Z)

The hunt is running across the codebase. Let me wait for its results before acting.

155. assistant (2026-07-02T06:11:33.260Z)

[external_agent_tool_call: TaskOutput]
input: {"task_id":"wip5pkgi0","block":true,"timeout":600000}
[/external_agent_tool_call]

156. assistant (2026-07-02T06:11:33.260Z)

[external_agent_tool_result]
<retrieval_status>success</retrieval_status>

<task_id>wip5pkgi0</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

```
{
  "summary": "Adversarial hunt for real, reachable defects to ship as individual PRs",
  "agentCount": 6,
  "logs": [
    "Find: 0 candidate(s) across 6 areas"
  ],
  "result": {
    "shippable": [],
    "needs_owner": [],
    "rejected": [],
```

```
    "note": "No candidate defects surfaced."
  }
}
</output>
[/external_agent_tool_result]
```

157. assistant (2026-07-02T06:11:33.260Z)

Zero candidates across all 6 areas ? and these finders did real work (763k tokens, 201 tool reads), and were explicitly told an empty list is a respected answer and not to manufacture findings. For a codebase with 848 tests and multiple hardening sweeps, a clean result is plausible.

But a single precision-tuned pass can suppress subtler issues, and you asked me to be thorough. Let me run one complementary pass decomposed by **bug-class** (cross-cutting all files, the proven "unguarded persisted-data" vein included) with a completeness critic, before I conclude there's nothing else to ship.

158. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_call: Workflow]
description: Second-pass bug-hunt by class + completeness critic
[/external_agent_tool_call]
```

159. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_result]
Workflow launched in background. Task ID: w9knuyga2
Summary: Second-pass defect hunt decomposed by bug-class, with a completeness critic
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-pass2-by-class-
wf_49c1018a-e67.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-pass2-by-class-
wf_49c1018a-e67.js"} to iterate without resending the script.)
Run ID: wf_49c1018a-e67
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bug-hunt-pass2-by-class-
wf_49c1018a-e67.js", resumeFromRunId: "wf_49c1018a-e67"}) ? completed agents return cached
results.

You will be notified when it completes. Use /workflows to watch live progress.
[/external_agent_tool_result]
```

160. assistant (2026-07-02T06:11:33.260Z)

Second pass running. Let me wait for it.

161. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_call: TaskOutput]
input: {"task_id":"w9knuyga2","block":true,"timeout":600000}
[/external_agent_tool_call]
```

162. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_result]
<retrieval_status>timeout</retrieval_status>

<task_id>w9knuyga2</task_id>
```

<task_type>local_workflow</task_type>

<status>running</status>
[/external_agent_tool_result]

163. assistant (2026-07-02T06:11:33.260Z)

Still running (it found candidates and is verifying them, or finders are still reading). Let me continue waiting.

164. assistant (2026-07-02T06:11:33.260Z)

[external_agent_tool_call: TaskOutput]
input: {"task_id":"w9knuyga2","block":true,"timeout":420000}
[/external_agent_tool_call]

165. assistant (2026-07-02T06:11:33.260Z)

[external_agent_tool_result]
<retrieval_status>success</retrieval_status>

<task_id>w9knuyga2</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

```
{
  "summary": "Second-pass defect hunt decomposed by bug-class, with a completeness critic",
  "agentCount": 15,
  "logs": [
    "Find(pass2): 4 candidate(s)",
    "Verify(pass2): 3/4 survived"
  ],
  "result": {
    "synthesis": {
      "shippable": [
        {
          "title": "Make v3/v4 migration ALTERs idempotent so an interrupted one-time upgrade can't permanently brick the DB",
          "file": "backend/infrastructure/database.py",
          "line": "156-159 (v3) and 168-178 (v4)",
          "bug": "The v3 block (`ALTER TABLE jobs ADD COLUMN policy` / `next_poll_at`) and v4 block (`ALTER TABLE videos ADD COLUMN user_id` / `ALTER TABLE candidate_segments ADD COLUMN user_id`) use bare, non-idempotent `ALTER TABLE ... ADD COLUMN` and only bump `PRAGMA user_version` at the END of the block. In CPython sqlite3 each DDL ALTER auto-commits independently of the surrounding `with conn:` scope in _connect, so if the process is interrupted (SIGKILL/power-loss, disk I/O error, or a later-statement OperationalError such as busy_timeout exhaustion) AFTER an ALTER commits but BEFORE the version bump, the column persists while user_version stays at the pre-block value. On the next open, _init_db re-runs the same bare ALTER and raises `sqlite3.OperationalError: duplicate column name: ...`. Since _init_db runs in __init__, every subsequent Database(db_path) construction throws and the app can never open that DB again. v1/v2/v5 blocks are deliberately re-runnable (CREATE ... IF NOT EXISTS); only v3/v4 break the crash-safe-ladder invariant.",
          "fix_summary": "Guard each bare ALTER with a column-existence check before running it, e.g. a small helper `def _has_col(cur, table, col): return col in {r[1] for r in cur.execute(f'PRAGMA table_info({table})')}` and only ALTER when the column is absent (equivalently, catch and ignore the 'duplicate column name' OperationalError per-ALTER). This makes an interrupted block safely re-applicable, matching the already-idempotent v1/v2/v5 blocks. The IF-NOT-EXISTS index statements in those blocks are already fine and unchanged. No schema/behavior change on the happy path.",
          "risk": "low",
          "test_plan": "Add a test in tests/test_database.py simulating a partial migration:
```

</output>

build a DB at user_version=3 whose `videos` table already has a `user_id` column (and a user_version=2 DB whose `jobs` table already has `policy`), then construct Database(db_path) and assert it opens cleanly and reaches user_version=5 (instead of raising 'duplicate column name'). Keep the existing test_init_db_idempotent_and_upgrades_old_db and test_migration_v3_db_upgrades_to_v5_preserving_rows green. Run full suite: python run_all_tests.py.",
          "owner_gated": false
        },
        {
          "title": "Unwrap the segmenter Result before len() in run_phase3_eval.py (motion is the default + documented segmenter)",
          "file": "scripts/run_phase3_eval.py",
          "line": "145-146",
          "bug": "Line 145 does `segs = segmenter.process_video(...)` and line 146 does `len(segs)`. Per the base.py contract, process_video returns a Result (Success|Failure). The motion segmenter ? which is the argparse default (`--segmenter` default=\"motion\", line 53) and the documented 'Free CPU baseline' example (docstring lines 10-13) ? returns `Success(value=segments_data)` (motion.py:224) or `Failure(...)` (motion.py:31). Success/Failure are plain dataclasses with no __len__, so `len(segs)` raises `TypeError: object of type 'Success' has no len()`, crashing the documented first-run command before any scoring/report. Other segmenters (gemini/yolo_hybrid/trajectory_hybrid) return bare lists and mask the bug. The driver never unwraps, unlike backend/main.py.",
          "fix_summary": "Mirror the backend/main.py pattern: after line 145, `result = segm...
[/external_agent_tool_result]

166. assistant (2026-07-02T06:11:33.260Z)

The second pass found **3 confirmed real defects** (4 candidates, 3 survived diverse refutation), all `owner_gated=false`. Before trusting any of it, I'll verify each against the actual master code myself (my pre-LGTM gate ? reproduce, don't trust the description). Let me read the relevant code.

167. assistant (2026-07-02T06:11:33.260Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-base\backend\infrastructure\database.py
[/external_agent_tool_call]

168. assistant (2026-07-02T06:11:33.260Z)

[external_agent_tool_result]
```
100                             video_id TEXT NOT NULL,
101                             detector TEXT NOT NULL,
102                             chunk_index INTEGER,
103                             start_time REAL NOT NULL,
104                             end_time REAL NOT NULL,
105                             duration REAL NOT NULL,
106                             confidence REAL,
107                             stats TEXT,
108                             detection_params TEXT,
109                             confirmed_segment_id INTEGER,
110                             human_label TEXT,
111                             notes TEXT,
112                             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113                             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114                             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
115                         )
116                     """)
117                     cursor.execute("""
118                         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
119                             ON candidate_segments(video_id, detector)
120                     """)
```

```
121                     cursor.execute("PRAGMA user_version = 1")
122
123                 if version < 2:
124                     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
125                     # /api/process request. `status` is the EXECUTION state of the job
126                     # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127                     # that failed validation) lives inside `result` JSON as
result["status"].
128                     # `result` is the full sync-era response dict (incl. report paths), so
the
129                     # observability report is the job's artifact, per the plan.
130                     cursor.execute("""
131                         CREATE TABLE IF NOT EXISTS jobs (
132                             id TEXT PRIMARY KEY,
133                             video_path TEXT NOT NULL,
134                             video_id TEXT,
135                             segmenter TEXT,
136                             player_pool TEXT,
137                             max_frames INTEGER,
138                             status TEXT NOT NULL DEFAULT 'queued',
139                             progress TEXT,
140                             error TEXT,
141                             result TEXT,
142                             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143                             started_at TIMESTAMP,
144                             finished_at TIMESTAMP
145                         )
146                     """)
147                     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
148                     cursor.execute("PRAGMA user_version = 2")
149
150                 if version < 3:
151                     # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152                     # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153                     # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154                     # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155                     # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156                     cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157                     cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158                     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159                     cursor.execute("PRAGMA user_version = 3")
160
161                 if version < 4:
162                     # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
163                     # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164                     # path can scope rows to an owner. **NULL = local install = byte-
identical to
165                     # today** ? every existing row stays ...
[/external_agent_tool_result]
```

169. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\results.py
[/external_agent_tool_call]
```

170. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_result]
1       """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3       This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5       The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6       (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8       We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9       This is intentionally small and incremental ? we start by applying it to the segmenter
10      and provider layers (the biggest sources of silent [] returns today).
11
12      The types are generic where it makes sense. Callers can still access .value on Success
13      or .message on Failure.
14
15      This module has no dependencies on the rest of the app.
16      """
17
18      from __future__ import annotations
19
20      from dataclasses import dataclass, field
21      from typing import Generic, TypeVar, List, Optional, Any, Union
22
23      T = TypeVar("T")
24
25
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
```

```
61
[/external_agent_tool_result]
```

171. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\scripts\run_phase3_eval.py
[/external_agent_tool_call]
```

172. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_result]
130                 skipped.append((vid, "no GT csv"))
131                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
132                 continue
133
134             video_id = get_file_md5(video_path)
135
136             # Segment unless score-only or already present (and not --reprocess).
137             if not args.score_only:
138                 already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
139                 if already and not args.reprocess:
140                     print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
141                 else:
142                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
143                     db.add_video(video_id, video_path, "processed", [])
144                     t0 = time.time()
145                     segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
146                     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
147
148             gt = load_annotations(csv_path)
149             report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
150             print(format_report_text(report, show_failures=args.show_failures))
151             print()
152             d = report_to_dict(report)
153             d["collector_video_id"] = vid
154             per_video.append(d)
155
156         agg = batch.aggregate(per_video, stages)
157         print(batch.format_aggregate(agg))
158         if skipped:
159             print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
160
161         if args.json_out:
162             with open(args.json_out, "w") as f:
163                 json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
164                            "aggregate": agg, "per_video": per_video,
[/external_agent_tool_result]
```

173. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\scripts\run_process_and_stitch.py
[/external_agent_tool_call]
```

174. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_result]
70              print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
71              db.add_video(video_id, video_path, "processed", [])
72
73              print("Running segmenter & indexing...", flush=True)
74              seg_name = cfg.indexing.default_segmenter
75              segmenter_cls = segmenter_registry.get(seg_name)
76              if not segmenter_cls:
77                  print(f"[FAIL] Segmenter '{seg_name}' not found.")
78                  return
79              print(f"Using segmenter: {seg_name}", flush=True)
80              segmenter = segmenter_cls(db, cfg)
81              segments = segmenter.process_video(video_id, video_path)
82              print(f"Successfully indexed {len(segments)} play segments.", flush=True)
83
84          if not segments:
85              print("No play segments detected. Cannot compile highlights.")
86              return
87
88
89          end_padding = cfg.stitching.end_padding
90          begin_padding = cfg.stitching.begin_padding
91          min_shot_count = cfg.stitching.min_shot_count
92
93          if cache_hit:
94              print("\n--- Stitching Customizations ---")
95              try:
96                  bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
97                  if bpad_input:
98                      begin_padding = float(bpad_input)
99
[/external_agent_tool_result]
```

175. assistant (2026-07-02T06:11:33.260Z)

```
Verified the two core facts: `Success`/`Failure` (backend/results.py) have **no `__len__`**, and
the v3/v4 migration blocks use **bare non-idempotent `ALTER`** while v1/v2/v5 are `IF NOT
EXISTS`. Let me confirm the segmenter-return facts and the canonical unwrap pattern before
deciding how to ship.
```

176. assistant (2026-07-02T06:11:33.260Z)

```
[external_agent_tool_call: Bash]
description: Verify segmenter return types and main.py unwrap
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
echo "=== motion.process_video return sites ==="
grep -nE "def process_video|return Success|return Failure|return \[|return segments|return
result" backend/pipeline/segmenters/motion.py | head -20
echo
echo "=== run_phase3_eval argparse --segmenter default ==="
grep -nE "add_argument..--segmenter|default=.motion|from backend|import"
scripts/run_phase3_eval.py | head -15
echo
echo "=== backend/main.py unwrap pattern (isinstance Failure / .value) ==="
grep -nE "process_video|isinstance.*Failure|\.value|Success|Failure|len\(" backend/main.py |
head -25
echo
echo "=== how do OTHER segmenters return? (gemini/yolo_hybrid/trajectory_hybrid) ==="
grep -rnE "def process_video|return Success|return Failure|-> .*Result|-> List"
backend/pipeline/segmenters/*.py | grep -iE "process_video|Success|Failure|Result|List" | head
-30
[/external_agent_tool_call]
```

177. assistant (2026-07-02T06:11:33.261Z)

```
[external_agent_tool_result]
=== motion.process_video return sites ===
20:    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:
31:            return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
224:        return Success(value=segments_data)

=== run_phase3_eval argparse --segmenter default ===
22:import os
23:import sys
24:import json
25:import time
26:import argparse
27:import logging
29:from dotenv import load_dotenv
32:from backend.infrastructure.database import Database
33:from backend.config import load_config
34:from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
36:from backend.pipeline.segmenters import segmenter_registry
37:from backend.eval.annotations import load_annotations
38:from backend.eval.harness import evaluate, format_report_text, report_to_dict
39:from backend.eval import batch
53:    p.add_argument("--segmenter", default="motion",

=== backend/main.py unwrap pattern (isinstance Failure / .value) ===
35:from backend.results import Failure  # standardized error/result contract
192:            result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
195:            # state (same destroy-after-confirm contract as the Failure branch), then let
201:        if isinstance(result, Failure):
213:        segments = result.value if hasattr(result, "value") else result
220:            "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
347:def process_video(req: ProcessRequest):
482:            received += len(chunk)
607:    if len(kept) < len(matched_segments):
609:                    f"{len(matched_segments) - len(kept)}/{len(matched_segments)} segments
below the floor.")
612:                            detail=f"All {len(matched_segments)} matching segments fall
below "
683:        avg_motion = sum(motions) / len(motions)
689:        print(f"  Sample Frame Count: {len(motions)}")
835:            result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
838:            if isinstance(result, Failure):
843:                segments = result.value if hasattr(result, "value") else result
844:                print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}")

=== how do OTHER segmenters return? (gemini/yolo_hybrid/trajectory_hybrid) ===
backend/pipeline/segmenters/base.py:24:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
backend/pipeline/segmenters/fusion_hybrid.py:143:                            idx_cfg: Dict[str,
Any]) -> List[int]:
backend/pipeline/segmenters/gemini.py:35:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend/pipeline/segmenters/motion.py:20:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend/pipeline/segmenters/motion.py:31:            return Failure(message=f"Could not open
video: {video_path}", is_recoverable=False)
backend/pipeline/segmenters/motion.py:224:        return Success(value=segments_data)
backend/pipeline/segmenters/trajectory_hybrid.py:98:                            idx_cfg:
```

```
Dict[str, Any]) -> List[int]:
backend/pipeline/segmenters/trajectory_hybrid.py:106:    def process_video(self, video_id: str,
video_path: str,  # type: ignore[override]
backend/pipeline/segmenters/trajectory_hybrid.py:108:                      max_frames:
Optional[int] = None) -> List[Dict[str, Any]]:
backend/pipeline/segmenters/trajectory_hybrid.py:277:            def
process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
backend/pipeline/segmenters/yolo_hybrid.py:57:    def process_video(self, video_id: str,
video_path: str,...
[/external_agent_tool_result]
```

178. assistant (2026-07-02T06:11:33.261Z)

All three confirmed real on master:
- **motion** returns `Success`/`Failure` (motion.py:31,224); `run_phase3_eval.py` defaults `--segmenter` to **`motion`** (line 53) ? the documented default command hits `len(Success)` ? `TypeError`.
- **`backend/main.py`** has the canonical unwrap (`isinstance(result, Failure)` ? handle; else `result.value if hasattr(result,"value") else result`) at lines 201-213 and 838-843 ? the scripts simply never adopted it.
- v3/v4 bare `ALTER` vs idempotent v1/v2/v5 confirmed.

I'll ship **two PRs** (separate subsystems, no stacking): (1) the DB migration idempotency fix ? highest value, fully testable in CI here; (2) the two script unwraps grouped (same bug-class). Let me start with PR #1. First, the migration function structure:

179. assistant (2026-07-02T06:11:33.261Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-
base\backend\infrastructure\database.py
[/external_agent_tool_call]
```

180. assistant (2026-07-02T06:11:33.261Z)

```
[external_agent_tool_result]
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
13
14          @contextmanager
15          def _connect(self):
16              """Transaction scope that also CLOSES the connection.
17
18              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19              never closes, so every call leaked its connection to GC (lingering file
20              handles on the WAL sidecars under Windows). Internal methods use this.
21              """
22              conn = self._get_connection()
23              try:
24                  with conn:
25                      yield conn
26              finally:
27                  conn.close()
28
```

```
        def _get_connection(self):
            # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
            # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
            conn = sqlite3.connect(self.db_path, timeout=30.0)
            conn.row_factory = sqlite3.Row
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
            conn.execute("PRAGMA busy_timeout = 30000")
            try:
                conn.execute("PRAGMA journal_mode = WAL")
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                # Videos Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS videos (
                        id TEXT PRIMARY KEY,
                        filepath TEXT NOT NULL,
                        processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                        status TEXT NOT NULL,
                        validation_results TEXT
                    )
                """)

                # Play Segments Table (Extensible metadata JSON)
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS play_segments (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        video_id TEXT NOT NULL,
                        start_time REAL NOT NULL,
                        end_time REAL NOT NULL,
                        duration REAL NOT NULL,
                        server TEXT,
                        receiver TEXT,
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
```

[/external_agent_tool_result]

181. assistant (2026-07-02T06:11:33.261Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hunt-base\backend\infrastructure\database.py
[/external_agent_tool_call]

182. assistant (2026-07-02T06:11:33.261Z)

[external_agent_tool_result]
```
88                  # Versioned migrations live here. PRAGMA user_version is the schema
89                  # contract ? bump it for every change and add an ordered `if version < N`
90                  # block below. Tests assert the expected version, so accidental drift fails
CI.
91                  version = cursor.execute("PRAGMA user_version").fetchone()[0]
92
93                  if version < 1:
94                      # v1: raw candidate detections (pre-confirmation), detector-agnostic.
95                      # Common fields are columns; detector-specific metrics go in `stats`
JSON,
96                      # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
97                      cursor.execute("""
98                          CREATE TABLE IF NOT EXISTS candidate_segments (
99                              id INTEGER PRIMARY KEY AUTOINCREMENT,
```
[/external_agent_tool_result]

183. assistant (2026-07-02T06:11:33.261Z)

Everything checks out. The cleanest idempotent fix keeps the `ALTER` statements as literals (no f-string SQL) and tolerates re-application. Let me set up PR #1's worktree from a fresh `origin/master` and read the existing DB tests to match conventions.

184. assistant (2026-07-02T06:11:33.261Z)

[external_agent_tool_call: Bash]
description: Fetch, check chip PR, create PR#1 worktree
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
```
git fetch origin 2>&1 | tail -2
echo "=== origin/master HEAD (did chip #210 merge yet?) ==="
git log --oneline -1 origin/master
gh pr view 210 --json state,title 2>&1 | head -3
echo "=== create PR#1 worktree ==="
git worktree add -b fix/db-migration-idempotent-alters .claude/worktrees/db-migration-fix
origin/master 2>&1 | tail -2
git -C .claude/worktrees/db-migration-fix log --oneline -1
git -C .claude/worktrees/db-migration-fix branch --show-current
```
[/external_agent_tool_call]

185. assistant (2026-07-02T06:11:33.261Z)

[external_agent_tool_result]
```
=== origin/master HEAD (did chip #210 merge yet?) ===
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
{"state":"OPEN","title":"fix(eval): fusion loaders skip windowing-only files on a mixed golden
corpus"}
=== create PR#1 worktree ===
branch 'fix/db-migration-idempotent-alters' set up to track 'origin/master'.
HEAD is now at 4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
fix/db-migration-idempotent-alters
```
[/external_agent_tool_result]

186. assistant (2026-07-02T06:11:33.261Z)

```
[external_agent_tool_result]
=== migration-related tests ===
4:def test_database_init(tmp_path):
16:def test_database_video(tmp_path):
31:def test_database_play_segment(tmp_path):
60:    "user_id",  # v4: personalization groundwork (nullable; NULL = local install)
64:def test_candidate_schema_contract(tmp_path):
80:def test_schema_version_is_current(tmp_path):
84:    version = conn.execute("PRAGMA user_version").fetchone()[0]
86:    # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
90:def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
106:    version = conn.execute("PRAGMA user_version").fetchone()[0]
114:def test_candidate_round_trip(tmp_path):
141:def test_candidate_link_and_unlabeled_filter(tmp_path):
160:def test_candidate_cascade_on_video_delete(tmp_path):
169:# --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
178:def test_v4_user_id_columns_present_and_nullable(tmp_path):
181:    assert "user_id" in _table_cols(db_path, "videos")
182:    assert "user_id" in _table_cols(db_path, "candidate_segments")
188:        assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
192:def test_v4_existing_writers_leave_user_id_null(tmp_path):
193:    """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
201:    vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1'").fetchone()[0]
202:    cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1'").fetchone()[0]
206:    # Existing readers are unaffected (no user_id filter in the query path).
210:def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
212:    columns/tables appear, user_id back-fills to NULL, and prior rows survive the ALTERs."""
222:    # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind version
224:    conn.execute("ALTER TABLE videos DROP COLUMN user_id")
225:    conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
226:    conn.execute("PRAGMA user_version = 3")
229:    assert "user_id" not in _table_cols(db_path, "videos")
231:    # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
234:    assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
238:    assert "user_id" in _table_cols(db_path, "videos")
239:    assert "user_id" in _table_cols(db_path, "candidate_segments")
249:def test_user_models_schema_contract(tmp_path):
254:        "id", "user_id", "version", "baseline_version", "feature_schema_hash",
260:def test_upsert_and_get_active_user_model_round_trip(tmp_path):
274:    assert m["user_id"] == "user-1"
285:def test_get_active_user_model_none_when_absent_or_inactive(tmp_path):
294:def test_upsert_is_idempotent_on_user_version(tmp_path):
=== total lines ===
351 tests/test_database.py
[/external_agent_tool_result]
```

187. assistant (2026-07-02T06:11:33.261Z)

Chip PR #210 still open (its session's call). My PR#1 worktree is on `origin/master`. Let me read the existing migration tests to match conventions.

188. assistant (2026-07-02T06:11:33.261Z)

```
[external_agent_tool_call: Bash]
description: Find migration tests in test_database.py
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-
fix
echo "=== migration-related tests ==="
grep -nE "def test_|user_version|duplicate|migrat|ALTER|policy|user_id" tests/test_database.py |
head -40
echo "=== total lines ==="
```

```
wc -l tests/test_database.py
[/external_agent_tool_call]
```

189. assistant (2026-07-02T06:11:33.261Z)

```
There's a `test_migration_v3_db_upgrades_to_v5_preserving_rows` for the happy re-migration. Mine
tests the *partial/interrupted* path (columns present, version behind). Let me read it + the
helper to match style.
```

190. assistant (2026-07-02T06:11:33.261Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-
fix\tests\test_database.py
[/external_agent_tool_call]
```

191. assistant (2026-07-02T06:11:33.261Z)

```
[external_agent_tool_result]
1       import sqlite3
2       from backend.infrastructure.database import Database
3
4       def test_database_init(tmp_path):
5           db_path = tmp_path / "test.db"
6           Database(str(db_path))  # instantiation creates the schema
7
8           conn = sqlite3.connect(str(db_path))
9           cursor = conn.cursor()
10          cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
11          tables = [r[0] for r in cursor.fetchall()]
12          assert "videos" in tables
13          assert "play_segments" in tables
14          conn.close()
[/external_agent_tool_result]
```

192. assistant (2026-07-02T06:11:33.261Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-
fix\tests\test_database.py
[/external_agent_tool_call]
```

193. assistant (2026-07-02T06:11:33.261Z)

```
[external_agent_tool_result]
169     # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
170
171     def _table_cols(db_path, table):
172         conn = sqlite3.connect(str(db_path))
173         cols = {c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()}
174         conn.close()
175         return cols
176
177
178     def test_v4_user_id_columns_present_and_nullable(tmp_path):
179         db_path = tmp_path / "test.db"
180         Database(str(db_path))
181         assert "user_id" in _table_cols(db_path, "videos")
182         assert "user_id" in _table_cols(db_path, "candidate_segments")
183
184         # The column is NULLABLE: PRAGMA table_info notnull flag must be 0.
185         conn = sqlite3.connect(str(db_path))
186         for table in ("videos", "candidate_segments"):
187             by_name = {c[1]: c for c in conn.execute(f"PRAGMA
```

```
table_info({table})").fetchall()}
            assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
        conn.close()


    def test_v4_existing_writers_leave_user_id_null(tmp_path):
        """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
        row a current writer produces is local (NULL) ? byte-identical to pre-v4
behaviour."""
        db_path = tmp_path / "test.db"
        db = Database(str(db_path))
        db.add_video("vid1", "vid1.mp4", "processed", [])
        db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)

        conn = sqlite3.connect(str(db_path))
        vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1'").fetchone()[0]
        cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1'").fetchone()[0]
        conn.close()
        assert vu is None
        assert cu is None
        # Existing readers are unaffected (no user_id filter in the query path).
        assert len(db.query_candidate_segments("vid1")) == 1


    def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
        """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the
new
        columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERs."""
        db_path = tmp_path / "legacy_v3.db"
        db = Database(str(db_path))  # fresh DB is born at the current version; simulate a
v3 state
        # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
        db.add_video("vidA", "a.mp4", "processed", [{"v": 1}])
        db.add_candidate_segment("vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings":
3})
        conn = sqlite3.connect(str(db_path))
        conn.execute("DROP INDEX IF EXISTS idx_videos_user")
        conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
        conn.execute("DROP TABLE IF EXISTS user_models")
        # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind
version
        # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
        conn.execute("ALTER TABLE videos DROP COLUMN user_id")
        conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
        conn.execute("PRAGMA user_version = 3")
        conn.commit()
        conn.close()
        assert "user_id" not in _table_cols(db_path, "videos")

        # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
        db2 = Database(str(db_path))
        conn = sqlite3.connect(str(db_path))
        assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
        assert "user_models" in {r[0] for r in conn.execute(
            "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
        conn.close()
        assert "user_id" in _table_cols(db_path, "videos")
        assert "user_id" in _table_cols(db_path, "candidate_segments")

        v = db2.get_video("vidA")
        assert v is not None and v["validation_results"] == [{"v": 1}]
        cands = db2.query_candidate_segments("vidA")
```

[/external_agent_tool_result]

194. assistant (2026-07-02T06:11:33.262Z)

Conventions clear. My test covers the *partial-apply* path (columns present, version behind) ? complementary to the existing happy-path test. Let me read the worktree's exact copy and implement.

195. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py
[/external_agent_tool_call]

196. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_result]
```
42              return conn
43
44          def _init_db(self):
45              """Initializes tables for videos, play_segments, and events."""
46              with self._connect() as conn:
47                  cursor = conn.cursor()
```
[/external_agent_tool_result]

197. assistant (2026-07-02T06:11:33.262Z)

Now implement the fix ? the idempotent-ALTER helper and the v3/v4 call sites:

198. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py
[/external_agent_tool_call]

199. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

200. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py
[/external_agent_tool_call]

201. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

202. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_call: Edit]

file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py
[/external_agent_tool_call]

203. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\backend\infrastructure\database.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

204. assistant (2026-07-02T06:11:33.262Z)

Now the regression test for the partial/interrupted-migration path:

205. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py
[/external_agent_tool_call]

206. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_result]
```
243         cands = db2.query_candidate_segments("vidA")
244         assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3
245
246
247     # --- v5: per-user model store CRUD ------------------------------------------------
248
249     def test_user_models_schema_contract(tmp_path):
250         db_path = tmp_path / "test.db"
```
[/external_agent_tool_result]

207. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py
[/external_agent_tool_call]

208. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

209. assistant (2026-07-02T06:11:33.262Z)

I left a nonsensical dead line in that test ? let me clean it up.

210. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py
[/external_agent_tool_call]

211. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

212. assistant (2026-07-02T06:11:33.262Z)

Now verify the test pins the bug: run it against the fix (expect pass), then temporarily revert the fix to confirm it fails without it.

213. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_call: Bash]
description: Verify the test pins the bug (passes with fix, fails without)
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-fix
echo "=== 1) new test WITH the fix (expect PASS) ==="
python -m pytest tests/test_database.py::test_migration_idempotent_on_interrupted_upgrade -q 2>&1 | tail -6
echo
echo "=== 2) revert ONLY database.py, re-run (expect FAIL: duplicate column name) ==="
git stash push -- backend/infrastructure/database.py 2>&1 | tail -1
python -m pytest tests/test_database.py::test_migration_idempotent_on_interrupted_upgrade -q 2>&1 | tail -8
echo "=== restore the fix ==="
git stash pop 2>&1 | tail -2
[/external_agent_tool_call]

214. assistant (2026-07-02T06:11:33.262Z)

[external_agent_tool_result]
=== 1) new test WITH the fix (expect PASS) ===
.   [100%]
1 passed in 0.18s

=== 2) revert ONLY database.py, re-run (expect FAIL: duplicate column name) ===
Saved working directory and index state WIP on fix/db-migration-idempotent-alters: 4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
                # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
>   cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
E   sqlite3.OperationalError: duplicate column name: policy

backend\infrastructure\database.py:156: OperationalError
========================== short test summary info ==========================
FAILED tests/test_database.py::test_migration_idempotent_on_interrupted_upgrade
1 failed in 0.32s
=== restore the fix ===
no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (3f83b3951c42cdc5d90d0bf9702de77b637a4824)
[/external_agent_tool_result]

215. assistant (2026-07-02T06:11:33.262Z)

The test pins the bug precisely ? passes with the fix, fails with `duplicate column name: policy` without it. Now the full local gate: DB tests, full suite, ruff, mypy, coverage.

216. assistant (2026-07-02T06:11:33.263Z)

[external_agent_tool_call: Bash]
description: Run DB tests, ruff, mypy on changed files
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-fix
echo "=== test_database.py (full) ==="

```
python -m pytest tests/test_database.py -q 2>&1 | tail -4
echo "=== ruff + mypy on changed files ==="
python -m ruff check backend/infrastructure/database.py tests/test_database.py 2>&1 | tail -3
python -m mypy backend/infrastructure/database.py 2>&1 | tail -3
[/external_agent_tool_call]
```

217. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_result]
=== test_database.py (full) ===
....................                                                     [100%]
20 passed in 0.83s
=== ruff + mypy on changed files ===
All checks passed!
Success: no issues found in 1 source file
[/external_agent_tool_result]
```

218. assistant (2026-07-02T06:11:33.263Z)

DB tests 20/20, ruff + mypy clean. Now the full suite + repo-wide lint/type + coverage gate:

219. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_call: Bash]
description: Repo-wide ruff, mypy, full suite with coverage
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-
fix
echo "=== ruff check . ===" && python -m ruff check . 2>&1 | tail -3
echo "=== mypy backend training ===" && python -m mypy backend training 2>&1 | tail -2
echo "=== full suite + coverage ===" && python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70 2>&1 | tail -6
[/external_agent_tool_call]
```

220. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_result]
=== ruff check . ===
All checks passed!
=== mypy backend training ===
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 104 source files
=== full suite + coverage ===
TOTAL                                              7992   1545    81%
Required test coverage of 70% reached. Total coverage: 80.67%
847 passed, 7 skipped, 7 warnings in 86.46s (0:01:26)
======================================================================
 RESULT: PASS - all tests green
======================================================================
[/external_agent_tool_result]
```

221. assistant (2026-07-02T06:11:33.263Z)

All green: ruff clean, mypy clean (104 files), suite 847 passed/7 skipped (the 2 litmus tests skip here since this worktree's `output/` is empty ? expected), coverage 80.67%. Let me review the diff, then commit.

222. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_call: Bash]
description: Review PR#1 diff
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-
fix
git diff -- backend/infrastructure/database.py tests/test_database.py
```

[/external_agent_tool_call]

223. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_result]
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index 59473ac..58153a9 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -41,6 +41,24 @@ class Database:
             pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
         return conn

+    @staticmethod
+    def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
+        """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
+
+        A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
+        transaction, so an interrupted v3/v4 block can leave the column added while
+        ``user_version`` stays un-bumped ? and the next open would re-run the same
+        ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
+        future ``Database(...)`` construction (``_init_db`` runs in ``__init__``). Swallow
+        ONLY that specific error so the ladder stays crash-safe, matching the re-runnable
+        ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
+        """
+        try:
+            cursor.execute(ddl)
+        except sqlite3.OperationalError as exc:
+            if "duplicate column name" not in str(exc).lower():
+                raise
+
     def _init_db(self):
         """Initializes tables for videos, play_segments, and events."""
         with self._connect() as conn:
@@ -152,9 +170,10 @@ class Database:
                 # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
                 # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any
worker
                 # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
-                # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
-                cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
-                cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
+                # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
+                # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
+                self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
+                self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
                 cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
                 cursor.execute("PRAGMA user_version = 3")

@@ -163,10 +182,11 @@ class Database:
                 # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
                 # path can scope rows to an owner. **NULL = local install = byte-identical to
                 # today** ? every existing row stays NULL and every existing query (which never
-                # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
+                # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive;
+                # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
                 # Additive ONLY: no served-behavior change rides on this slice.
-                cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
```

```
-                 cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
+                 self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
+                 self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
                  # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path
free
                  # of index churn while making the eventual per-user lookups cheap.
                  cursor.execute(
diff --git a/tests/test_database.py b/tests/test_database.py
index 5ccdb49..8909208 100644
--- a/tests/test_da...
[/external_agent_tool_result]
```

224. assistant (2026-07-02T06:11:33.263Z)

Clean diff. Let me commit (re-verify HEAD, stage explicit paths):

225. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_call: Bash]
description: Commit PR#1 and verify clean branch
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-
fix
git branch --show-current && git log --oneline -1
git add backend/infrastructure/database.py tests/test_database.py
git status --porcelain
git commit -F - <<'EOF'
fix(db): make v3/v4 migration ALTERs idempotent so an interrupted upgrade can't brick the DB

The v1/v2/v5 migration blocks are re-runnable (CREATE ... IF NOT EXISTS), but v3
(`ALTER TABLE jobs ADD COLUMN policy / next_poll_at`) and v4 (`ALTER TABLE videos
/ candidate_segments ADD COLUMN user_id`) used bare ADD COLUMNs and only bump
`PRAGMA user_version` at the END of the block.

A DDL ALTER auto-commits independently of the surrounding `with conn:` scope, so
if the process is interrupted (crash/power-loss/IO error/lock timeout) AFTER an
ALTER commits but BEFORE the version bump, the column persists while user_version
stays stale. The next open re-runs the same bare ALTER and raises
`OperationalError: duplicate column name`. Since `_init_db` runs in `__init__`,
every subsequent `Database(db_path)` then throws ? the DB is permanently unopenable.

Fix: a small `_add_column_idempotent(cursor, ddl)` helper runs each ADD COLUMN and
swallows ONLY the 'duplicate column name' OperationalError, making the v3/v4 steps
re-runnable like the rest of the ladder. Happy path unchanged (columns added,
version bumped exactly as before); no schema/behavior/default change.

Regression test `test_migration_idempotent_on_interrupted_upgrade` reproduces the
half-applied state (columns present, user_version rewound to 2) and asserts a clean
re-open to v5. Verified it FAILS without the fix (`duplicate column name: policy`)
and passes with it.

Local: full suite 847 passed / 7 skipped, ruff clean, mypy clean (104 files),
coverage 80.67% (floor 70). Pure-sqlite, runs in CI (no GPU).

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
echo "=== only my commit atop origin/master? ==="
git log --oneline origin/master..HEAD
git diff --stat origin/master..HEAD
[/external_agent_tool_call]
```

226. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_result]
fix/db-migration-idempotent-alters
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
M  backend/infrastructure/database.py
M  tests/test_database.py
[fix/db-migration-idempotent-alters d140722] fix(db): make v3/v4 migration ALTERs idempotent so
an interrupted upgrade can't brick the DB
 2 files changed, 59 insertions(+), 6 deletions(-)
=== only my commit atop origin/master? ===
d140722 fix(db): make v3/v4 migration ALTERs idempotent so an interrupted upgrade can't brick
the DB
 backend/infrastructure/database.py | 32 ++++++++++++++++++++++++++------
 tests/test_database.py             | 33 +++++++++++++++++++++++++++++++++
 2 files changed, 59 insertions(+), 6 deletions(-)
[/external_agent_tool_result]
```

227. assistant (2026-07-02T06:11:33.263Z)

```
Clean single commit. Push and open PR #1:
```

228. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_call: Bash]
description: Push and open PR#1
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-
fix
git push -u origin fix/db-migration-idempotent-alters 2>&1 | tail -4
cat > /tmp/pr1_body.md <<'EOF'
```

Problem

```
The SQLite migration ladder (`backend/infrastructure/database.py::_init_db`) is mostly crash-
safe ? v1/v2/v5 use re-runnable `CREATE ... IF NOT EXISTS`. But **v3** (`ALTER TABLE jobs ADD
COLUMN policy / next_poll_at`) and **v4** (`ALTER TABLE videos / candidate_segments ADD COLUMN
user_id`) used bare `ADD COLUMN`s and only bump `PRAGMA user_version` at the **end** of the
block.

A DDL `ALTER` auto-commits independently of the surrounding `with conn:` transaction. So if the
process is interrupted (crash / power-loss / disk IO error / lock-timeout) **after** an `ALTER`
commits but **before** the version bump, the column persists while `user_version` stays stale.
On the next open, `_init_db` re-runs the same bare `ALTER` ? `OperationalError: duplicate column
name`. Because `_init_db` runs in `__init__`, **every subsequent `Database(db_path)` then throws
? the DB is permanently unopenable.**
```

Fix

```
A small `_add_column_idempotent(cursor, ddl)` helper runs each `ADD COLUMN` and swallows
**only** the `duplicate column name` `OperationalError`, making v3/v4 re-runnable like the rest
of the ladder. The happy path is unchanged (columns added, version bumped exactly as before). No
schema / behavior / default change.
```

Test

```
`test_migration_idempotent_on_interrupted_upgrade` reproduces the half-applied state (v3/v4
columns present, `user_version` rewound to 2) and asserts a clean re-open to v5. Verified it
**fails without the fix** (`duplicate column name: policy`) and passes with it. Pure-sqlite ?
runs in CI here, no GPU.
```

Verification (local)

```
- Full suite: **847 passed, 7 skipped** (the 7 skips are the optional-dep / golden-corpus-absent
tests)
- `ruff check .` clean · `mypy backend training` clean (104 files)
- Coverage **80.67%** (`--cov-fail-under=70`)
```

Context

```
Found by an adversarial bug-hunt after the litmus fix
([#209](https://github.com/avidullu/badminton-highlight-indexer/pull/209)). Pure crash-safety
hardening that brings v3/v4 in line with the already-idempotent v1/v2/v5 blocks. The trigger
window is narrow today (no legacy sub-v4 on-disk DBs in the wild yet), but the failure mode is a
permanent, unrecoverable lockout ? cheap to prevent.

? Generated with [Claude Code](https://claude.com/claude-code)
EOF
gh pr create --base master --head fix/db-migration-idempotent-alters \
  --title "fix(db): make v3/v4 migration ALTERs idempotent (interrupted upgrade can brick the
DB)" \
  --body-file /tmp/pr1_body.md 2>&1 | tail -3
[/external_agent_tool_call]
```

229. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_result]
remote:
branch 'fix/db-migration-idempotent-alters' set up to track 'origin/fix/db-migration-idempotent-
alters'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      fix/db-migration-idempotent-alters -> fix/db-migration-idempotent-alters
https://github.com/avidullu/badminton-highlight-indexer/pull/212
[/external_agent_tool_result]
```

230. assistant (2026-07-02T06:11:33.263Z)

```
PR **[#212](https://github.com/avidullu/badminton-highlight-indexer/pull/212)** open. PR #1 is
committed+pushed (no uncommitted work), so I can safely set up PR #2 in its own worktree while
#1's CI runs. Let me create it and read the two scripts.
```

231. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_call: Bash]
description: Create PR#2 worktree and read script imports/call sites
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git worktree add -b fix/scripts-unwrap-segmenter-result .claude/worktrees/scripts-unwrap
origin/master 2>&1 | tail -2
echo "=== run_phase3_eval.py imports (1-45) ==="
sed -n '28,45p' .claude/worktrees/scripts-unwrap/scripts/run_phase3_eval.py
echo "=== run_phase3_eval.py call-site context (141-147) ==="
sed -n '141,147p' .claude/worktrees/scripts-unwrap/scripts/run_phase3_eval.py
echo "=== run_process_and_stitch.py imports (1-40) ==="
sed -n '1,40p' .claude/worktrees/scripts-unwrap/scripts/run_process_and_stitch.py
[/external_agent_tool_call]
```

232. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_result]
branch 'fix/scripts-unwrap-segmenter-result' set up to track 'origin/master'.
HEAD is now at 4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
=== run_phase3_eval.py imports (1-45) ===

from dotenv import load_dotenv
load_dotenv()  # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters

from backend.infrastructure.database import Database
from backend.config import load_config
from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
# Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
from backend.pipeline.segmenters import segmenter_registry
```

```
from backend.eval.annotations import load_annotations
from backend.eval.harness import evaluate, format_report_text, report_to_dict
from backend.eval import batch

logger = logging.getLogger(__name__)


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(
=== run_phase3_eval.py call-site context (141-147) ===
            else:
                print(f"[proc] {vid}: segmenting with {args.segmenter}...")
                db.add_video(video_id, video_path, "processed", [])
                t0 = time.time()
                segs = segmenter.process_video(video_id, video_path, max_frames=args.max_frames)
                print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")

=== run_process_and_stitch.py imports (1-40) ===
import os
import json
import logging
from dotenv import load_dotenv

load_dotenv()
```

Central root-logger setup ? all backend modules use logging.getLogger(__name__). Quiet third-party HTTP chatter by default; re-honoured against the config knob (logging.verbose_http) once the config is loaded in main().

```
from backend.utils.logging_setup import setup_logging
setup_logging()

from backend.infrastructure.database import Database
from backend.config import load_config
from backend.utils.hashing import compute_video_id
from backend.pipeline.segmenters import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.tools.export import format_rally_summary

logger = logging.getLogger(__name__)

def main():
    video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
    output_dir = r"C:\Users\avidu\Videos\Testing"
    output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"

    print("Loading config...")
    cfg = load_config()  # typed AppConfig ? single source of defaults
    # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter quiet).
    setup_logging(verbose_http=cfg.logging.verbose_http)

    print("Verifying files exist...")
    if not os.path.exists(video_path):
        print(f"Error: Video file not found at {video_path}")
        return

    os.makedirs(cfg.output.output_dir, exist_ok=True)
    db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
    db = Database(db_path)
[/external_agent_tool_result]
```

233. assistant (2026-07-02T06:11:33.263Z)

```
[external_agent_tool_result]
```

```
40          db = Database(db_path)
41
42          # Calculate MD5 of the video to uniquely identify it
43          import time
44          print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
45          t0 = time.time()
46          video_id = compute_video_id(video_path)
47          print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
48
49          segments = []
50          cached_video = db.get_video(video_id)
51          cache_hit = False
52
53          if cached_video and cached_video.get("status") == "processed":
54              segments = db.query_play_segments(video_id, {})
55              if len(segments) > 0:
56                  print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
57                  choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
58                  if choice == '1':
59                      print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
60                      cache_hit = True
61                  else:
62                      print("[CACHE] Ignoring cache...", flush=True)
63              else:
64                  print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
65
66          if not cache_hit:
67              print("[CACHE] Initializing clean indexing...", flush=True)
68              db.clear_video_data(video_id)
69
70              print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
71              db.add_video(video_id, video_path, "processed", [])
72
73              print("Running segmenter & indexing...", flush=True)
74              seg_name = cfg.indexing.default_segmenter
75              segmenter_cls = segmenter_registry.get(seg_name)
76              if not segmenter_cls:
77                  print(f"[FAIL] Segmenter '{seg_name}' not found.")
78                  return
79              print(f"Using segmenter: {seg_name}", flush=True)
80              segmenter = segmenter_cls(db, cfg)
81              segments = segmenter.process_video(video_id, video_path)
82              print(f"Successfully indexed {len(segments)} play segments.", flush=True)
83
84          if not segments:
85              print("No play segments detected. Cannot compile highlights.")
86              return
87
88
89          end_padding = cfg.stitching.end_padding
90          begin_padding = cfg.stitching.begin_padding
91          min_shot_count = cfg.stitching.min_shot_count
92
93          if cache_hit:
94              print("\n--- Stitching Customizations ---")
95              try:
96                  bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
97                  if bpad_input:
```

```
98                      begin_padding = float(bpad_input)
99
100                 pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
101                 if pad_input:
102                     end_padding = float(pad_input)
103
104                 shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
105                 if shot_input:
106                     min_shot_count = int(shot_input)
107             except ValueError:
108                 print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")
109
110         print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
111
112         # Filter segments where shot_count >= min_shot_count
113         time_pairs = []
114         skipped_count = 0
115         for s in segments:
116             meta = s.get("metadata", {})
117             if isinstance(meta, str):
118                 try:
119                     meta = json.loads(meta)
120                 except Exception:
121                     meta = {}
122             shot_count = meta.get("shot_count", 0)
123             if shot_count >= min_shot_count:
124                 time_pairs.append((s["start_time"], s["end_time"]))
125             else:
126                 skippe...
[/external_agent_tool_result]
```

234. assistant (2026-07-02T06:11:33.263Z)

Let me read the full `run_process_and_stitch.py` flow to fix it without breaking the cache path (`segments` is used after the branch):

235. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
unwrap\scripts\run_process_and_stitch.py
[/external_agent_tool_call]
```

236. assistant (2026-07-02T06:11:33.264Z)

`segments` lifecycle is clear (init `[]`, cache path = real list, non-cache path = the Result to unwrap). Let me check what's importable here (cv2/torch) to gauge how far I can verify the script fixes locally.

237. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_call: Bash]
description: Check cv2/torch availability and loop scope
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
for m in cv2 torch numpy; do python -c "import $m; print('$m', getattr($m,'__version__','?'))"
2>&1 | head -1; done
echo "=== can we import the segmenter registry (pulls motion?cv2)? ==="
python -c "from backend.pipeline.segmenters import segmenter_registry; print('registry OK,
segmenters:', sorted(segmenter_registry._registry.keys()) if
hasattr(segmenter_registry,'_registry') else 'n/a')" 2>&1 | tail -3
```

```
echo "=== confirm skipped/loop var in run_phase3_eval ==="
grep -nE "skipped = |for vid|skipped\.append" scripts/run_phase3_eval.py | head
[/external_agent_tool_call]
```

238. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_result]
cv2 4.13.0
torch 2.6.0+cu124
numpy 2.4.6
=== can we import the segmenter registry (pulls motion?cv2)? ===
registry OK, segmenters: n/a
=== confirm skipped/loop var in run_phase3_eval ===
117:    skipped = []
126:            skipped.append((vid, "no video file"))
130:            skipped.append((vid, "no GT csv"))
[/external_agent_tool_result]
```

239. assistant (2026-07-02T06:11:33.264Z)

cv2/torch/numpy are all available here (the CLAUDE.md note is stale) and `skipped`/`vid` are in scope. The scripts themselves aren't unit-testable (hardcoded paths, `input()` prompts, argparse + real-video deps), so to get a *real* regression test I'll add a tiny generic helper to `backend/results.py` ? `unwrap_or_failure()` ? DRYing the unwrap pattern (duplicated 4× across main.py + these scripts) and making it unit-testable. Let me check for an existing results test and read the worktree's results.py.

240. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_call: Bash]
description: Check for results test and baseline script compile
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
ls tests/test_results.py 2>&1 | head -1
echo "=== existing results tests (if any) ==="
grep -rln "from backend.results\|import results\|unwrap" tests/ 2>/dev/null | head
echo "=== py_compile both scripts (baseline syntax OK) ==="
python -m py_compile scripts/run_phase3_eval.py scripts/run_process_and_stitch.py && echo
"compile OK"
[/external_agent_tool_call]
```

241. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_result]
ls: cannot access 'tests/test_results.py': No such file or directory
=== existing results tests (if any) ===
tests/test_base.py
=== py_compile both scripts (baseline syntax OK) ===
compile OK
[/external_agent_tool_result]
```

242. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
unwrap\backend\results.py
[/external_agent_tool_call]
```

243. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_result]
18      from __future__ import annotations
19
20      from dataclasses import dataclass, field
21      from typing import Generic, TypeVar, List, Optional, Any, Union
```

```
22
23      T = TypeVar("T")
24
25
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
[/external_agent_tool_result]
```

244. assistant (2026-07-02T06:11:33.264Z)

Now add the `unwrap_or_failure` helper to `backend/results.py`:

245. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
unwrap\backend\results.py
[/external_agent_tool_call]
```

246. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
unwrap\backend\results.py has been updated successfully. (file state is current in your context
? no need to Read it back)
[/external_agent_tool_result]
```

247. assistant (2026-07-02T06:11:33.264Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
```

unwrap\backend\results.py
[/external_agent_tool_call]

248. assistant (2026-07-02T06:11:33.264Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\backend\results.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

249. assistant (2026-07-02T06:11:33.264Z)

Now apply the unwrap in both scripts. First `run_phase3_eval.py`:

250. assistant (2026-07-02T06:11:33.264Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_phase3_eval.py
[/external_agent_tool_call]

251. assistant (2026-07-02T06:11:33.264Z)

[external_agent_tool_result: error]
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
[/external_agent_tool_result]

252. assistant (2026-07-02T06:11:33.264Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_phase3_eval.py
[/external_agent_tool_call]

253. assistant (2026-07-02T06:11:33.264Z)

[external_agent_tool_result]
```
34      from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
35      # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36      from backend.pipeline.segmenters import segmenter_registry
37      from backend.eval.annotations import load_annotations
38      from backend.eval.harness import evaluate, format_report_text, report_to_dict
39      from backend.eval import batch
```
[/external_agent_tool_result]

254. assistant (2026-07-02T06:11:33.264Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_phase3_eval.py
[/external_agent_tool_call]

255. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_phase3_eval.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

256. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_phase3_eval.py
[/external_agent_tool_call]

257. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_result]
```
142                 else:
143                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
144                     db.add_video(video_id, video_path, "processed", [])
145                     t0 = time.time()
146                     segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
147                     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
148
```
[/external_agent_tool_result]

258. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_phase3_eval.py
[/external_agent_tool_call]

259. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_phase3_eval.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

260. assistant (2026-07-02T06:11:33.265Z)

Now `run_process_and_stitch.py` ? import and call site:

261. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_process_and_stitch.py
[/external_agent_tool_call]

262. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_result]
```
15      from backend.config import load_config
16      from backend.utils.hashing import compute_video_id
17      from backend.pipeline.segmenters import segmenter_registry
18      from backend.pipeline.stitcher.compiler import VideoCompiler
19      from backend.tools.export import format_rally_summary
20
```
[/external_agent_tool_result]

263. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_process_and_stitch.py
[/external_agent_tool_call]

264. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_process_and_stitch.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

265. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_process_and_stitch.py
[/external_agent_tool_call]

266. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_result]
```
79                  return
80              print(f"Using segmenter: {seg_name}", flush=True)
81              segmenter = segmenter_cls(db, cfg)
82              segments = segmenter.process_video(video_id, video_path)
83              print(f"Successfully indexed {len(segments)} play segments.", flush=True)
84
85          if not segments:
86              print("No play segments detected. Cannot compile highlights.")
87              return
```
[/external_agent_tool_result]

267. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_process_and_stitch.py
[/external_agent_tool_call]

268. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\scripts\run_process_and_stitch.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

269. assistant (2026-07-02T06:11:33.265Z)

Now a focused unit test for the helper that also documents the original crash:

270. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_call: Write]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\tests\test_results.py
[/external_agent_tool_call]

271. assistant (2026-07-02T06:11:33.265Z)

[external_agent_tool_result]
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\tests\test_results.py (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

272. assistant (2026-07-02T06:11:33.266Z)

Now verify PR #2: py_compile, the new test, confirm imports resolve, then the full gate.

273. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_call: Bash]
description: Verify scripts compile/import and run new test
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
echo "=== __main__ guards present (safe to import)? ==="
grep -nE '__name__ == .__main__.' scripts/run_phase3_eval.py scripts/run_process_and_stitch.py
echo "=== py_compile both scripts ==="
python -m py_compile scripts/run_phase3_eval.py scripts/run_process_and_stitch.py && echo
"compile OK"
echo "=== imports resolve (new unwrap_or_failure import) ==="
python -c "import sys; sys.path.insert(0,'scripts'); import run_phase3_eval,
run_process_and_stitch; print('script imports OK')" 2>&1 | tail -3
echo "=== new results test ==="
python -m pytest tests/test_results.py -q 2>&1 | tail -4
[/external_agent_tool_call]
```

274. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_result]
=== __main__ guards present (safe to import)? ===
scripts/run_phase3_eval.py:180:if __name__ == "__main__":
scripts/run_process_and_stitch.py:160:if __name__ == "__main__":
=== py_compile both scripts ===
compile OK
=== imports resolve (new unwrap_or_failure import) ===
script imports OK
=== new results test ===
....                                                                     [100%]
4 passed in 0.07s
[/external_agent_tool_result]
```

275. assistant (2026-07-02T06:11:33.266Z)

Both scripts compile and import cleanly; 4 new tests pass. Since cv2 is available, let me also do a quick real-segmenter sanity check (motion on a bogus path ? Failure flows through the helper), then run the full gate.

276. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_call: Bash]
description: Integration sanity check + ruff + mypy
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
echo "=== integration sanity: real motion segmenter Failure flows through unwrap_or_failure ==="
python - <<'PY' 2>&1 | tail -5
import tempfile, os
from backend.infrastructure.database import Database
from backend.config import load_config
from backend.pipeline.segmenters import segmenter_registry
from backend.results import unwrap_or_failure, Failure
db = Database(os.path.join(tempfile.mkdtemp(), "t.db"))
cfg = load_config()
seg = segmenter_registry.get("motion")(db, cfg)
res = seg.process_video("deadbeef", "/no/such/video.mp4")   # cv2 can't open -> Failure
segs, failure = unwrap_or_failure(res)
print("raw result type:", type(res).__name__)
print("unwrapped -> segs:", segs, "| failure is Failure:", isinstance(failure, Failure))
assert segs is None and isinstance(failure, Failure), "expected Failure path"
print("OK: motion Failure handled cleanly (no TypeError, guard fires)")
PY
echo "=== ruff + mypy ==="
python -m ruff check . 2>&1 | tail -2
```

```
python -m mypy backend training 2>&1 | tail -2
[/external_agent_tool_call]
```

277. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_result]
=== integration sanity: real motion segmenter Failure flows through unwrap_or_failure ===
Segmenter could not open video: /no/such/video.mp4
raw result type: Failure
unwrapped -> segs: None | failure is Failure: True
OK: motion Failure handled cleanly (no TypeError, guard fires)
=== ruff + mypy ===
All checks passed!
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 104 source files
[/external_agent_tool_result]
```

278. assistant (2026-07-02T06:11:33.266Z)

```
Integration check confirms the fix works against the **real** motion segmenter (Failure flows
through cleanly, no TypeError). ruff + mypy clean. Now the full suite + coverage:
```

279. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_call: Bash]
description: Full suite with coverage for PR#2
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70 2>&1
| tail -6
[/external_agent_tool_call]
```

280. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_result]
TOTAL                                                 7990   1544    81%
Required test coverage of 70% reached. Total coverage: 80.68%
850 passed, 7 skipped, 7 warnings in 86.14s (0:01:26)
======================================================================
 RESULT: PASS - all tests green
======================================================================
[/external_agent_tool_result]
```

281. assistant (2026-07-02T06:11:33.266Z)

```
All green: 850 passed, 7 skipped, coverage 80.68%. Let me review the PR #2 diff and commit.
```

282. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_call: Bash]
description: Review PR#2 diff
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
git diff --stat
echo "============ full diff ============"
git diff -- backend/results.py scripts/run_phase3_eval.py scripts/run_process_and_stitch.py
[/external_agent_tool_call]
```

283. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_result]
 backend/results.py                | 20 ++++++++++++++++++-
 scripts/run_phase3_eval.py        | 11 ++++++++++-
 scripts/run_process_and_stitch.py |  9 ++++++++-
 3 files changed, 37 insertions(+), 3 deletions(-)
```

```
============ full diff ============
diff --git a/backend/results.py b/backend/results.py
index d6b1ea4..3c354c7 100644
--- a/backend/results.py
+++ b/backend/results.py
@@ -18,7 +18,7 @@ This module has no dependencies on the rest of the app.
 from __future__ import annotations

 from dataclasses import dataclass, field
-from typing import Generic, TypeVar, List, Optional, Any, Union
+from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union

 T = TypeVar("T")

@@ -58,3 +58,21 @@ class Failure:
 # Result[ValueType] means Success[ValueType] | Failure
 # (the Failure "error" type is fixed for this contract)
 Result = Union[Success[T], Failure]
+
+
+def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
+    """Split a segmenter/provider return into ``(value, failure)`` so a caller branches once.
+
+    - ``Failure``            -> ``(None, failure)``
+    - ``Success(value=v)``   -> ``(v, None)``
+    - a bare value (legacy code that predates this contract and still returns a raw list)
+      -> ``(value, None)``
+
+    This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline; callers
+    that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
+    on a ``Success`` dataclass. ``input`` is typed ``Any`` because the contract is
heterogeneous
+    (Success | Failure | legacy bare value)."""
+    if isinstance(result, Failure):
+        return None, result
+    value = result.value if hasattr(result, "value") else result
+    return value, None
diff --git a/scripts/run_phase3_eval.py b/scripts/run_phase3_eval.py
index 56e21ca..dd1263e 100644
--- a/scripts/run_phase3_eval.py
+++ b/scripts/run_phase3_eval.py
@@ -34,6 +34,7 @@ from backend.config import load_config
 from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
 # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
 from backend.pipeline.segmenters import segmenter_registry
+from backend.results import unwrap_or_failure
 from backend.eval.annotations import load_annotations
 from backend.eval.harness import evaluate, format_report_text, report_to_dict
 from backend.eval import batch
@@ -142,7 +143,15 @@ def main(argv=None) -> int:
                 print(f"[proc] {vid}: segmenting with {args.segmenter}...")
                 db.add_video(video_id, video_path, "processed", [])
                 t0 = time.time()
-                segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
+                # process_video returns a Result (Success|Failure) for some segmenters
+                # (e.g. the default `motion`); unwrap before len() so this doesn't crash with
+                # TypeError on a Success dataclass ? mirrors the unwrap in backend/main.py.
+                segs, failure = unwrap_or_failure(
+                    segmenter.process_video(video_id, video_path, max_frames=args.max_frames))
+                if failure is not None:
+                    print(f"        -> FAILED: {failure.message}")
+                    skipped.append((vid, f"segment failed: {failure.message}"))
+                    continue
```

```
                print(f"       -> {len(segs)} segments in {time.time() - t0:.0f}s")

        gt = load_annotations(csv_path)
diff --git a/scripts/run_process_and_stitch.py b/scripts/run_process_and_stitch.py
index b8ffef6..ea7c2de 100644
--- a/scripts/run_process_and_stitch.py
+++ b/scripts/run_process_and_stitch.py
@@ -16,6 +16,7 @@ from backend.config import load_config
 from backend.utils.hashing import compute_video_id
 from backend.pipeline.segmenters import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
+from backend.results imp...
[/external_agent_tool_result]
```

284. assistant (2026-07-02T06:11:33.266Z)

Small docstring typo ? the param is `result`, not `input`. Let me fix it.

285. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
unwrap\backend\results.py
[/external_agent_tool_call]
```

286. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-
unwrap\backend\results.py has been updated successfully. (file state is current in your context
? no need to Read it back)
[/external_agent_tool_result]
```

287. assistant (2026-07-02T06:11:33.266Z)

Now commit, push, and open PR #2:

288. assistant (2026-07-02T06:11:33.266Z)

```
[external_agent_tool_call: Bash]
description: Commit PR#2 and verify clean branch
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
git branch --show-current && git log --oneline -1
git add backend/results.py scripts/run_phase3_eval.py scripts/run_process_and_stitch.py
tests/test_results.py
git status --porcelain
git commit -F - <<'EOF'
fix(scripts): unwrap the segmenter Result before len()/truthiness (motion crashes the default
path)

`SegmenterABC.process_video` returns a `Result` (Success|Failure). `backend/main.py`
unwraps it (isinstance(Failure) ? handle; else `.value`), but two standalone
scripts use the return directly:

- `scripts/run_phase3_eval.py` does `len(segmenter.process_video(...))`. The
  `--segmenter` default is `motion` (also the documented baseline command), and
  motion returns `Success(value=...)` / `Failure(...)`. Success/Failure are plain
  dataclasses with no `__len__`, so the documented first-run command crashes with
  `TypeError: object of type 'Success' has no len()` before any scoring.
- `scripts/run_process_and_stitch.py` does the same `len()` plus an
  `if not segments:` empty-guard that is dead code on a dataclass (always False),
  so an empty/failed result would silently fall through to stitching.

The bare-list segmenters (gemini/yolo_hybrid/trajectory_hybrid) masked the bug.
```

```
Fix: add a small generic `backend/results.unwrap_or_failure(result) ->
(value, failure)` helper (the same unwrap main.py does inline, now DRY + unit-
tested) and use it in both scripts, handling Failure explicitly. No behavior change
for the bare-list segmenters; the empty-guard now sees a real list.

Tests: `tests/test_results.py` covers the helper (Success / Failure / legacy bare
value) and pins the original `len(Success)` TypeError. Also verified against the
REAL motion segmenter: `process_video` on a bogus path returns a Failure that flows
through the helper cleanly (no TypeError).

Local: full suite 850 passed / 7 skipped, ruff clean, mypy clean (104 files),
coverage 80.68% (floor 70). Caveat: the scripts need GPU/cv2 to run a full motion
pass E2E (owner-side); the fix correctness is covered by the unit test + the real-
segmenter Failure-path check + parity with the tested backend/main.py pattern.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
git log --oneline origin/master..HEAD
git diff --stat origin/master..HEAD
[/external_agent_tool_call]
```

289. assistant (2026-07-02T06:11:33.267Z)

```
[external_agent_tool_result]
fix/scripts-unwrap-segmenter-result
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
warning: in the working copy of 'tests/test_results.py', LF will be replaced by CRLF the next
time Git touches it
M  backend/results.py
M  scripts/run_phase3_eval.py
M  scripts/run_process_and_stitch.py
A  tests/test_results.py
[fix/scripts-unwrap-segmenter-result 03d11d9] fix(scripts): unwrap the segmenter Result before
len()/truthiness (motion crashes the default path)
 4 files changed, 76 insertions(+), 3 deletions(-)
 create mode 100644 tests/test_results.py
03d11d9 fix(scripts): unwrap the segmenter Result before len()/truthiness (motion crashes the
default path)
 backend/results.py                | 20 ++++++++++++++++++-
 scripts/run_phase3_eval.py        | 11 ++++++++++-
 scripts/run_process_and_stitch.py |  9 ++++++++-
 tests/test_results.py             | 39 ++++++++++++++++++++++++++++++++++++++++
 4 files changed, 76 insertions(+), 3 deletions(-)
[/external_agent_tool_result]
```

290. assistant (2026-07-02T06:11:33.267Z)

```
Clean single commit. Push and open PR #2:
```

291. assistant (2026-07-02T06:11:33.267Z)

```
[external_agent_tool_call: Bash]
description: Push and open PR#2
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
git push -u origin fix/scripts-unwrap-segmenter-result 2>&1 | tail -3
cat > /tmp/pr2_body.md <<'EOF'
```

Problem

```
`SegmenterABC.process_video` returns a `Result` (`Success | Failure`). `backend/main.py` unwraps
it (`isinstance(result, Failure)` ? handle; else `result.value if hasattr(result, "value") else
result`), but two standalone scripts consume the return directly:

- **`scripts/run_phase3_eval.py`** ? `len(segmenter.process_video(...))`. The `--segmenter`
default is **`motion`** (also the documented "Free CPU baseline" command), and motion returns
```

`Success(value=...)` / `Failure(...)`. `Success`/`Failure` are plain dataclasses with **no `__len__`**, so the documented first-run command crashes with `TypeError: object of type 'Success' has no len()` before any scoring.
- **`scripts/run_process_and_stitch.py`** ? same `len()` crash, plus an `if not segments:` empty-guard that is **dead code** on a dataclass (always False), so an empty/failed result would silently fall through to stitching.

The bare-list segmenters (gemini / yolo_hybrid / trajectory_hybrid) masked the bug.

Fix

Add a small generic helper `backend/results.unwrap_or_failure(result) -> (value, failure)` ? the same unwrap `main.py` does inline, now DRY and unit-tested ? and use it in both scripts, handling `Failure` explicitly. No behavior change for the bare-list segmenters; the empty-guard now operates on a real list.

Tests

- `tests/test_results.py` covers the helper (`Success` / `Failure` / legacy bare value) and pins the original `len(Success)` `TypeError`.
- Verified against the **real** motion segmenter: `process_video` on a bogus path returns a `Failure` that flows through the helper cleanly (no `TypeError`, guard fires).

Verification (local)

- Full suite: **850 passed, 7 skipped**
- `ruff check .` clean · `mypy backend training` clean (104 files)
- Coverage **80.68%** (`--cov-fail-under=70`)

Caveat: a full `motion` pass needs GPU/cv2 to run E2E (owner-side). The fix correctness here is covered by the unit test + the real-segmenter Failure-path check + parity with the already-tested `backend/main.py` unwrap. Found by an adversarial bug-hunt after [#209](https://github.com/avidullu/badminton-highlight-indexer/pull/209).

? Generated with [Claude Code](https://claude.com/claude-code)
EOF
gh pr create --base master --head fix/scripts-unwrap-segmenter-result \
  --title "fix(scripts): unwrap segmenter Result before len() (motion crashes the default eval path)" \
  --body-file /tmp/pr2_body.md 2>&1 | tail -3
[/external_agent_tool_call]

292. assistant (2026-07-02T06:11:33.267Z)

[external_agent_tool_result]
branch 'fix/scripts-unwrap-segmenter-result' set up to track 'origin/fix/scripts-unwrap-segmenter-result'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]   fix/scripts-unwrap-segmenter-result -> fix/scripts-unwrap-segmenter-result
https://github.com/avidullu/badminton-highlight-indexer/pull/213
[/external_agent_tool_result]

293. assistant (2026-07-02T06:11:33.267Z)

Both PRs open: **#212** (DB migration) and **#213** (scripts unwrap). Now the same due-diligence as before ? adversarial review of each diff plus CI. Let me launch a combined review workflow and check both PRs' CI in parallel.

294. assistant (2026-07-02T06:11:33.267Z)

[external_agent_tool_call: Workflow]
description: Adversarial review of PR#212 and PR#213
[/external_agent_tool_call]

295. assistant (2026-07-02T06:11:33.267Z)

```
[external_agent_tool_result]
Workflow launched in background. Task ID: wz4qtf4k3
Summary: Adversarial pre-merge review of the DB-migration and scripts-unwrap fix PRs
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e3b53aed-f4e
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer
--claude-worktrees-scripts-unwrap\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\review-
two-fix-prs-wf_e3b53aed-f4e.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-
worktrees-scripts-unwrap\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\review-two-fix-
prs-wf_e3b53aed-f4e.js"} to iterate without resending the script.)
Run ID: wf_e3b53aed-f4e
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-scripts-
unwrap\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\review-two-fix-prs-
wf_e3b53aed-f4e.js", resumeFromRunId: "wf_e3b53aed-f4e"}) ? completed agents return cached
results.

You will be notified when it completes. Use /workflows to watch live progress.
[/external_agent_tool_result]
```

296. assistant (2026-07-02T06:11:33.267Z)

```
[external_agent_tool_call: Bash]
description: Check CI status for both PRs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
for pr in 212 213; do
  echo "=== PR #$pr checks ==="
  gh pr checks $pr 2>&1 | head -10
  echo "--- mergeable ---"
  gh pr view $pr --json mergeStateStatus,mergeable | head -3
done
[/external_agent_tool_call]
```

297. assistant (2026-07-02T06:11:33.267Z)

```
[external_agent_tool_result]
=== PR #212 checks ===
Full test suite (offline, mocked)       pass    2m9s    https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27731509959/job/82039380259
Golden fixtures cross-OS (macos-latest) pass    16s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27731509959/job/82039380287
Golden fixtures cross-OS (ubuntu-latest)        pass    18s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731509959/job/82039380330
Golden fixtures cross-OS (windows-latest)       pass    51s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731509959/job/82039380303
Mypy (lenient baseline) pass    34s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27731509959/job/82039380268
Ruff lint       pass    12s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27731509959/job/82039380297
Syntax sweep + import smoke (no GPU stack)      pass    18s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731509959/job/82039380313
--- mergeable ---
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}
=== PR #213 checks ===
Full test suite (offline, mocked)       pending 0       https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27731763134/job/82040168742
Golden fixtures cross-OS (windows-latest)       pending 0
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731763134/job/82040168669
Golden fixtures cross-OS (macos-latest) pass    19s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27731763134/job/82040168719
Golden fixtures cross-OS (ubuntu-latest)        pass    22s
```

https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731763134/job/82040168701
Mypy (lenient baseline) pass 30s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731763134/job/82040168694
Ruff lint pass 9s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731763134/job/82040168654
Syntax sweep + import smoke (no GPU stack) pass 18s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27731763134/job/82040168703
--- mergeable ---
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}
[/external_agent_tool_result]

298. assistant (2026-07-02T06:11:33.267Z)

PR #212 CI is fully green (CLEAN); PR #213 has the full suite still running. The review workflow is running. Let me wait for the review verdicts.

299. assistant (2026-07-02T06:11:33.267Z)

[external_agent_tool_call: TaskOutput]
input: {"task_id":"wz4qtf4k3","block":true,"timeout":420000}
[/external_agent_tool_call]

300. assistant (2026-07-02T06:11:33.267Z)

[external_agent_tool_result]
<retrieval_status>timeout</retrieval_status>

<task_id>wz4qtf4k3</task_id>

<task_type>local_workflow</task_type>

<status>running</status>
[/external_agent_tool_result]

301. assistant (2026-07-02T06:11:33.267Z)

Still running. Let me continue waiting for the review.

302. assistant (2026-07-02T06:11:33.267Z)

[external_agent_tool_call: TaskOutput]
input: {"task_id":"wz4qtf4k3","block":true,"timeout":420000}
[/external_agent_tool_call]

303. assistant (2026-07-02T06:11:33.267Z)

[external_agent_tool_result]
<retrieval_status>success</retrieval_status>

<task_id>wz4qtf4k3</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

```
{
  "summary": "Adversarial pre-merge review of the DB-migration and scripts-unwrap fix PRs",
  "agentCount": 10,
  "logs": [
    "gatekeeper verdicts: pr212-db-migration=merge · pr213-scripts-unwrap=merge"
  ],
  "result": {
    "verdicts": [
```

```
      {
        "pr": "pr212-db-migration",
        "verdict": "merge",
        "must_fixes": [],
        "should_fixes": [
          "Optional (non-blocking) test coverage: the regression test only exercises the all-
columns-present interruption. Consider parametrizing/adding a case for a partial interruption
(only one of v4's two ALTERs landed) to pin the per-statement granularity. I independently
verified this partial case already re-applies cleanly to v5, so it is a coverage nicety, not a
masked bug. tests/test_database.py:247-279.",
          "Optional: backend/infrastructure/database.py:60 (the `raise` re-raise branch in
_add_column_idempotent) is the one new line not directly asserted by a test. I confirmed it
works (a 'no such table' OperationalError correctly propagates), but a tiny test asserting non-
duplicate errors re-raise would close the last gap."
        ],
        "rationale": "Independently verified, not rubber-stamped. The bug is real and the fix is
correct, surgical, and well-tested. (1) Scope: single commit d140722, only database.py (new
_add_column_idempotent static helper wired at the four v3/v4 ALTER sites) + test_database.py; it
is the only commit ahead of origin/master. The #213/results.py/main.py/scripts-unwrap clause in
one lens is a template carry-over and genuinely does not apply. (2) Bug confirmed: reverting
database.py to origin/master makes the new test fail with exactly `sqlite3.OperationalError:
duplicate column name: policy`; the fix makes it pass. The ladder bumps PRAGMA user_version at
the END of each `if version < N` block, so a crash after an auto-committed ALTER but before the
bump bricks every future Database() open ? the documented failure mode. (3) Swallow is correctly
narrow: on SQLite 3.50.4 I reproduced `duplicate column name: b` (matched, swallowed) vs `no
such table: nope` (not matched, re-raised), so unrelated/corrupt-DB errors still surface. (4)
Partial mid-v4 interruption (only one of two ALTERs landed) re-applies cleanly to v5 ? verified,
confirming the repeated reviewer nit is non-blocking. (5) Pre-LGTM gate: full suite 847 passed /
7 skipped, ruff clean and mypy clean on both touched files; PR's 80.67% coverage vs 70 floor is
consistent with the green full suite. Happy path is byte-identical (NULL=local behavior
unchanged, version contract tests green). The only findings across all four lenses are optional
test-coverage nits, no genuine must-fix."
      },
      {
        "pr": "pr213-scripts-unwrap",
        "verdict": "merge",
        "must_fixes": [],
        "should_fixes": [
          "Prose accuracy (non-blocking, no code change): the commit subject \"motion crashes
the default path\" and tests/test_results.py:32-33 docstring imply both scripts default to
motion. Only scripts/run_phase3_eval.py:54 defaults to motion (a real default-path crash);
scripts/run_process_and_stitch.py uses cfg.indexing.default_segmenter which defaults to
yolo_hybrid (bare list, no crash) ? its crash is latent, only under motion config. The inline
comment at run_process_and_stitch.py:82 ('for some segmenters (e.g. motion)') is already
correct.",
          "Optional follow-up (out of scope): fold the now-four hand-rolled Success/Failure
unwraps (backend/main.py:201/213 and :838/843, tools/generate_highlights.py:168) onto
unwrap_or_failure so there is a single unwrap implementation. tools/generate_highlights.py is
correct as-is (handles Failure, reads segments from the DB, never len()s the raw result).",
          "Optional test addition: pin unwrap_or_failure(Success(value=[])) -> ([], None)
explicitly...
[/external_agent_tool_result]
```


304. assistant (2026-07-02T06:11:33.267Z)

Both PRs: **verdict=merge, 0 must-fixes** ? each independently re-verified (reverting database.py reproduces the exact `duplicate column name`; SQLite error-matching is correctly narrow; the scripts' default-segmenter analysis).

The reviews surfaced no blockers, but two cheap, legitimate improvements worth addressing before merge (my gate: resolve review findings):
1. **PR #213 accuracy fix (important per my no-overclaim rule):** the test docstring implies *both* scripts default to `motion`, but only `run_phase3_eval` does ? `run_process_and_stitch` defaults to `yolo_hybrid` (its crash is latent under a motion config). I'll correct that and add the `Success(value=[])` ? `([], None)` assertion the empty-guard depends on.

2. **PR #212:** add a tiny test for the re-raise branch (the one uncovered new line).

Let me make those small strengthening edits. First PR #213:

305. assistant (2026-07-02T06:11:33.267Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\tests\test_results.py
[/external_agent_tool_call]

306. assistant (2026-07-02T06:11:33.267Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\scripts-unwrap\tests\test_results.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

307. assistant (2026-07-02T06:11:33.267Z)

Now PR #212 ? add the re-raise branch test. Let me check its test imports and add the test:

308. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py
[/external_agent_tool_call]

309. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_result]
1	import sqlite3
2	from backend.infrastructure.database import Database
3
[/external_agent_tool_result]

310. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py
[/external_agent_tool_call]

311. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

312. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py
[/external_agent_tool_call]

313. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_result]

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\db-migration-fix\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

314. assistant (2026-07-02T06:11:33.268Z)

Run the targeted tests + lint on both worktrees:

315. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_call: Bash]
description: PR#212 targeted tests + ruff
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-fix
echo "=== PR#212: test_database.py + ruff ==="
python -m pytest tests/test_database.py -q 2>&1 | tail -3
python -m ruff check tests/test_database.py 2>&1 | tail -2
[/external_agent_tool_call]

316. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_call: Bash]
description: PR#213 targeted tests + ruff
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
echo "=== PR#213: test_results.py + ruff ==="
python -m pytest tests/test_results.py -q 2>&1 | tail -3
python -m ruff check tests/test_results.py 2>&1 | tail -2
[/external_agent_tool_call]

317. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_result]
=== PR#212: test_database.py + ruff ===
..... [100%]
21 passed in 0.92s
All checks passed!
[/external_agent_tool_result]

318. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_result]
=== PR#213: test_results.py + ruff ===
..... [100%]
5 passed in 0.08s
All checks passed!
[/external_agent_tool_result]

319. assistant (2026-07-02T06:11:33.268Z)

Both targeted suites green (+1 test each). Now the full gate on both (the follow-ups are test-only additions, but I'll match CI):

320. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_call: Bash]
description: PR#212 full suite + lint + type + coverage
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-fix
echo "########## PR#212 full gate ##########"
python -m ruff check . 2>&1 | tail -1
python -m mypy backend training 2>&1 | tail -1
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70 2>&1 | tail -4

[/external_agent_tool_call]

321. assistant (2026-07-02T06:11:33.268Z)

```
[external_agent_tool_result]
########## PR#212 full gate ##########
All checks passed!
Success: no issues found in 104 source files
848 passed, 7 skipped, 7 warnings in 109.28s (0:01:49)
========================================================================
 RESULT: PASS - all tests green
========================================================================
[/external_agent_tool_result]
```

322. assistant (2026-07-02T06:11:33.268Z)

```
[external_agent_tool_call: Bash]
description: PR#213 full suite + lint + type + coverage
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
echo "########## PR#213 full gate ##########"
python -m ruff check . 2>&1 | tail -1
python -m mypy backend training 2>&1 | tail -1
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70 2>&1
| tail -4
[/external_agent_tool_call]
```

323. assistant (2026-07-02T06:11:33.268Z)

```
[external_agent_tool_result]
########## PR#213 full gate ##########
All checks passed!
Success: no issues found in 104 source files
851 passed, 7 skipped, 7 warnings in 93.69s (0:01:33)
========================================================================
 RESULT: PASS - all tests green
========================================================================
[/external_agent_tool_result]
```

324. assistant (2026-07-02T06:11:33.268Z)

```
Both full gates green (PR #212: 848 passed; PR #213: 851 passed; ruff + mypy clean on each). Let
me commit the follow-ups, push (CI re-runs), then merge each on green. First PR #212:
```

325. assistant (2026-07-02T06:11:33.268Z)

```
[external_agent_tool_call: Bash]
description: Commit and push PR#212 follow-up
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-
fix
git branch --show-current
git add tests/test_database.py
git status --porcelain
git commit -F - <<'EOF'
test(db): cover the idempotent-ALTER re-raise branch

Adds test_add_column_idempotent_swallows_only_duplicate ? asserts the helper
swallows a re-applied 'duplicate column name' but still PROPAGATES any other
OperationalError (e.g. a missing table), so the narrow swallow can't mask a
genuinely broken migration. Closes the one uncovered new line from the review.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
git log --oneline origin/master..HEAD
git push 2>&1 | tail -2
```

[/external_agent_tool_call]

326. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_result]
fix/db-migration-idempotent-alters
M tests/test_database.py
[fix/db-migration-idempotent-alters e720483] test(db): cover the idempotent-ALTER re-raise
branch
 1 file changed, 17 insertions(+)
e720483 test(db): cover the idempotent-ALTER re-raise branch
d140722 fix(db): make v3/v4 migration ALTERs idempotent so an interrupted upgrade can't brick
the DB
To https://github.com/avidullu/badminton-highlight-indexer.git
   d140722..e720483 fix/db-migration-idempotent-alters -> fix/db-migration-idempotent-alters
[/external_agent_tool_result]

327. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_call: Bash]
description: Commit and push PR#213 follow-up
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap
git branch --show-current
git add tests/test_results.py
git status --porcelain
git commit -F - <<'EOF'
test(results): pin empty-Success unwrap + correct the motion-default prose

- Add test_unwrap_empty_success_yields_empty_list ? Success(value=[]) -> ([], None),
  the path run_process_and_stitch's `if not segments` empty-guard depends on.
- Fix the regression-test docstring accuracy: only run_phase3_eval DEFAULTS to the
  motion segmenter; run_process_and_stitch hits the same crash only when CONFIGURED
  to use motion (its default is yolo_hybrid, a bare list). No over-claim.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
git log --oneline origin/master..HEAD
git push 2>&1 | tail -2
[/external_agent_tool_call]

328. assistant (2026-07-02T06:11:33.268Z)

[external_agent_tool_result]
fix/scripts-unwrap-segmenter-result
warning: in the working copy of 'tests/test_results.py', LF will be replaced by CRLF the next
time Git touches it
M tests/test_results.py
[fix/scripts-unwrap-segmenter-result 11d779f] test(results): pin empty-Success unwrap + correct
the motion-default prose
 1 file changed, 13 insertions(+), 2 deletions(-)
11d779f test(results): pin empty-Success unwrap + correct the motion-default prose
03d11d9 fix(scripts): unwrap the segmenter Result before len()/truthiness (motion crashes the
default path)
To https://github.com/avidullu/badminton-highlight-indexer.git
   03d11d9..11d779f fix/scripts-unwrap-segmenter-result -> fix/scripts-unwrap-segmenter-result
[/external_agent_tool_result]

329. assistant (2026-07-02T06:11:33.268Z)

Both follow-ups pushed; CI re-runs. Let me watch PR #212 to green.

330. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_call: Bash]

```
description: Watch PR#212 CI to completion
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
gh pr checks 212 --watch --interval 15 2>&1 | tail -10
echo "=== exit $? ==="
gh pr view 212 --json mergeStateStatus,mergeable | head -3
[/external_agent_tool_call]
```

331. assistant (2026-07-02T06:11:33.269Z)

```
[external_agent_tool_result]
Mypy (lenient baseline) pass    26s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732272522/job/82041727034
Ruff lint       pass    9s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732272522/job/82041726999
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732272522/job/82041727031
Full test suite (offline, mocked)       pass    2m7s    https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27732272522/job/82041727015
Golden fixtures cross-OS (macos-latest) pass    19s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27732272522/job/82041727058
Golden fixtures cross-OS (ubuntu-latest)        pass    21s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732272522/job/82041727080
Golden fixtures cross-OS (windows-latest)       pass    52s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732272522/job/82041727053
Mypy (lenient baseline) pass    26s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732272522/job/82041727034
Ruff lint       pass    9s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27732272522/job/82041726999
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732272522/job/82041727031
=== exit 0 ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}
[/external_agent_tool_result]
```

332. assistant (2026-07-02T06:11:33.269Z)

```
PR #212 fully green (CLEAN) and review verdict was merge/0-must-fix. Merging it (squash):
```

333. assistant (2026-07-02T06:11:33.269Z)

```
[external_agent_tool_call: Bash]
description: Squash-merge PR#212
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
gh pr merge 212 --squash --delete-branch 2>&1 | tail -4
echo "=== verify ==="
gh pr view 212 --json state,mergeCommit | head -3
git fetch origin master 2>&1 | tail -1
git log --oneline -1 origin/master
[/external_agent_tool_call]
```

334. assistant (2026-07-02T06:11:33.269Z)

```
[external_agent_tool_result]
failed to delete local branch fix/db-migration-idempotent-alters: failed to run git: error:
cannot delete branch 'fix/db-migration-idempotent-alters' used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-fix'

=== verify ===
{"mergeCommit":{"oid":"c7baf03217a2d78625e0e58fa90b79d90daf233c"},"state":"MERGED"}
   4bd9d2d..c7baf03  master     -> origin/master
c7baf03 fix(db): make v3/v4 migration ALTERs idempotent (interrupted upgrade can brick the DB)
(#212)
[/external_agent_tool_result]
```

335. assistant (2026-07-02T06:11:33.269Z)

PR **#212 MERGED** (`c7baf03`), master advanced. Now PR #213 ? watch its CI (re-ran after the follow-up) to green:

336. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_call: Bash]
description: Watch PR#213 CI to completion
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
gh pr checks 213 --watch --interval 15 2>&1 | tail -9
echo "=== exit $? ==="
gh pr view 213 --json mergeStateStatus,mergeable | head -3
[/external_agent_tool_call]

337. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_result]
Ruff lint pass 11s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746755
Syntax sweep + import smoke (no GPU stack) pass 24s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746754
Full test suite (offline, mocked) pass 2m22s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746771
Golden fixtures cross-OS (macos-latest) pass 15s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746773
Golden fixtures cross-OS (ubuntu-latest) pass 21s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746756
Golden fixtures cross-OS (windows-latest) pass 47s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746779
Mypy (lenient baseline) pass 31s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746777
Ruff lint pass 11s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746755
Syntax sweep + import smoke (no GPU stack) pass 24s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27732279062/job/82041746754
=== exit 0 ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}
[/external_agent_tool_result]

338. assistant (2026-07-02T06:11:33.269Z)

PR #213 fully green (CLEAN). Merging it (squash):

339. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_call: Bash]
description: Squash-merge PR#213
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
gh pr merge 213 --squash --delete-branch 2>&1 | tail -3
echo "=== verify ==="
gh pr view 213 --json state,mergeCommit | head -3
git fetch origin master 2>&1 | tail -1
git log --oneline -3 origin/master
[/external_agent_tool_call]

340. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_result]
failed to delete local branch fix/scripts-unwrap-segmenter-result: failed to run git: error: cannot delete branch 'fix/scripts-unwrap-segmenter-result' used by worktree at 'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap'

=== verify ===

{"mergeCommit":{"oid":"bd330855302b92be57179b30443d32335a0d2a52"},"state":"MERGED"}
   c7baf03..bd33085 master   -> origin/master
bd33085 fix(scripts): unwrap segmenter Result before len() (motion crashes the default eval
path) (#213)
c7baf03 fix(db): make v3/v4 migration ALTERs idempotent (interrupted upgrade can brick the DB)
(#212)
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
[/external_agent_tool_result]

341. assistant (2026-07-02T06:11:33.269Z)

Both PRs **MERGED** ? master is now `bd33085 (#213)` ? `c7baf03 (#212)` ? `4bd9d2d (#209)`. Let
me clean up the worktrees/branches and check on the chip PR #210.

342. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_call: Bash]
description: Clean up worktrees/branches, check chip PR
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
for wt in db-migration-fix scripts-unwrap hunt-base; do
  git worktree remove .claude/worktrees/$wt --force 2>&1 | tail -1 && echo "removed $wt" || echo
"leftover dir: $wt (harmless)"
done
git worktree prune
git branch -D fix/db-migration-idempotent-alters fix/scripts-unwrap-segmenter-result 2>&1 | tail
-2
echo "=== chip PR #210 status ==="
gh pr view 210 --json state,mergeStateStatus,title 2>&1 | head -4
echo "=== remaining fix/* worktrees (should be none of mine) ==="
git worktree list | grep -iE "db-migration|scripts-unwrap|hunt-base|ablation-schema" || echo
"(all my worktrees cleaned)"
[/external_agent_tool_call]

343. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_result]
removed db-migration-fix
removed scripts-unwrap
removed hunt-base
Deleted branch fix/db-migration-idempotent-alters (was e720483).
Deleted branch fix/scripts-unwrap-segmenter-result (was 11d779f).
=== chip PR #210 status ===
{"mergeStateStatus":"UNKNOWN","state":"OPEN","title":"fix(eval): fusion loaders skip windowing-
only files on a mixed golden corpus"}
=== remaining fix/* worktrees (should be none of mine) ===
(all my worktrees cleaned)
[/external_agent_tool_result]

344. assistant (2026-07-02T06:11:33.269Z)

All my worktrees and branches are cleaned; the chip PR #210 remains its session's
responsibility. Let me record this in the live checkpoint.

345. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md
[/external_agent_tool_call]

346. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_result: error]
<tool_use_error>File has been modified since read, either by the user or by a linter. Read it

again before attempting to write it.</tool_use_error>
[/external_agent_tool_result]

347. assistant (2026-07-02T06:11:33.269Z)

The memory file changed since I read it (likely another session). Let me re-read the top and re-apply.

348. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

349. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_result]
1 ---
2 name: current-state
3 description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not only on request). What just landed, what's next, open decisions."
4 metadata:
5 node_type: memory
6 type: project
7 originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8 ---
9
10 # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12 **Checkpoint:** 2026-06-18 (? **ABLATION LITMUS MIXED-SCHEMA FIX ? MERGED #209 (`4bd9d2d`), issue #208 closed**). The pre-existing `test_ablation.py` litmus `KeyError: 'shuttle'` (flagged 2026-06-17 as OUT-OF-SCOPE `task_5878d224`) is FIXED + shipped to master. Root cause: `ablation.load_videos()` read `c["shuttle"]`/`c["person"]` unconditionally, so once R3/R4 corpus growth added 5 clips persisted via the velocity-windowing/distill path (leaner rows, no cue dicts: Badminton_BXH_2, GX010128, GX030094, mahadevpura_1, mahadevpura_2) it crashed whenever the full n=11 corpus sat in `output/` (the 6 original fusion-schema files: adarsh_avi_singles, gbaaddy, kushagra_singles, largetest_doubles, mahadevpura_singles, testlarge_short). Fix (option a, most-robust): `_is_fusion_schema()` predicate ? loader SKIPS leaner files (info-log, no crash); `_golden_present()` counts only fusion files so the skip-gate matches the loader. Litmus still reproduces Gate-A (?F1?0.074, 6/6, p?0.031 ? the 6 fusion files ARE the original n=6). +2 regression tests. **DUE DILIGENCE:** local full suite 848 pass/5 skip + ruff + mypy(104) + cov 80.66% (floor 70); **GitHub CI all 8 checks green**; **adversarial 4-lens review workflow ? verdict=merge, 0 must-fix** (independently re-verified the bug, the litmus reproduction, the 2-file scope). Owner reported issue #208 + opened PR #209 + squash-merged on green. Worktree cleaned; local+remote branch deleted. ? **Sibling latent bug still open (`task_8dd4efde`):** `fusion_compare.load_videos` + `fusion_audio.load_videos_with_audio` have the IDENTICAL unconditional `c["shuttle"]` access ? not caught by any test (synthetic tmp corpora only) but would crash the same CLIs on the mixed corpus. Separate PR (the review independently confirmed this scope call). Possible follow-up nit: litmus `>=6`-gate vs `==6`-assert would go red if a 7TH fusion clip is ever added (pre-existing; this fix improved the opposite drift). [[eval-dual-set-regression]] [[working-agreement-merging]]
13
14 **Checkpoint:** 2026-06-18 ~00:10 (? **REMAINING-3 4K REELS DONE ? ALL 6 BOXHILL-17 CLIPS NOW HAVE 4K** + cost-scaling doc + 10/10 visual QA ? all on G:). Owner asked for "just 4K highlights of the remaining 3, all local no Gemini." Delivered:
15 - ? **GX010138/139/140 4K reels** (the larger 9/10/12.8-min clips): **45/47/68 rallies**, 1.77/1.89/2.72 GB, 3840×2160, watermarked. Published to G: as `? - Highlights (4K).mp4`. Robust detached Scheduled Task `KhelSutraBoxhill4KRemaining` (keep-awake + self-restart; unregistered on completion).
16 - ? **DOGFOODED `--detect-from` end-to-end on fresh videos** (the merged #205 feature): `--video <4K original> --detect-from <1080p proxy>` ? detect on proxy, stitch 4K from original, reel named after --video (NO `_1080p` naming bug this time). Log confirmed "detecting on proxy ? stitching reel from ?". Validated beyond the unit tests.

[/external_agent_tool_result]

350. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

351. assistant (2026-07-02T06:11:33.269Z)

[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

352. assistant (2026-07-02T06:11:33.270Z)

Done. Beyond the chip (which its own session owns), I hunted for other fixes, found three real defects, and shipped them as two merged PRs with full due diligence.

What I shipped

PR	Fix	Severity
[#212](https://github.com/avidullu/badminton-highlight-indexer/pull/212) ? `c7baf03`	**DB can't be permanently bricked by an interrupted migration.** v3/v4 used bare non-idempotent `ALTER ... ADD COLUMN` (v1/v2/v5 are `IF NOT EXISTS`). A DDL ALTER auto-commits independently of the surrounding transaction, so a crash *after* the ALTER but *before* the end-of-block `PRAGMA user_version` bump leaves the column present + version stale ? next open re-runs the ALTER ? `duplicate column name` ? `__init__` throws **forever**. Fixed with a narrow `_add_column_idempotent` helper.	High (permanent lockout)
[#213](https://github.com/avidullu/badminton-highlight-indexer/pull/213) ? `bd33085`	**`len()`/truthiness on a raw segmenter `Result`.** `run_phase3_eval.py` (default `--segmenter motion`, the documented baseline) and `run_process_and_stitch.py` crashed with `TypeError` on the `Success`/`Failure` dataclass; the empty-guard was dead code. Fixed with a new DRY `backend/results.unwrap_or_failure()` mirroring `main.py`'s unwrap.	Medium

How I found them

A two-pass adversarial bug-hunt over `origin/master` (~15 agents): **pass 1 (by-area) returned zero** ? the codebase is genuinely clean; **pass 2 (by bug-class + a completeness critic)** surfaced 3 candidates, all of which survived diverse refutation. I verified each against the actual code myself (reverting the DB fix reproduces the exact `duplicate column name`; the script crash confirmed against the real motion segmenter) ? not trusting the agents' descriptions.

Due diligence on each (your pre-LGTM gate)

- Local: full suite green (848 / 851), `ruff` clean, `mypy` clean (104 files), coverage ~80.7% (floor 70). The DB regression test is **proven to fail without the fix**.
- GitHub CI: all 8 checks green on both.
- Adversarial 4-lens review workflow per PR ? **both `verdict=merge`, 0 must-fixes**, independently re-verified.
- I addressed the reviews' minor should-fixes *before* merging (a re-raise-branch test for #212; an empty-`Success` test and a no-overclaim docstring correction for #213).
- Squash-merged, worktrees/branches cleaned.

Deliberately not merged

- **Chip [#210](https://github.com/avidullu/badminton-highlight-indexer/pull/210)** (sibling `fusion_compare`/`fusion_audio` loaders) ? still open and owned by its spawned session; I stayed off those files to avoid racing it on the shared checkout. Flag me if you'd like me to take it over.
- Two **non-bug cleanups** the hunt noted (left as-is): folding the four hand-rolled

`Success`/`Failure` unwraps onto the new helper, and the pre-existing litmus `>=6`/`==6`
fragility from the #209 review. Say the word if you want either.

One incidental finding: this dev container actually **has numpy/scipy/cv2/torch** now (the
`CLAUDE.md` "lacks numpy/cv2/torch" note is stale), which let me verify the pure-logic and
segmenter-`Failure` paths locally ? the full GPU pipeline still can't run here.

353. user (2026-07-02T06:11:33.270Z)

So all the pending PRs are merged. From your context is there anything gated on me or we good to
pick up new work?

354. assistant (2026-07-02T06:11:33.270Z)

Let me check the actual current state rather than answer from memory.

355. assistant (2026-07-02T06:11:33.270Z)

[external_agent_tool_call: Bash]
description: Check open/merged PRs and master state
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git fetch origin 2>&1 | tail -2
echo "=== OPEN PRs ==="
gh pr list --state open --limit 30
echo
echo "=== recently merged (last 6) ==="
gh pr list --state merged --limit 6 --json number,title,mergedAt -q '.[] | "\(.number)
\(.title)"'
echo
echo "=== origin/master HEAD ==="
git log --oneline -4 origin/master
[/external_agent_tool_call]

356. assistant (2026-07-02T06:11:33.270Z)

[external_agent_tool_result]
=== OPEN PRs ===

=== recently merged (last 6) ===
218 docs(recording): assert single-match capture as the #1 rule (now MEASURED)
217 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config
216 chore(eval): re-baseline the nightly after the R1 milestone flip
215 refactor(eval): single source of truth for the _is_fusion_schema predicate
214 docs(archive): LOVO threshold re-sweep at n=13 ? milestone thresholds hold
213 fix(scripts): unwrap segmenter Result before len() (motion crashes the default eval path)

=== origin/master HEAD ===
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
b460526 docs(recording): assert single-match capture as the #1 rule (now MEASURED) (#218)
ab03043 refactor(eval): single source of truth for the _is_fusion_schema predicate (#215)
ff51b37 chore(eval): re-baseline the nightly after the R1 milestone flip (#204) (#216)
[/external_agent_tool_result]

357. assistant (2026-07-02T06:11:33.270Z)

Zero open PRs ? and a lot has merged since (other sessions/you have been busy: #214?#218,
including the R1 flip `#204`). Let me confirm the chip's fate and the local checkout state
before I answer.

358. assistant (2026-07-02T06:11:33.270Z)

[external_agent_tool_call: Bash]
description: Check chip #210, #215 scope, local checkout state
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer

```
echo "=== chip PR #210 (sibling loaders) final state ==="
gh pr view 210 --json number,state,title,mergedAt -q '"#\(.number) \(.state) ? \(.title)"'
echo "=== #215 (single-source _is_fusion_schema) ? did it absorb the sibling-loader fix? ==="
gh pr view 215 --json body -q '.body' 2>&1 | head -20
echo
echo "=== local main checkout: branch + dirty? (clean-slate / nightly self-activation) ==="
git branch --show-current
git status -sb | head -8
echo "=== other live worktrees still around ==="
git worktree list | wc -l
[/external_agent_tool_call]
```

359. assistant (2026-07-02T06:11:33.270Z)

```
[external_agent_tool_result]
=== chip PR #210 (sibling loaders) final state ===
#210 MERGED ? fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus
=== #215 (single-source _is_fusion_schema) ? did it absorb the sibling-loader fix? ===
```

Why

```
Follow-up diligence cleanup. PRs [#209](https://github.com/avidullu/badminton-highlight-
indexer/pull/209) (ablation litmus) and [#210](https://github.com/avidullu/badminton-highlight-
indexer/pull/210) (fusion compare/audio loaders) each landed an **independent, byte-identical**
copy of `_is_fusion_schema` ? the guard that lets the golden-feature loaders skip leaner
velocity-windowing files on a mixed corpus. The duplication was deliberate at the time (to avoid
stacking #210 on the then-unmerged #209). Both are now merged on `master`, so it can be retired.
```

What

```
Consolidate to **one definition** in `backend/eval/experiment.py`, placed next to
`VideoCandidates` ? the type these loaders build, and the module all three already import. This
means:
- **No new import edges** (`ablation`, `fusion_compare`, `fusion_audio` already import
`experiment`).
- **No circular-import risk** ? `experiment` imports none of the three consumers.

The three loaders ? `ablation.load_videos`, `fusion_compare.load_videos`,
`fusion_audio.load_videos_with_audio` ? now import the shared predicate; **their bodies are
unchanged** and the predicate logic is identical. Pure refactor, **no behaviour change**.

Details:
- Dropped the now-unused `Sequence` import from `fusion_compare` (it was added only for the
local def). `ablation` keeps `Sequence` (still used in 5 other signatures).
- Repointed `fusion_audio`'s predicate import from `fusion_compare` ? `experiment` (single
source).
- Moved the predicate's **canonical unit test** to `tests/test_experiment.py` (its new home) and
removed the duplicate from `tests/test_ablation.py`. `test_ablation`'s `_golden_present` keeps
referencing `ablation._is_fusion_schema` ? valid via the re-import, consistent with the existing
`_derive_video_path` cross-module pattern.
```

Verification (local)

```
- `python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` ? **855 passed, 7
skipped**, coverage **80.70%** (?70 floor, no regression).

=== local main checkout: branch + dirty? (clean-slate / nightly self-activation) ===
docs/golden-regression-fixtures-plan
```

docs/golden-regression-fixtures-plan...origin/docs/golden-regression-fixtures-plan [ahead 18, behind 1]

```
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
=== other live worktrees still around ===
20
[/external_agent_tool_result]
```

360. assistant (2026-07-02T06:11:33.270Z)

Good ? full picture now. Notable: the **chip #210 merged** (sibling loaders), **#215** already consolidated the duplicated `_is_fusion_schema` (one of my flagged cleanups ? done by another session), and the **owner-gated R1 flip `#204` merged** (with #216/#217 re-baselining the nightly). The corpus also grew to **n=13** (#214). Let me read the authoritative next-steps from master to pin down what's genuinely owner-gated.

361. assistant (2026-07-02T06:11:33.270Z)

[external_agent_tool_call: Bash]
description: Scan master NEXT_STEPS for owner-gated items
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== golden corpus size + any owner-gated dispatch in NEXT_STEPS (from master) ==="
git show origin/master:docs/NEXT_STEPS.md 2>/dev/null | grep -iEnA1
"owner|gate|dispatch|blocked|IAA|awaiting|product-go|TODO|next up|pick up" | head -50
[/external_agent_tool_call]

362. assistant (2026-07-02T06:11:33.270Z)

[external_agent_tool_result]
=== golden corpus size + any owner-gated dispatch in NEXT_STEPS (from master) ===
4:owner-gated ? see the dedicated section below, the desktop-agent pick-up). Prior: 2026-06-10
(audit?M0?Gemini-battery session; later same day the **UI-alpha track opened**:
5-ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra` repo, local-first
delivery ? see
--
16:**M0 in-container quick wins are COMPLETE:** ship-gate provisional blocker (**#170**),
train/eval split guard (**#171**), the **doc-consistency sweep** (**#174** ? corrected 16
drifted docs vs the code), and the **P3 person-feature fusion litmus machinery is already
present + suite-green** (`backend/eval/fusion_compare.py` + `ablation.py`: person-driven ?F1 /
bootstrap CI / paired sign test; the real-golden `skipif` litmus waits only on GPU-extracted
data). ? CI is red repo-wide due to a **GitHub Actions minutes/billing outage** (account-level,
not code) ? work is local-gated + `--admin`-merged until Actions is restored (Settings ? Billing
? Actions minutes / spending limit).
17-
--
19:1. **[owner / GPU box]** `python -m backend.eval.fusion_golden --manifest
output/human_lovo_manifest.json --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-
data\features\2026-06-15-n6"` ? produces `*_golden_features.json` for the 6 golden videos. (GPU
+ WSL + videos + trajectory CSVs + `.rallies.csv` labels are owner-side only.)
20-2. Artifacts land in the **shared cloud folder** `C:\Users\avidu\OneDrive\rally-
annotator\golden-data\` (OneDrive auto-syncs both ways). Convention + workflow:
[`GOLDEN_DATA_SHARING.md`](GOLDEN_DATA_SHARING.md); tracked in **#175**.
21:3. **[CPU dev/agent session]** `python -m backend.eval.fusion_compare --features-dir
<shared>\features\2026-06-15-n6 --record` + `python -m backend.eval.ablation --features-dir
<shared>\... --granularity group` ? real person-feature ?F1; copy the JSONs into `output/` to
also pass the `skipif` litmus tests (`tests/test_ablation.py::test_litmus_reproduces_gate_a`).
22-
23:**Also queued (owner-gated / pending input):**
24:- Ship-gate target calibration ? **#172** (owner decision; needs corpus growth).
25:- Tests reorganization ? blocked on the owner's grouping metric (provide it ? an agent will
do it).
26-- Restore CI ? the Actions outage is account-level (billing/minutes), not a repo/config bug.
--
28:**Leaving (bulky / M2 ? no start without owner go):** ActionFormer (M0.4), event-index DB
migration (M0.1), Gemini-silver purge (M0.3), cost-to-Gen-2 benchmark + ByteTrack deprecation,
multisport, repo rename, PMF execution.
29-
--
54:## Rally-detection quality track (design merged 2026-06-13, owner-gated; implementation in
progress)
55:**This is the track a desktop/local agent is slated to pick up.** Design docs landed (PR #112
+ session). All implementation is **owner-gated** (per explicit in the plan); each step = ONE PR
off `master`.

56-
--
60: `docs/EXPERIMENT_HARNESS.md` (A/B + shadow + promotion gate, zero-regression) ·
61- `docs/ROORA_LABELING_GUIDELINES.md` (v8) · `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer
mental model).
62:- **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a rally"
bug is **NOT a missing capability** ? `backend/pipeline/detectors/rally_gate.py` already
implements the net-crossing **"Gate A"** (`apply_gate(..., min_crossings=2)`) that kills
service-feed/held-shuttle windows, but it is **only called from eval drivers** (and there was no
`min_crossings` knob in `IndexingConfig`). **Wiring Gate A into the live segmenter (now via
fusion_hybrid) was the single cheapest, highest-confidence quality win.** (P2 FusionSegmenter
specifics: registry `fusion_hybrid`, **default-OFF**, seeds from substrate, persists
`net_crossings` + optional person features, optional served Gate A/B via config knobs;
adversarially reviewed; suite green. See `tests/test_served_gate_a_regression.py` for the honest
finding that on production windowing Gate A is neutral/negative in some...
[/external_agent_tool_result]

363. assistant (2026-07-02T06:11:33.270Z)

Confirmed ? **we're good to pick up new work. Nothing is blocking me.**

State of play

- **Open PRs: 0.** Everything merged, including the two I shipped this session (#212, #213), the
chip **#210** (sibling loaders ? its own session landed it), and a wave of follow-ups from other
sessions: **#204** (the owner-gated R1 milestone flip!), #214 (LOVO re-sweep at **n=13**),
#215 (consolidated the duplicated `_is_fusion_schema` into one source ? one of the cleanups
I'd flagged, now done), #216/#217 (nightly re-baseline + all-OFF floor), #218 (docs).
- No worktrees or branches of mine left dangling.

Gated on *you* ? decisions/inputs only you can give (each unblocks a *specific* track; none blocks general w

- **IAA 2nd-annotator study** ? the one thing still blocking **R2 ? calibration** (the tolerance
thresholds). I can't manufacture inter-annotator agreement ? needs a second human labeling 2?3
golden clips.
- **Ship-gate target ratification (#172)** ? your call on the F1 bar (floor 0.48 / multi-court
ref 0.66 are computed; needs your sign-off).
- **Any owner-gated track *start*** ? the platform async-job slice and the owned-model Phase-0
both say "no start without owner go." If you want either next, just say go.
- **More golden data / GPU-side extraction** ? owner-side (videos + GPU + WSL), only if we want
to grow past n=13.

One housekeeping flag (not blocking)

The local **main checkout is on `docs/golden-regression-fixtures-plan`** (ahead 18 / behind 1 of
its remote), not `master` ? so per your clean-slate convention the nightly-regression task stays
dormant until it's back on master, and that branch has unmerged commits worth a decide-or-
drop. There are also ~20 lingering worktrees from past sessions I could prune.

What I can pick up right now (no gating)

1. **Fresh bug-hunt on the new master** ? it moved a lot (R1 flip, nightly baselines, #215
refactor); new code = worth another adversarial pass.
2. **Small cleanup** ? fold the remaining hand-rolled `Success`/`Failure` unwraps (`main.py` ×2,
`generate_highlights.py`) onto the new `unwrap_or_failure`, and the litmus `>=6`/`==6`
fragility.
3. **Tidy-up** ? prune stale worktrees / get the main checkout back on master so the nightly re-
activates.

Want me to take one of those, or do you have something specific in mind?

364. user (2026-07-02T06:11:33.270Z)

why just one why not all the three! I have been meaning to pause development and do a
refactoring effort seeing that a bunch of files have grown a lot, so please find bugs and also
some critical tech debt which we can clean up , just make the PRs and ensure teh quality does
not regress and I will take a look at them!

365. assistant (2026-07-02T06:11:33.271Z)

Love it ? let's do all three, plus the refactoring sweep you're after. To be clear on the mode: I'll **open PRs for your review and not merge them** (you said you'll take a look), each one its own branch off `master`, with the full quality gate (suite + ruff + mypy + coverage) green so nothing regresses.

Let me start by syncing to the new master and mapping where the bloat actually is, so I target the *critical* tech debt rather than guess.

366. assistant (2026-07-02T06:11:33.271Z)

```
[external_agent_tool_call: Bash]
description: Sync, create base worktree, map largest files
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git fetch origin 2>&1 | tail -2
git worktree add --detach .claude/worktrees/refactor-base origin/master 2>&1 | tail -2
cd .claude/worktrees/refactor-base
echo "=== HEAD ==="
git log --oneline -1
echo "=== top 30 largest source files (LOC) ==="
find backend scripts training -name '*.py' -not -path '*/__pycache__/*' -exec wc -l {} + | sort
-rn | head -32
[/external_agent_tool_call]
```

367. assistant (2026-07-02T06:11:33.271Z)

```
[external_agent_tool_result]
Preparing worktree (detached HEAD a010667)
HEAD is now at a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config
(#217)
=== HEAD ===
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
=== top 30 largest source files (LOC) ===
 18860 total
  1113 backend/eval/golden_fixtures.py
   871 backend/main.py
   738 backend/infrastructure/database.py
   699 backend/pipeline/detectors/wasb_infer.py
   633 scripts/nightly_regression.py
   620 backend/eval/ablation.py
   579 backend/eval/experiment.py
   506 backend/pipeline/detectors/tracknet_runner.py
   472 backend/pipeline/segmenters/gemini.py
   392 backend/pipeline/stitcher/compiler.py
   380 backend/eval/rally_seg_eval.py
   369 backend/pipeline/segmenters/yolo_hybrid.py
   357 backend/pipeline/detectors/person_detector.py
   337 backend/pipeline/segmenters/trajectory_hybrid.py
   334 backend/config/models.py
   291 backend/eval/per_user_regression.py
   277 backend/eval/tolerance_metrics.py
   233 backend/eval/gemini_refine.py
   230 backend/eval/export_wasb_segments.py
   227 backend/pipeline/segmenters/motion.py
   226 backend/eval/segmentation_metrics.py
   224 training/gen0/harness.py
   223 backend/reporting/harvest.py
   218 backend/eval/eval_partial_labels.py
   216 backend/utils/run_telemetry.py
   215 backend/pipeline/detectors/fusion_features.py
   210 backend/eval/windowing.py
   209 backend/eval/per_user_gate.py
   203 backend/eval/promotion.py
   199 backend/pipeline/detectors/wasb_runner.py
```

```
   199 backend/eval/served_gate_a.py
[/external_agent_tool_result]
```

368. assistant (2026-07-02T06:11:33.271Z)

```
Clear bloat map: `golden_fixtures.py` (1113), `main.py` (871), `database.py` (738),
`wasb_infer.py` (699), `ablation.py` (620), `experiment.py` (579). Let me also get a function-
length signal (the real debt is often a giant function inside a file) before launching
discovery.
```

369. assistant (2026-07-02T06:11:33.271Z)

```
[external_agent_tool_call: Bash]
description: Find the longest/most-complex functions
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
python - <<'PY'
import ast, os
rows = []
for root, _, files in os.walk('backend'):
    if '__pycache__' in root: continue
    for f in files:
        if not f.endswith('.py'): continue
        p = os.path.join(root, f)
        try:
            tree = ast.parse(open(p, encoding='utf-8').read())
        except Exception:
            continue
        for node in ast.walk(tree):
            if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                if node.end_lineno and node.lineno:
                    length = node.end_lineno - node.lineno
                    # count branch-y statements as a rough complexity proxy
                    branches = sum(isinstance(n, (ast.If, ast.For, ast.While, ast.Try, ast.With,
ast.ExceptHandler, ast.BoolOp)) for n in ast.walk(node))
                    rows.append((length, branches, f"{p}:{node.lineno} {node.name}"))
rows.sort(reverse=True)
print("=== 22 longest functions (len | branchy-nodes | location) ===")
for length, br, loc in rows[:22]:
    print(f"{length:4d} | {br:3d} | {loc}")
PY
[/external_agent_tool_call]
```

370. assistant (2026-07-02T06:11:33.271Z)

```
[external_agent_tool_result]
=== 22 longest functions (len | branchy-nodes | location) ===
 434 |  71 | backend\pipeline\segmenters\gemini.py:35 process_video
 309 |  45 | backend\pipeline\segmenters\yolo_hybrid.py:57 process_video
 231 |  30 | backend\pipeline\segmenters\trajectory_hybrid.py:106 process_video
 204 |  24 | backend\pipeline\segmenters\motion.py:20 process_video
 202 |  36 | backend\pipeline\stitcher\compiler.py:190 compile_highlights
 187 |   6 | backend\infrastructure\database.py:62 _init_db
 186 |  13 | backend\eval\golden_fixtures.py:598 freeze_real_fixture
 185 |  25 | backend\pipeline\detectors\wasb_infer.py:359 run
 168 |  23 | backend\main.py:700 main
 132 |  16 | backend\pipeline\detectors\tracknet_runner.py:256 trajectory_to_action_windows
 128 |   8 | backend\eval\per_user_gate.py:81 should_promote_for_user
 122 |   7 | backend\eval\per_user_regression.py:169 evaluate_no_regression
 116 |  13 | backend\pipeline\detectors\person_detector.py:140 extract_action_windows_from_chunk
 115 |   1 | backend\eval\ablation.py:457 export_pdf
 112 |  10 | backend\main.py:123 _execute_processing
 109 |  18 | backend\eval\rally_seg_eval.py:267 main
 107 |  18 | backend\providers\gemini.py:38 analyze_video
 102 |  16 | backend\pipeline\validators\pts_continuity.py:14 validate
```

```
  95 |   8 | backend\pipeline\validators\relevance.py:15 validate
  93 |   7 | backend\eval\promotion.py:110 promote_if_served_safe
  90 |  14 | backend\pipeline\detectors\tracknet_runner.py:120 run_predict
  90 |  10 | backend\eval\golden_fixtures.py:1019 main
[/external_agent_tool_result]
```

371. assistant (2026-07-02T06:11:33.271Z)

Two clear debt signals now: monster functions ? `gemini.py process_video` (**434 lines / 71 branches**), `yolo_hybrid` (309), `trajectory_hybrid` (231), `motion` (204), `compiler.compile_highlights` (202) ? and god-modules ? `golden_fixtures.py` (1113), `main.py` (871), `database.py` (738).

Let me run a discovery workflow: a fresh bug-hunt on the changed master **plus** a tech-debt analysis that proposes only *safe, behavior-preserving, well-tested* refactors (each rated by risk + test coverage), then synthesizes a prioritized PR worklist.

372. assistant (2026-07-02T06:11:33.271Z)

```
[external_agent_tool_call: Workflow]
description: Discover bugs + safe refactors, produce PR worklist
[/external_agent_tool_call]
```

373. assistant (2026-07-02T06:11:33.271Z)

```
[external_agent_tool_result]
Workflow launched in background. Task ID: wwq5a0lfa
Summary: Find real bugs + propose safe high-value refactors on the grown files; prioritized PR
worklist
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer
--claude-worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bughunt-
and-techdebt-discovery-wf_e5140909-890.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-
worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bughunt-and-
techdebt-discovery-wf_e5140909-890.js"} to iterate without resending the script.)
Run ID: wf_e5140909-890
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-refactor-
base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\bughunt-and-techdebt-discovery-
wf_e5140909-890.js", resumeFromRunId: "wf_e5140909-890"}) ? completed agents return cached
results.

You will be notified when it completes. Use /workflows to watch live progress.
[/external_agent_tool_result]
```

374. assistant (2026-07-02T06:11:33.271Z)

Discovery is running across the codebase. Let me wait for the prioritized worklist.

375. assistant (2026-07-02T06:11:33.271Z)

```
[external_agent_tool_call: TaskOutput]
input: {"task_id":"wwq5a0lfa","block":true,"timeout":600000}
[/external_agent_tool_call]
```

376. assistant (2026-07-02T06:11:33.271Z)

```
[external_agent_tool_result]
<retrieval_status>timeout</retrieval_status>

<task_id>wwq5a0lfa</task_id>
```

<task_type>local_workflow</task_type>

<status>running</status>
[/external_agent_tool_result]

377. assistant (2026-07-02T06:11:33.271Z)

[external_agent_tool_call: TaskOutput]
input: {"task_id":"wwq5a0lfa","block":true,"timeout":600000}
[/external_agent_tool_call]

378. assistant (2026-07-02T06:11:33.271Z)

[external_agent_tool_result]
<retrieval_status>success</retrieval_status>

<task_id>wwq5a0lfa</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

```
{
  "summary": "Find real bugs + propose safe high-value refactors on the grown files; prioritized
PR worklist",
  "agentCount": 18,
  "logs": [
    "Bugs: 4 candidate(s)",
    "Bugs confirmed: 1/4",
    "Debt: 18 proposal(s)"
  ],
  "result": {
    "plan": {
      "prs": [
        {
          "rank": 1,
          "type": "bug",
          "title": "config loader: degrade to defaults+warn when config.json is valid JSON but
not an object",
          "files": "backend/config/loader.py (load_config, ~lines 65-86); tests:
tests/test_config.py",
          "summary": "load_config crashes at process startup when config.json parses to a non-
object (e.g. [], 42, \"x\"). json.load() succeeds so the read/parse try/except does not catch
it; then for a non-empty non-mapping, _unknown_key_paths runs .items() (AttributeError), and for
any non-mapping the {**defaults, **file_data} unpack raises TypeError ('list' object is not a
mapping). Reproduced locally with config.json='[]' -> TypeError. This violates the loader's
documented contract (docstring lines 60-61: missing/unreadable -> fully-defaulted AppConfig, bad
input non-fatal) and turns an operator typo into a hard server/CLI startup crash via
main.py::main -> load_app_config.",
          "plan": "Immediately after json.load (before the `if file_data:` block), coerce a non-
dict result to the no-op path: `if not isinstance(file_data, dict): logger.warning(f\"{cfg_path}
is not a JSON object; ignoring and using defaults.\"); file_data = {}`. This restores the
'defaults + warn, never fatal' behavior for both the falsy ([], 0, \"\") and truthy ([1], \"x\",
42) non-object cases in one guard. No other control flow changes.",
          "risk": "low",
          "test_plan": "Add tests to tests/test_config.py: write config.json containing each of
`[]`, `[1,2]`, `\"x\"`, `42`, `true`, `null`; assert load_config returns a default-equal
AppConfig and does not raise, and that a warning is logged (caplog). Keep existing valid-object
and missing-file tests green. Run full suite + ruff + mypy; coverage must stay >=70 (this adds a
previously-uncovered branch, so coverage should rise slightly).",
          "recommend_now": true
        },
        {
          "rank": 2,
```

          "type": "refactor",
          "title": "database: extract per-table CREATE + each migration block out of _init_db
into named helpers",
          "files": "backend/infrastructure/database.py (_init_db, ~lines 62-247, 185L) -> same
module/class _create_base_tables(cursor) + _migrate_to_v1..v5(self, cursor); tests:
tests/test_database.py, tests/test_async_jobs.py, tests/test_job_worker_loop.py",
          "summary": "_init_db is the longest method in the class: three base-table CREATEs plus
five sequential `if version < N` migration blocks with embedded DDL, all inline. Verified the
ladder is clean (v1->v5, each guarded by `if version < N`, each ending in `PRAGMA user_version =
N`, idempotent ALTERs via _add_column_idempotent). The migration contract is heavily pinned by
tests (schema-version-is-current, idempotent-and-upgrades-old-db, v3->v5-preserving-rows,
idempotent-on-interrupted-upgrade).",
          "plan": "Pure intra-class extraction. Pull each `if version < N:` body verbatim into
`_migrate_to_vN(self, cursor)` and the three base CREATEs into `_create_base_tables(cursor)`.
_init_db becomes: open connection (same single `with self._connect()` block), create base
tables, read PRAGMA user_version, call the migration helpers in order each still guarded by the
SAME `if version < N` check, commit. No SQL text, ordering, PRAGMA bumps, or cursor/commit
semantics change. All DDL still runs on one cursor inside one connection block to preserve the
auto-commit/interruption behavior the tests pin.",
          "risk": "low",
          "test_plan": "Run tests/test_database.py (incl. the int...
[/external_agent_tool_result]

379. assistant (2026-07-02T06:11:33.271Z)

[external_agent_tool_result]
Workflow launched in background. Task ID: wyh5c4cgs
Summary: Implement vetted behavior-preserving refactors in isolated worktrees; open PRs for
owner review
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_87c49086-be4
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer
--claude-worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\safe-
refactor-fleet-wf_87c49086-be4.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer--claude-
worktrees-refactor-base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\safe-refactor-
fleet-wf_87c49086-be4.js"} to iterate without resending the script.)
Run ID: wf_87c49086-be4
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-badminton-highlight-indexer--claude-worktrees-refactor-
base\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\safe-refactor-fleet-
wf_87c49086-be4.js", resumeFromRunId: "wf_87c49086-be4"}) ? completed agents return cached
results.

You will be notified when it completes. Use /workflows to watch live progress.
[/external_agent_tool_result]

380. assistant (2026-07-02T06:11:33.271Z)

Outstanding discovery output: **1 confirmed bug** (config loader crashes on a non-object `config.json` ? reproduced) and a **vetted, risk-rated refactor worklist** of 9 items (8 low-risk, 1 medium), with the genuinely-risky/under-tested ones correctly *deferred* (motion's random-heavy path, wasb GPU `run()`, the main.py job-worker seam, etc.).

Key correction I'll apply: the agents assumed "no GPU/torch/cv2 here," but this container **does** have them ? so every PR's full gate runs locally before it's opened.

Plan: I'll **implement the bug myself** (most important to verify personally), and **fan out the low-risk refactors to worktree-isolated agents** that each implement ? run the full gate ? open a PR for your review (never merge). Let me launch the refactor fleet, then do the bug in parallel.

381. assistant (2026-07-02T06:11:33.271Z)

[external_agent_tool_call: Workflow]
description: Fan out vetted refactors to isolated worktrees, open PRs
[/external_agent_tool_call]

382. assistant (2026-07-02T06:11:33.271Z)

The refactor fleet (7 isolated worktrees) is running. While it works, let me implement the config-loader bug myself. First, read the loader to confirm the exact crash.

383. assistant (2026-07-02T06:11:33.271Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\refactor-base\backend\config\loader.py
[/external_agent_tool_call]

384. assistant (2026-07-02T06:11:33.271Z)

[external_agent_tool_result]

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      from pathlib import Path
14      from typing import Any
15
16      from pydantic import BaseModel
17
18      from .models import AppConfig
19
20      logger = logging.getLogger(__name__)
21
22      DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25      def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
26          """Dotted paths in `data` that the typed config will not honor.
27
28          Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
29          kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30          typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31          user's intent is lost ? collect every such key so load_config can warn.
32          """
33          unknown: list[str] = []
34          fields = model_cls.model_fields
35          for key, val in data.items():
36              if key not in fields:
37                  unknown.append(prefix + key)
38                  continue
39              ann = fields[key].annotation
40              if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
41                  unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
42          return unknown
43
```

```
44
45      def _get_builtin_defaults() -> dict[str, Any]:
46          """Return a plain dict of the current built-in defaults.
47
48          The AppConfig model is the single source of truth for defaults.
49          """
50          return AppConfig().model_dump(mode="json")
51
52
53      def load_config(path: str | Path | None = None) -> AppConfig:
54          """Load configuration with the following precedence:
55
56          1. Built-in defaults defined in the Pydantic models.
57          2. Values from the JSON file (if present and readable).
58          3. (Future) environment / CLI overrides.
59
60          Missing file or unreadable file ? returns a fully defaulted AppConfig.
61          Unknown keys in the file are tolerated (extra="allow" on the model).
62          """
63          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65          file_data: dict[str, Any] = {}
66          if cfg_path.exists():
67              try:
68                  with open(cfg_path, "r", encoding="utf-8") as f:
69                      file_data = json.load(f)
70              except Exception as exc:
71                  # Non-fatal: continue with defaults + whatever we could parse.
72                  # In production we might log here.
73                  logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75          if file_data:
76              unknown = _unknown_key_paths(file_data, AppConfig)
77              if unknown:
78                  logger.warning(
79                      "[CONFIG WARNING] %s contains keys the typed config does not "
80                      "recognize (typo'd or removed knobs ? top-level extras are kept "
81                      "but unread; unknown section keys are silently dropped): %s",
82                      cfg_path, ", ".join(sorted(unknown)),
83                  )
84
85          # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
86          merged = {**_get_builtin_defaults(), **file_data}
87
88          return AppConfig.model_validate(merged)
89
90
91      def save_default_config(path: str | Path | None = None) -> Path:
92          """Write a fresh default config.json.
93
94          Used by scripts/doctor.py diagnostics and the --setup CLI path.
95          Overwrites if the file exists.
96          """
97          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
98          cfg = AppConfig()
99          cfg_path.parent.mkdir(parents=True, exist_ok=True)
100         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
101         return cfg_path
102
103
104     # Backwards-compatibl...
[/external_agent_tool_result]
```

385. assistant (2026-07-02T06:11:33.271Z)

Confirmed the bug: a valid-but-non-object `config.json` either hits `.items()` on a non-dict (truthy case) or `{**defaults, **file_data}` unpacking a non-mapping (falsy case) ? both crash startup, violating the "defaults + warn, never fatal" contract. Let me set up the bug worktree and read the existing config tests.

386. assistant (2026-07-02T06:11:33.272Z)

```
[external_agent_tool_call: Bash]
description: Create bug worktree, reproduce crash, read test_config
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git worktree add -b fix/config-loader-non-object-json .claude/worktrees/config-bug origin/master
2>&1 | tail -2
echo "=== reproduce the crash on origin/master ==="
cd .claude/worktrees/config-bug
python - <<'PY' 2>&1 | tail -6
import tempfile, os
from backend.config.loader import load_config
for bad in ['[]', '[1,2]', '"x"', '42', 'true', 'null']:
    d = tempfile.mkdtemp(); p = os.path.join(d, "config.json")
    open(p, "w").write(bad)
    try:
        load_config(p); print(f"{bad!r:8} -> OK (no crash)")
    except Exception as e:
        print(f"{bad!r:8} -> {type(e).__name__}: {e}")
PY
echo "=== existing test_config.py structure ==="
grep -nE "def test_|tmp_path|config.json|caplog|monkeypatch" tests/test_config.py | head -25
[/external_agent_tool_call]
```

387. assistant (2026-07-02T06:11:33.272Z)

```
[external_agent_tool_result]
branch 'fix/config-loader-non-object-json' set up to track 'origin/master'.
HEAD is now at a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config
(#217)
=== reproduce the crash on origin/master ===
'[]'     -> TypeError: 'list' object is not a mapping
'[1,2]'  -> AttributeError: 'list' object has no attribute 'items'
'"x"'    -> AttributeError: 'str' object has no attribute 'items'
'42'     -> AttributeError: 'int' object has no attribute 'items'
'true'   -> AttributeError: 'bool' object has no attribute 'items'
'null'   -> TypeError: 'NoneType' object is not a mapping
=== existing test_config.py structure ===
17:def test_load_default_config():
18:    # Force load without the project's config.json so we test the model built-in defaults
19:    cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
27:def test_logging_config_default_quiets_http():
29:    cfg = load_config("/nonexistent/path/12345/config.json")
33:def test_skip_ai_handoff_default_off():
35:    cfg = load_config("/nonexistent/path/12345/config.json")
39:def test_skip_ai_handoff_file_override():
48:def test_logging_config_file_override():
57:def test_fusion_config_defaults_reproduce_control():
65:    cfg = load_config("/nonexistent/path/12345/config.json")
75:def test_fusion_config_gate_a_opt_in_via_min_crossings():
86:def test_served_default_segmenter_is_gemini():
87:    """The shipped config.json served default is the prior detector (gemini) ? the #146 flip
90:    cfg = load_config(os.path.join(repo_root, "config.json"))
96:def test_fusion_config_file_override():
114:def test_file_override():
137:def test_extra_keys_tolerated():
138:    """Old or future keys in config.json must not break loading."""
155:def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
159:    p = tmp_path / "typo.json"
166:    with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
```

```
169:    joined = " ".join(r.message for r in caplog.records)
175:def test_recognized_config_does_not_warn(tmp_path, caplog):
176:    p = tmp_path / "clean.json"
[/external_agent_tool_result]
```

388. assistant (2026-07-02T06:11:33.272Z)

All 6 non-object inputs crash on master. Let me read the worktree's loader + a test pattern to match, then fix.

389. assistant (2026-07-02T06:11:33.272Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-bug\backend\config\loader.py
[/external_agent_tool_call]

390. assistant (2026-07-02T06:11:33.272Z)

[external_agent_tool_result]

```
63          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65          file_data: dict[str, Any] = {}
66          if cfg_path.exists():
67              try:
68                  with open(cfg_path, "r", encoding="utf-8") as f:
69                      file_data = json.load(f)
70              except Exception as exc:
71                  # Non-fatal: continue with defaults + whatever we could parse.
72                  # In production we might log here.
73                  logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75          if file_data:
76              unknown = _unknown_key_paths(file_data, AppConfig)
77              if unknown:
78                  logger.warning(
79                      "[CONFIG WARNING] %s contains keys the typed config does not "
80                      "recognize (typo'd or removed knobs ? top-level extras are kept "
81                      "but unread; unknown section keys are silently dropped): %s",
82                      cfg_path, ", ".join(sorted(unknown)),
83                  )
84
85          # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
86          merged = {**_get_builtin_defaults(), **file_data}
87
88          return AppConfig.model_validate(merged)
```

[/external_agent_tool_result]

391. assistant (2026-07-02T06:11:33.272Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-bug\tests\test_config.py
[/external_agent_tool_call]

392. assistant (2026-07-02T06:11:33.272Z)

[external_agent_tool_result]

```
137     def test_extra_keys_tolerated():
138         """Old or future keys in config.json must not break loading."""
139         with tempfile.TemporaryDirectory() as td:
140             p = os.path.join(td, "with_extra.json")
141             data = {
```

```
142                 "sport": "badminton",
143                 "indexing": {"default_segmenter": "yolo_hybrid"},
144                 "some_future_key": "ignored_for_now",
145                 "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
146             }
147             with open(p, "w") as f:
148                 json.dump(data, f)
149 
150             cfg = load_config(p)
151             assert cfg.indexing.default_segmenter == "yolo_hybrid"
152             assert cfg.validation.relevance.court_pixels_min_ratio == 0.1
153 
154 
155     def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
156         """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
157         section keys are silently dropped, so without this warning a typo just makes
158         the knob a no-op (the audit's 'silently dropped config' class)."""
159         p = tmp_path / "typo.json"
160         p.write_text(json.dumps({
161             "some_future_key": 1,                                  # top-level extra
162             "indexing": {"velocty_threshold": 0.5},                # typo'd section key
163             "validation": {"relevance": {"unknown_gate": True}},   # doubly nested
164         }))
165         import logging
166         with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
167             cfg = load_config(str(p))
168         assert isinstance(cfg, AppConfig)  # still non-fatal
169         joined = " ".join(r.message for r in caplog.records)
170         assert "some_future_key" in joined
171         assert "indexing.velocty_threshold" in joined
172         assert "validation.relevance.unknown_gate" in joined
173 
174 
175     def test_recognized_config_does_not_warn(tmp_path, caplog):
176         p = tmp_path / "clean.json"
177         p.write_text(json.dumps({
178             "sport": "badminton",
179             "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":
0.02}},
180         }))
181         import logging
[/external_agent_tool_result]
```

393. assistant (2026-07-02T06:11:33.272Z)

Now implement the fix ? coerce a non-object `config.json` to the no-op path:

394. assistant (2026-07-02T06:11:33.272Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-
bug\backend\config\loader.py
[/external_agent_tool_call]
```

395. assistant (2026-07-02T06:11:33.272Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-
bug\backend\config\loader.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
[/external_agent_tool_result]
```

396. assistant (2026-07-02T06:11:33.272Z)

Now the regression test. Let me check the imports and add it:

397. assistant (2026-07-02T06:11:33.272Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-bug\tests\test_config.py
[/external_agent_tool_call]

398. assistant (2026-07-02T06:11:33.272Z)

[external_agent_tool_result]
```
1       """Basic tests for the typed configuration package.
2
3       These ensure:
4       - Defaults are sensible and come from the models.
5       - File overrides work.
6       - Old config files with extra keys are tolerated.
7       - save_default_config produces loadable output.
8       """
9       import json
10      import tempfile
11      import os
12
13
14      from backend.config import load_config, save_default_config, AppConfig
15
16
```
[/external_agent_tool_result]

399. assistant (2026-07-02T06:11:33.272Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-bug\tests\test_config.py
[/external_agent_tool_call]

400. assistant (2026-07-02T06:11:33.272Z)

[external_agent_tool_result]
```
175     def test_recognized_config_does_not_warn(tmp_path, caplog):
176         p = tmp_path / "clean.json"
177         p.write_text(json.dumps({
178             "sport": "badminton",
179             "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":
0.02}},
180         }))
181         import logging
182         with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
183             load_config(str(p))
184         assert not [r for r in caplog.records if "CONFIG WARNING" in r.message]
185
186
187     def test_save_default_config_roundtrip():
188         with tempfile.TemporaryDirectory() as td:
189             p = os.path.join(td, "saved.json")
190             saved = save_default_config(p)
191             assert str(saved) == p
192             assert os.path.exists(p)
193
194             cfg = load_config(p)
195             assert cfg.indexing.default_segmenter == "yolo_hybrid"
196             assert cfg.validation.min_blur_threshold == 20.0
197
```

```
198
199     def test_model_is_the_source_of_truth():
200         """Changing a default in the model should be reflected without touching json."""
201         # This is a documentation test more than anything.
202         cfg = AppConfig()
203         assert cfg.indexing.tracknet.velocity_threshold == 0.02
204
205
206     def test_output_dir_field_default_and_override():
207         """output_dir now lives on OutputConfig (was a write-only runtime hack), so
208         /api/compile and the CLI resolve the same value. Default + file override + the
209         runtime mutation that backend/main.py::main performs for --output-dir."""
210         cfg = AppConfig()
211         assert cfg.output.output_dir == "output"  # built-in default
212
213         # The CLI override path: main() assigns args.output_dir onto the typed model.
214         cfg.output.output_dir = "/tmp/custom_out"
[/external_agent_tool_result]
```

401. assistant (2026-07-02T06:11:33.272Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-
bug\tests\test_config.py
[/external_agent_tool_call]
```

402. assistant (2026-07-02T06:11:33.272Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\config-
bug\tests\test_config.py has been updated successfully. (file state is current in your context ?
no need to Read it back)
[/external_agent_tool_result]
```

403. assistant (2026-07-02T06:11:33.272Z)

Verify the test pins the bug (passes with fix, fails without), then the full gate:

404. assistant (2026-07-02T06:11:33.272Z)

```
[external_agent_tool_call: Bash]
description: Verify test pins the config bug
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/config-bug
echo "=== new test WITH fix (expect pass) ==="
python -m pytest tests/test_config.py::test_non_object_config_degrades_to_defaults -q 2>&1 |
tail -4
echo "=== revert loader, re-run (expect FAIL) ==="
git stash push -- backend/config/loader.py 2>&1 | tail -1
python -m pytest tests/test_config.py::test_non_object_config_degrades_to_defaults -q 2>&1 |
tail -5
git stash pop 2>&1 | tail -1
[/external_agent_tool_call]
```

405. assistant (2026-07-02T06:11:33.272Z)

```
[external_agent_tool_result]
=== new test WITH fix (expect pass) ===
.                                                                        [100%]
1 passed in 0.56s
=== revert loader, re-run (expect FAIL) ===
Saved working directory and index state WIP on fix/config-loader-non-object-json: a010667
feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)

backend\config\loader.py:86: TypeError
```

```
========================== short test summary info ==========================
FAILED tests/test_config.py::test_non_object_config_degrades_to_defaults - Ty...
1 failed in 0.87s
Dropped refs/stash@{0} (48b6cd0640e8f3b9a151f9b40b1b7210a7507b9c)
[/external_agent_tool_result]
```

406. assistant (2026-07-02T06:11:33.272Z)

Test pins the bug (passes with fix, fails with `TypeError` without). Now the full gate:

407. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_call: Bash]
description: Config bug PR full gate
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/config-bug
python -m ruff check . 2>&1 | tail -1
python -m mypy backend training 2>&1 | tail -1
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70 2>&1
| tail -4
[/external_agent_tool_call]
```

408. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_result]
All checks passed!
Success: no issues found in 104 source files
857 passed, 7 skipped, 7 warnings in 140.25s (0:02:20)
======================================================================
 RESULT: PASS - all tests green
======================================================================
[/external_agent_tool_result]
```

409. assistant (2026-07-02T06:11:33.273Z)

Full gate green (857 passed, ruff + mypy clean, coverage ?70). Commit, push, open PR (for your review ? not merging):

410. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_call: Bash]
description: Commit and push config bug fix
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/config-bug
git add backend/config/loader.py tests/test_config.py
git status --porcelain
git commit -F - <<'EOF'
fix(config): degrade to defaults + warn when config.json is valid JSON but not an object

`load_config` documents "bad input is non-fatal ? fully-defaulted AppConfig", but a config.json
that parses to a non-object crashed startup: `json.load` succeeds (so the read try/except
doesn't
catch it), then a truthy non-mapping (`[1]`, `"x"`, `42`, `true`) hits `.items()` in
`_unknown_key_paths` (AttributeError) and any non-mapping (`[]`, `null`, ...) breaks the
`{**defaults, **file_data}` unpack (TypeError). Reproduced for all of `[]`/`[1,2]`/`"x"`/`42`/
`true`/`null`. Via `main.py::main -> load_app_config` this turns an operator typo into a hard
server/CLI startup crash.

Fix: one guard after `json.load` ? if the parsed value isn't a dict, log a warning and fall back
to `{}` (defaults). Restores the documented contract for every non-object case; happy path
(missing file, valid object, unknown-key warnings) unchanged.

Regression test `test_non_object_config_degrades_to_defaults` covers all six non-object shapes
(asserts default-equal AppConfig + the warning). Verified it FAILS without the fix (TypeError).

Local gate: full suite 857 passed / 7 skipped, ruff clean, mypy clean (104 files), coverage
```

```
>=70.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
git log --oneline origin/master..HEAD
git push -u origin fix/config-loader-non-object-json 2>&1 | tail -2
[/external_agent_tool_call]
```

411. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_result]
M  backend/config/loader.py
M  tests/test_config.py
[fix/config-loader-non-object-json d5e5ad7] fix(config): degrade to defaults + warn when
config.json is valid JSON but not an object
 2 files changed, 29 insertions(+)
d5e5ad7 fix(config): degrade to defaults + warn when config.json is valid JSON but not an object
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      fix/config-loader-non-object-json -> fix/config-loader-non-object-json
[/external_agent_tool_result]
```

412. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_call: Bash]
description: Open config bug PR for review
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/config-bug
cat > /tmp/cfgbug.md <<'EOF'
## Problem (startup crash)

`load_config` documents *"bad input is non-fatal ? fully-defaulted AppConfig"*, but a
`config.json` that is **valid JSON but not an object** crashes startup. `json.load` succeeds (so
the read `try/except` doesn't catch it), then:
- a **truthy** non-mapping (`[1]`, `"x"`, `42`, `true`) hits `.items()` in `_unknown_key_paths`
? `AttributeError`
- any non-mapping (`[]`, `null`, ?) breaks the `{**defaults, **file_data}` unpack ? `TypeError`

Reproduced for all of `[]` / `[1,2]` / `"x"` / `42` / `true` / `null`. Via `main.py::main ?
load_app_config`, an operator typo (e.g. a stray top-level `[]`) becomes a hard server/CLI
**startup crash**.

## Fix

One guard after `json.load`: if the parsed value isn't a `dict`, log a warning and fall back to
`{}` (defaults). Restores the documented "defaults + warn, never fatal" contract for every non-
object shape. Happy path (missing file, valid object, unknown-key warnings) unchanged.

## Test

`test_non_object_config_degrades_to_defaults` covers all six non-object shapes (asserts a
default-equal `AppConfig` + the warning). **Verified it fails without the fix** (`TypeError`).

## Verification (local ? this machine has the full env)

Full suite **857 passed / 7 skipped**, `ruff` clean, `mypy backend training` clean (104 files),
coverage ?70 (adds a previously-uncovered branch).

Found by an adversarial bug-hunt during the refactoring sweep. Opened for review ? not merged.

? Generated with [Claude Code](https://claude.com/claude-code)
EOF
gh pr create --base master --head fix/config-loader-non-object-json \
  --title "fix(config): non-object config.json must not crash startup (degrade to defaults +
warn)" \
```

```
  --body-file /tmp/cfgbug.md 2>&1 | tail -2
[/external_agent_tool_call]
```

413. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_result]
https://github.com/avidullu/badminton-highlight-indexer/pull/221
[/external_agent_tool_result]
```

414. assistant (2026-07-02T06:11:33.273Z)

```
Config bug PR [#221](https://github.com/avidullu/badminton-highlight-indexer/pull/221) is open
for your review. Now let me collect the refactor fleet's results.
```

415. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_call: TaskOutput]
input: {"task_id":"wyh5c4cgs","block":true,"timeout":600000}
[/external_agent_tool_call]
```

416. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_result]
<retrieval_status>success</retrieval_status>

<task_id>wyh5c4cgs</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>
{
  "summary": "Implement vetted behavior-preserving refactors in isolated worktrees; open PRs for
owner review",
  "agentCount": 7,
  "logs": [
    "PRs opened: 7/7 ? https://github.com/avidullu/badminton-highlight-indexer/pull/223
https://github.com/avidullu/badminton-highlight-indexer/pull/224
https://github.com/avidullu/badminton-highlight-indexer/pull/225
https://github.com/avidullu/badminton-highlight-indexer/pull/227
https://github.com/avidullu/badminton-highlight-indexer/pull/222
https://github.com/avidullu/badminton-highlight-indexer/pull/226
https://github.com/avidullu/badminton-highlight-indexer/pull/228"
  ],
  "result": {
    "results": [
      {
        "slug": "db-init-extract",
        "branch": "refactor/db-init-helpers",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/223",
        "diffstat": "backend/infrastructure/database.py | 349
++++++++++++++++++-------------- 1 file changed, 184 insertions(+), 165 deletions(-)",
        "suite": "856 passed, 7 skipped",
        "ruff": "All checks passed!",
        "mypy": "Success: no issues found in 104 source files",
        "coverage": "80.75% (floor 70; Required test coverage of 70% reached)",
        "behavior_note": "Pure intra-class cut-paste extraction. _init_db's single `with
self._connect()` block is unchanged: it now calls _create_base_tables(cursor) (the
videos/play_segments/events CREATEs), reads `version = cursor.execute(\"PRAGMA
user_version\").fetchone()[0]`, then calls _migrate_to_v1..._migrate_to_v5(cursor) each guarded
by the SAME `if version < N:` checks in the SAME order. Every block body ? SQL text, the `PRAGMA
user_version = N` bumps, the _add_column_idempotent ALTER calls, and the trailing conn.commit()
```

plus the `# future: if version < 6` comment ? was moved verbatim. No logic, branch conditions, SQL, log strings, return types, ordering, thresholds, or constants changed; all DDL still runs on the one cursor in the one connection/transaction block. The crash-safety/idempotency tests (test_migration_idempotent_on_interrupted_upgrade, test_migration_v3_db_upgrades_to_v5_preserving_rows, test_schema_version_is_current) and async-job tests stay green.",
        "caveat": "Branch was cut from origin/master (a010667); only backend/infrastructure/database.py was staged (explicit path) and only one commit (8c0fa48) sits ahead of origin/master ? no sibling commits. The new helper methods are intentionally untyped (cursor param has no annotation) to match the surrounding pre-existing _init_db style, which mypy accepts without error. Full suite/coverage ran here as the env has pytest/ruff/mypy/cv2/torch/numpy; no GPU/real-video path was exercised (none touched by this refactor). PR left OPEN for owner review ? not merged."
      },
      {
        "slug": "compiler-trim-extract",
        "branch": "refactor/compiler-trim-one-segment",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/224",
        "diffstat": "backend/pipeline/stitcher/compiler.py | 220 ++++++++++++++++++----------- 1 file changed, 120 insertions(+), 100 deletions(-)",
        "suite": "856 passed, 7 skipped",
        "ruff": "All checks passed!",
        "mypy": "Success: no issues found in 104 source files",
        "coverage": "80.76% (floor 70; --cov-fail-under=70 reached)",
        "behavior_note": "Cut-paste extraction of the per-segment for-loop body in VideoCompiler.compile_highlights into a new instance method _trim_one_segment(self, idx, raw_start, raw_end, segments, video_duration, tmp_dir, raw_video_path, watermark_filter) -> (clip_path_or_None, skip_record_or_None). In the original loop each segment resolved to exactly one outcome ? it either appended one clip (no skip) or `continue`d after ap...
[/external_agent_tool_result]

417. assistant (2026-07-02T06:11:33.273Z)

All 7 refactor agents succeeded ? **8 PRs total now open** for your review (each self-reported a green local gate). But agent self-reports aren't enough for a "no regression" guarantee, so let me **independently verify** via GitHub CI and confirm each branch is exactly one clean commit off master.

418. assistant (2026-07-02T06:11:33.273Z)

[external_agent_tool_call: Bash]
description: Verify commit count + CI status for all 8 PRs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git fetch origin 2>&1 | tail -1
echo "=== all OPEN PRs ==="
gh pr list --state open --limit 30 --json number,title,headRefName -q '.[] | "#\(.number) \(.headRefName) ? \(.title)"'
echo
echo "=== per-PR: commits-off-master (must be 1) + CI rollup ==="
for pr in 221 222 223 224 225 226 227 228; do
  br=$(gh pr view $pr --json headRefName -q .headRefName 2>/dev/null)
  n=$(git rev-list --count origin/master..origin/$br 2>/dev/null)
  echo "--- PR #$pr ($br): $n commit(s) ahead of master ---"
  gh pr checks $pr 2>&1 | awk '{print $1"\t"$2}' | sort | uniq -c | sed 's/^/   /'
done
[/external_agent_tool_call]

419. assistant (2026-07-02T06:11:33.273Z)

[external_agent_tool_result]
=== all OPEN PRs ===
#228 refactor/main-uploads-apirouter ? refactor(api): extract chunked-upload island from main.py into backend.api.uploads router

```
#227  refactor/yolo-hybrid-process-video-helpers  ?  refactor(segmenter): decompose yolo_hybrid
process_video into phase helpers
#226  refactor/ablation-pdf-module  ?  refactor(eval): split ablation export_pdf into
backend/eval/ablation_pdf
#225  refactor/gemini-process-video-helpers  ?  refactor(gemini): decompose process_video into
named private helpers
#224  refactor/compiler-trim-one-segment  ?  refactor(stitcher): extract per-segment trim into
_trim_one_segment
#223  refactor/db-init-helpers  ?  refactor(db): extract _init_db base-table + per-version
migration helpers
#222  refactor/tracknet-dedup-and-active-frames  ?  refactor(tracknet): dedup midpoint-split +
extract _active_frames helper
#221  fix/config-loader-non-object-json  ?  fix(config): non-object config.json must not crash
startup (degrade to defaults + warn)
#220  feat/long-rally-lens  ?  feat(eval): P7 long-rally (>5s) quality lens + reel knob (both
default-OFF)
#219  docs/desktop-app-plan  ?  docs(desktop): scope a cross-platform desktop app + the de-WSL
native-compute port

=== per-PR: commits-off-master (must be 1) + CI rollup ===
--- PR #221 (fix/config-loader-non-object-json): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #222 (refactor/tracknet-dedup-and-active-frames): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #223 (refactor/db-init-helpers): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #224 (refactor/compiler-trim-one-segment): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #225 (refactor/gemini-process-video-helpers): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #226 (refactor/ablation-pdf-module): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #227 (refactor/yolo-hybrid-process-video-helpers): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
           1 Mypy       (lenient
           1 Ruff       lint
           1 Syntax     sweep
--- PR #228 (refactor/main-uploads-apirouter): 1 commit(s) ahead of master ---
           1 Full       test
           3 Golden     fixtures
```

```
          1 Mypy       (lenient
          1 Ruff       lint
          1 Syntax     sweep
[/external_agent_tool_result]
```

420. assistant (2026-07-02T06:11:33.273Z)

All 8 branches are exactly **1 clean commit off master**. My grouping hid the pass/fail column ? let me pull the actual CI conclusion per PR (and wait on any still running).

421. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_call: Bash]
description: Get pass/fail/pending CI rollup per PR
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
for pr in 221 222 223 224 225 226 227 228; do
  state=$(gh pr view $pr --json statusCheckRollup -q '[.statusCheckRollup[].conclusion] |
(map(select(.=="FAILURE"))|length) as $f | (map(select(.==null or .==""")) | length) as $p | if
$f>0 then "FAIL" elif $p>0 then "PENDING" else "PASS" end' 2>/dev/null)
  echo "PR #$pr: $state"
done
[/external_agent_tool_call]
```

422. assistant (2026-07-02T06:11:33.273Z)

```
[external_agent_tool_result]
PR #221: PASS
PR #222: PASS
PR #223: PASS
PR #224: PASS
PR #225: PASS
PR #226: PASS
PR #227: PASS
PR #228: PENDING
[/external_agent_tool_result]
```

423. assistant (2026-07-02T06:11:33.273Z)

7/8 CI **PASS** (independently confirming the agents' green gates); only the medium-risk **#228** (uploads router) is still running. That's exactly the one worth my own eyes ? it had to resolve a circular import and preserve the `global_config` seam. Let me spot-check its diff while CI finishes.

424. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_call: Bash]
description: Spot-check #228 config seam and wiring
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== #228 diffstat ==="
git diff --stat origin/master..origin/refactor/main-uploads-apirouter
echo
echo "=== the config seam: how uploads.py resolves global_config ==="
git show origin/refactor/main-uploads-apirouter:backend/api/uploads.py | grep -nE
"global_config|import backend.main|_main|def _upload_cfg|_abort_upload|APIRouter|router =" |
head -20
echo
echo "=== main.py: what replaced the island (import + include_router) ==="
git diff origin/master..origin/refactor/main-uploads-apirouter -- backend/main.py | grep -nE
"^\+|^-" | grep -iE "uploads|include_router|import|global_config|Request" | head -20
[/external_agent_tool_call]
```

425. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_result]
```

```
=== #228 diffstat ===
 backend/api/__init__.py       |   1 +
 backend/api/uploads.py        | 155 ++++++++++++++++++++++++++++++++++++++++++++
 backend/main.py               | 138 ++++----------------------------------
 tests/test_upload_download.py |   3 +-
 4 files changed, 165 insertions(+), 132 deletions(-)

=== the config seam: how uploads.py resolves global_config ===
10:The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
13:``backend.main.global_config`` flow through unchanged).
20:from fastapi import APIRouter, HTTPException, Request
26:router = APIRouter()
41:def _upload_cfg():
45:    import backend.main as _main
46:    sec = getattr(_main.global_config, "security", None)
54:def _abort_upload(upload_id: str) -> None:
114:        _abort_upload(upload_id)
154:    _abort_upload(upload_id)

=== main.py: what replaced the island (import + include_router) ===
9:-from fastapi import FastAPI, HTTPException, Request
10:+from fastapi import FastAPI, HTTPException
18:-# Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
19:-# proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
22:-# aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
25:+# backend.api.uploads as a self-contained APIRouter. It resolves the live startup config via
26:+# backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
28:+from backend.api import uploads as uploads_api
30:-_uploads: Dict[str, Dict[str, Any]] = {}
31:-_uploads_lock = threading.Lock()
34:-class UploadInitRequest(BaseModel):
39:-class UploadCompleteRequest(BaseModel):
44:-    sec = getattr(global_config, "security", None)
45:-    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
53:-    with _uploads_lock:
54:-        st = _uploads.pop(upload_id, None)
63:-def upload_init(req: UploadInitRequest):
81:-    with _uploads_lock:
82:-        _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0, "bytes":
0}
88:-async def upload_part(upload_id: str, index: int, request: Request):
[/external_agent_tool_result]
```

426. assistant (2026-07-02T06:11:33.274Z)

The lazy `import backend.main as _main` correctly breaks the cycle while reading the live `global_config`. Let me confirm the `include_router` wiring and the `upload_dir` fallback are intact, then wait for #228's CI.

427. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_call: Bash]
description: Confirm router wiring and config fallback in #228
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== include_router wired in main.py? ==="
git show origin/refactor/main-uploads-apirouter:backend/main.py | grep -nE
"include_router|uploads_api"
echo "=== _upload_cfg full body (default fallback preserved?) ==="
git show origin/refactor/main-uploads-apirouter:backend/api/uploads.py | sed -n '41,52p'
echo "=== route decorators (paths/status preserved?) ==="
git show origin/refactor/main-uploads-apirouter:backend/api/uploads.py | grep -nE
"@router\.(post|put|delete|get)|status_code|HTTPException\(" | head
[/external_agent_tool_call]
```

428. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_result]
=== include_router wired in main.py? ===
406:from backend.api import uploads as uploads_api
408:app.include_router(uploads_api.router)
=== _upload_cfg full body (default fallback preserved?) ===
def _upload_cfg():
    # Resolved lazily (import inside the function) to read the LIVE startup config that
    # main() sets on the module global, and to avoid a circular import at module load
    # (main.py imports this router; this module must not import main at import time).
    import backend.main as _main
    sec = getattr(_main.global_config, "security", None)
    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
    max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
    part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
    exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions", None) or
["mp4", "mov"])]
    return upload_dir, max_bytes, part_bytes, exts

=== route decorators (paths/status preserved?) ===
64:@router.post("/api/upload/init")
71:        raise HTTPException(status_code=400, detail=str(e)) from e
74:        raise HTTPException(status_code=400,
77:        raise HTTPException(status_code=413,
89:@router.put("/api/upload/{upload_id}/part/{index}")
96:        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
98:        raise HTTPException(status_code=409,
115:         raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-init to
retry.")
123:@router.post("/api/upload/{upload_id}/complete")
130:        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
[/external_agent_tool_result]
```

429. assistant (2026-07-02T06:11:33.274Z)

#228 is a clean, behavior-preserving extraction: router wired, exact config defaults preserved, live `global_config` read via the lazy import, all route paths/status codes intact. Let me wait for its CI to confirm green.

430. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_call: Bash]
description: Watch #228 CI to completion
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
gh pr checks 228 --watch --interval 15 2>&1 | tail -9
echo "=== exit $? ==="
[/external_agent_tool_call]
```

431. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_result]
Ruff lint       pass    9s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27746206451/job/82084999737
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27746206451/job/82084999723
Full test suite (offline, mocked)       pass    2m13s   https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27746206451/job/82084999706
Golden fixtures cross-OS (macos-latest) pass    24s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27746206451/job/82084999773
Golden fixtures cross-OS (ubuntu-latest)        pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27746206451/job/82084999731
Golden fixtures cross-OS (windows-latest)       pass    59s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27746206451/job/82084999755
```

```
Mypy (lenient baseline) pass    32s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27746206451/job/82084999859
Ruff lint       pass    9s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27746206451/job/82084999737
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27746206451/job/82084999723
=== exit 0 ===
[/external_agent_tool_result]
```

432. assistant (2026-07-02T06:11:33.274Z)

All **8 PRs are green on CI** ? independently confirmed, not just agent self-reports. Let me clean up my worktrees (keeping the PR branches intact) and record the checkpoint.

433. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_call: Bash]
description: Clean up worktrees, list final open PRs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== worktrees before cleanup ==="
git worktree list | grep -iE "config-bug|refactor" || echo "(none of mine)"
echo "=== remove MY worktrees (branches/PRs stay intact) ==="
git worktree remove .claude/worktrees/config-bug --force 2>&1 | tail -1 && echo "removed config-
bug"
git worktree remove .claude/worktrees/refactor-base --force 2>&1 | tail -1 && echo "removed
refactor-base"
git worktree prune
echo "=== any leftover fleet worktrees? (auto-isolation should have cleaned) ==="
git worktree list | grep -iE "refactor|isolat|wf_|agent" || echo "(fleet worktrees cleaned)"
echo "=== final: my 8 open PRs ==="
gh pr list --state open --author '@me' --json number,title -q '.[] | "#\(.number)  \(.title)"'
2>/dev/null | head -12
[/external_agent_tool_call]
```

434. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_result]
=== worktrees before cleanup ===
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/config-bug
d5e5ad7 [fix/config-loader-non-object-json]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
a010667 (detached HEAD)
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-1
8c0fa48 [refactor/db-init-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-2
2f30631 [refactor/compiler-trim-one-segment]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-3
39853b2 [refactor/gemini-process-video-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-4
c0ac2f9 [refactor/yolo-hybrid-process-video-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-5
79bd7f1 [refactor/tracknet-dedup-and-active-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-6
d105026 [refactor/ablation-pdf-module]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-7
9c61554 [refactor/main-uploads-apirouter]
=== remove MY worktrees (branches/PRs stay intact) ===
removed config-bug
removed refactor-base
=== any leftover fleet worktrees? (auto-isolation should have cleaned) ===
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed
7e0881a [perf/wasb-fast-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-1
8c0fa48 [refactor/db-init-helpers]
```

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-2
2f30631 [refactor/compiler-trim-one-segment]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-3
39853b2 [refactor/gemini-process-video-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-4
c0ac2f9 [refactor/yolo-hybrid-process-video-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-5
79bd7f1 [refactor/tracknet-dedup-and-active-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-6
d105026 [refactor/ablation-pdf-module]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_87c49086-be4-7
9c61554 [refactor/main-uploads-apirouter]
=== final: my 8 open PRs ===
#228  refactor(api): extract chunked-upload island from main.py into backend.api.uploads router
#227  refactor(segmenter): decompose yolo_hybrid process_video into phase helpers
#226  refactor(eval): split ablation export_pdf into backend/eval/ablation_pdf
#225  refactor(gemini): decompose process_video into named private helpers
#224  refactor(stitcher): extract per-segment trim into _trim_one_segment
#223  refactor(db): extract _init_db base-table + per-version migration helpers
#222  refactor(tracknet): dedup midpoint-split + extract _active_frames helper
#221  fix(config): non-object config.json must not crash startup (degrade to defaults + warn)
#220  feat(eval): P7 long-rally (>5s) quality lens + reel knob (both default-OFF)
#219  docs(desktop): scope a cross-platform desktop app + the de-WSL native-compute port
[/external_agent_tool_result]
```

435. assistant (2026-07-02T06:11:33.274Z)

```
The fleet worktrees persisted (they hold committed branches). The branches are pushed and back
the PRs, so removing the working dirs is safe. Let me prune them.
```

436. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_call: Bash]
description: Remove fleet worktrees, confirm branches intact
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
for w in 1 2 3 4 5 6 7; do
  git worktree remove ".claude/worktrees/wf_87c49086-be4-$w" --force 2>&1 | tail -1 && echo
"removed wf-$w"
done
git worktree prune
echo "=== remaining worktrees (mine should all be gone) ==="
git worktree list | grep -iE "wf_87c49086|config-bug|refactor-base" || echo "(all my sweep
worktrees cleaned)"
echo "=== confirm the 8 PR branches still exist on origin (intact) ==="
git ls-remote --heads origin 'refactor/*' 'fix/config-*' | wc -l
[/external_agent_tool_call]
```

437. assistant (2026-07-02T06:11:33.274Z)

```
[external_agent_tool_result]
removed wf-1
removed wf-2
removed wf-3
removed wf-4
removed wf-5
removed wf-6
removed wf-7
=== remaining worktrees (mine should all be gone) ===
(all my sweep worktrees cleaned)
=== confirm the 8 PR branches still exist on origin (intact) ===
8
[/external_agent_tool_result]
```

438. assistant (2026-07-02T06:11:33.274Z)

All cleaned up; all 8 PR branches intact. Let me record the checkpoint, then summarize.

439. assistant (2026-07-02T06:11:33.274Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

440. assistant (2026-07-02T06:11:33.274Z)

[external_agent_tool_result]
9
10   # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12   **Checkpoint:** 2026-06-18 (? **P7 LONG-RALLY (>5s) QUALITY LENS + REEL KNOB SHIPPED ? PR [#220](https://github.com/avidullu/badminton-highlight-indexer/pull/220), both default-OFF, awaiting owner review of the NUMBERS**). Owner: "bake in a long-rally lens (eval) + product knob today." Delivered + measured n=15:
[/external_agent_tool_result]

441. assistant (2026-07-02T06:11:33.274Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

442. assistant (2026-07-02T06:11:33.274Z)

[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

443. assistant (2026-07-02T06:11:33.275Z)

Done ? the full sweep landed as **8 review-ready PRs**, all green on CI, none merged (yours to review).

1 bug fix

PR	What
[#221](https://github.com/avidullu/badminton-highlight-indexer/pull/221)	**Startup crash:** a valid-but-non-object `config.json` (`[]`, `42`, `"x"`, `null`) crashed `load_config` (`.items()` / dict-unpack), violating its "non-fatal ? defaults" contract ? i.e. an operator typo took down the server/CLI. One `isinstance` guard ? defaults + warn. Regression test **proven to fail without the fix**. I implemented and verified this one personally.

7 behavior-preserving refactors of the grown files

PR	Target	Debt removed
[#225](https://github.com/avidullu/badminton-highlight-indexer/pull/225)	`gemini.process_video`	the **472-line / 71-branch monster** ? 6 named helpers + a ~40-line orchestrator
[#227](https://github.com/avidullu/badminton-highlight-indexer/pull/227)	`yolo_hybrid.process_video`	309 lines ? 5 phase helpers
[#223](https://github.com/avidullu/badminton-highlight-indexer/pull/223)	`database._init_db`	187-line migration ladder ? `_create_base_tables` + `_migrate_to_vN`
[#224](https://github.com/avidullu/badminton-highlight-indexer/pull/224)	`compiler.compile_highlights`	~100-line trim loop ? `_trim_one_segment`
[#228](https://github.com/avidullu/badminton-highlight-indexer/pull/228)	`main.py` (871 LOC)

| chunked-upload island ? `backend/api/uploads.py` APIRouter (?138 lines from main) |
| [#226](https://github.com/avidullu/badminton-highlight-indexer/pull/226) | `ablation.py` (620 LOC) | reportlab PDF builder ? `ablation_pdf.py` |
| [#222](https://github.com/avidullu/badminton-highlight-indexer/pull/222) | `tracknet_runner.py` | dedup the copy-pasted midpoint-split + extract `_active_frames` |

How I kept quality from regressing

- **Each refactor is a pure cut-paste** (no logic/branch/log/return/ordering/constant changes), backed by existing tests ? gemini & yolo even have **output-equivalence guards**.
- **Independently verified, not just agent self-reports:** every PR is **exactly one clean commit off `master`** and **GitHub CI is fully green** (full suite + ruff + mypy + golden-fixtures on ubuntu/macos/windows). Local gates ~857 passed, coverage ~80.7% (floor 70).
- I **spot-checked the one medium-risk PR (#228)** myself ? confirmed the router wiring, the exact config defaults, the live `global_config` read, and the lazy-import that breaks a real circular dependency.

What I deliberately did *not* do (flagged, not forced)

The discovery pass found ~11 more refactors but **deferred them** as unsafe-without-tests-first: `motion.process_video` (random-heavy, no oracle), `wasb_infer.run()` (GPU-only), the `main.py` job-worker seam, the `golden_fixtures.py` split, the `database` write-method dedup, `rally_seg_eval` reporting. Each needs characterization tests before a safe refactor ? a good follow-up sweep if you want it. Also: if you merge #228, `docs/CODE_MAP.md` needs a one-line update (upload symbols moved out of `backend.main`).

Take your time reviewing ? and if you want, I can merge the ones you approve and turn the deferred list into "add tests ? then refactor" PRs next.

444. user (2026-07-02T06:11:33.275Z)

please merge the PRs when all the test suites and regression tests are green

445. assistant (2026-07-02T06:11:33.275Z)

On it. Per your condition I'll re-verify green right before each merge ? and because these all landed independently, I'll also do a **combined-state check** (merge all 8 together and run the full suite) so the *aggregate* doesn't regress, not just each PR in isolation. First, sync and re-confirm CI + mergeability.

446. assistant (2026-07-02T06:11:33.275Z)

```
[external_agent_tool_call: Bash]
description: Re-verify CI + mergeability for all 8 PRs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git fetch origin 2>&1 | tail -2
echo "=== current origin/master ==="
git log --oneline -1 origin/master
echo "=== re-verify each PR: CI rollup + mergeable ==="
for pr in 221 222 223 224 225 226 227 228; do
  read state mergeable < <(gh pr view $pr --json statusCheckRollup,mergeable -q
'([.statusCheckRollup[].conclusion] | (map(select(.=="FAILURE"))|length) as $f |
(map(select(.==null or .==""))|length) as $p | if $f>0 then "FAIL" elif $p>0 then "PENDING" else
"PASS" end) + " " + .mergeable')
  echo "PR #$pr: CI=$state  mergeable=$mergeable"
done
[/external_agent_tool_call]
```

447. assistant (2026-07-02T06:11:33.275Z)

```
[external_agent_tool_result]
=== current origin/master ===
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
=== re-verify each PR: CI rollup + mergeable ===
PR #221: CI=PASS  mergeable=MERGEABLE
```

```
PR #222: CI=PASS  mergeable=MERGEABLE
PR #223: CI=PASS  mergeable=MERGEABLE
PR #224: CI=PASS  mergeable=MERGEABLE
PR #225: CI=PASS  mergeable=MERGEABLE
PR #226: CI=PASS  mergeable=MERGEABLE
PR #227: CI=PASS  mergeable=MERGEABLE
PR #228: CI=PASS  mergeable=MERGEABLE
[/external_agent_tool_result]
```

448. assistant (2026-07-02T06:11:33.275Z)

All 8: **CI PASS, MERGEABLE**, and master hasn't drifted (still a010667). Now the combined-state check ? merge all 8 together and run the full suite:

449. assistant (2026-07-02T06:11:33.275Z)

```
[external_agent_tool_call: Bash]
description: Merge all 8 branches into a combined worktree
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git worktree add --detach .claude/worktrees/combined-check origin/master 2>&1 | tail -1
cd .claude/worktrees/combined-check
git checkout -b combined-verify 2>&1 | tail -1
for br in fix/config-loader-non-object-json refactor/tracknet-dedup-and-active-frames
refactor/db-init-helpers refactor/compiler-trim-one-segment refactor/gemini-process-video-
helpers refactor/ablation-pdf-module refactor/yolo-hybrid-process-video-helpers refactor/main-
uploads-apirouter; do
  out=$(git merge --no-edit "origin/$br" 2>&1)
  if echo "$out" | grep -qiE "conflict|error"; then echo "CONFLICT on $br:"; echo "$out" | tail
-3; else echo "merged OK: $br"; fi
done
echo "=== combined diffstat vs master ==="
git diff --stat origin/master..HEAD | tail -3
[/external_agent_tool_call]
```

450. assistant (2026-07-02T06:11:33.275Z)

```
[external_agent_tool_result]
HEAD is now at a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config
(#217)
Switched to a new branch 'combined-verify'
merged OK: fix/config-loader-non-object-json
merged OK: refactor/tracknet-dedup-and-active-frames
merged OK: refactor/db-init-helpers
merged OK: refactor/compiler-trim-one-segment
merged OK: refactor/gemini-process-video-helpers
merged OK: refactor/ablation-pdf-module
merged OK: refactor/yolo-hybrid-process-video-helpers
merged OK: refactor/main-uploads-apirouter
=== combined diffstat vs master ===
 tests/test_config.py                          |  20 +
 tests/test_upload_download.py                 |   3 +-
 13 files changed, 1088 insertions(+), 802 deletions(-)
[/external_agent_tool_result]
```

451. assistant (2026-07-02T06:11:33.275Z)

All 8 merged cleanly (no conflicts). Now the binding aggregate gate ? full suite + ruff + mypy + coverage on the combined state:

452. assistant (2026-07-02T06:11:33.275Z)

```
[external_agent_tool_call: Bash]
description: Full gate on the combined 8-PR state
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/combined-check
```

```
echo "=== ruff ===" && python -m ruff check . 2>&1 | tail -1
echo "=== mypy ===" && python -m mypy backend training 2>&1 | tail -1
echo "=== full suite + coverage ===" && python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70 2>&1 | tail -5
[/external_agent_tool_call]
```

453. assistant (2026-07-02T06:11:33.275Z)

```
[external_agent_tool_result]
=== ruff ===
All checks passed!
=== mypy ===
Success: no issues found in 107 source files
=== full suite + coverage ===
Required test coverage of 70% reached. Total coverage: 80.94%
857 passed, 7 skipped, 7 warnings in 88.32s (0:01:28)
======================================================================
 RESULT: PASS - all tests green
======================================================================
[/external_agent_tool_result]
```

454. assistant (2026-07-02T06:11:33.275Z)

Combined state green ? ruff clean, mypy clean (107 files), full suite **857 passed**,
coverage **80.94%**. The aggregate of all 8 does not regress. Safe to merge. Let me clean up the
check worktree and merge all 8 (squash).

455. assistant (2026-07-02T06:11:33.275Z)

```
[external_agent_tool_call: Bash]
description: Squash-merge all 8 PRs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git worktree remove .claude/worktrees/combined-check --force 2>&1 | tail -1; git branch -D
combined-verify 2>&1 | tail -1
echo "=== merging all 8 (squash, delete branch) ==="
for pr in 221 222 223 224 225 226 227 228; do
  res=$(gh pr merge $pr --squash --delete-branch 2>&1 | grep -ivE "^$" | tail -1)
  state=$(gh pr view $pr --json state -q .state 2>/dev/null)
  echo "PR #$pr -> state=$state  ($res)"
done
[/external_agent_tool_call]
```

456. assistant (2026-07-02T06:11:33.275Z)

```
[external_agent_tool_result]
error: failed to delete 'C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/combined-check': Permission denied
Deleted branch combined-verify (was 9aee56f).
=== merging all 8 (squash, delete branch) ===
PR #221 -> state=MERGED  ()
PR #222 -> state=MERGED  ()
PR #223 -> state=MERGED  ()
PR #224 -> state=MERGED  ()
PR #225 -> state=MERGED  ()
PR #226 -> state=MERGED  ()
PR #227 -> state=MERGED  ()
PR #228 -> state=MERGED  ()
[/external_agent_tool_result]
```

457. assistant (2026-07-02T06:11:33.275Z)

All **8 PRs MERGED**. Let me confirm master advanced and watch the master-branch CI (the final
combined-state run on master itself).

458. assistant (2026-07-02T06:11:33.275Z)

```
[external_agent_tool_call: Bash]
description: Confirm master advanced + find master CI run
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git fetch origin 2>&1 | tail -1
echo "=== master now (should show the 8 squash merges on top) ==="
git log --oneline -10 origin/master
echo "=== any of my branches left on origin? (should be 0) ==="
git ls-remote --heads origin 'refactor/*' 'fix/config-*' | wc -l
echo "=== latest master CI run ==="
gh run list --branch master --limit 1 2>&1 | head -3
[/external_agent_tool_call]
```

459. assistant (2026-07-02T06:11:33.276Z)

```
[external_agent_tool_result]
   a010667..e67add1  master     -> origin/master
=== master now (should show the 8 squash merges on top) ===
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
8092a39 refactor(segmenter): decompose yolo_hybrid process_video into phase helpers (#227)
f28ce05 refactor(eval): split ablation export_pdf into backend/eval/ablation_pdf (#226)
90dcc57 refactor(gemini): decompose process_video into named private helpers (#225)
18883b5 refactor(stitcher): extract per-segment trim into _trim_one_segment (#224)
af90e32 refactor(db): extract _init_db base-table + per-version migration helpers (#223)
596f7d3 refactor(tracknet): dedup midpoint-split + extract _active_frames helper (#222)
93d7116 fix(config): degrade to defaults + warn when config.json is valid JSON but not an object
(#221)
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
b460526 docs(recording): assert single-match capture as the #1 rule (now MEASURED) (#218)
=== any of my branches left on origin? (should be 0) ===
0
=== latest master CI run ===
queued          refactor(api): extract chunked-upload island from main into backend.a?  CI
master  push    27746642325     12s     2026-06-18T08:24:19Z
[/external_agent_tool_result]
```

460. assistant (2026-07-02T06:11:33.276Z)

```
Master advanced with all 8 squash-merges; no branches left behind. The master-branch CI is
queued ? let me watch it to green as the official confirmation (I already verified the combined
state locally, but this is the binding check).
```

461. assistant (2026-07-02T06:11:33.276Z)

```
[external_agent_tool_call: Bash]
description: Watch master CI run to completion
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
run_id=$(gh run list --branch master --limit 1 --json databaseId -q '.[0].databaseId')
echo "Watching master CI run $run_id ..."
gh run watch "$run_id" --interval 20 --exit-status 2>&1 | tail -12
echo "=== final conclusion ==="
gh run view "$run_id" --json conclusion,status -q '"status=\(.status)
conclusion=\(.conclusion)"'
[/external_agent_tool_call]
```

462. assistant (2026-07-02T06:11:33.276Z)

```
[external_agent_tool_result]
Watching master CI run 27746642325 ...
  ? Complete job
? Full test suite (offline, mocked) in 2m13s (ID 82086482120)
  ? Set up job
```

```
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? System libs for opencv-python
  ? Install CPU-only torch (no CUDA wheels on a CPU runner)
  ? Install package (editable) + dev tooling
  ? Run full suite (with coverage floor)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
=== final conclusion ===
status=completed conclusion=success
[/external_agent_tool_result]
```

463. assistant (2026-07-02T06:11:33.276Z)

Master CI: **success** ? the merged master is officially green. Let me prune the one leftover worktree dir and update the checkpoint.

464. assistant (2026-07-02T06:11:33.276Z)

```
[external_agent_tool_call: Bash]
description: Prune leftover worktree, confirm no open PRs of mine
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
if [ -d .claude/worktrees/combined-check ]; then rm -rf .claude/worktrees/combined-check 2>&1 |
tail -1; fi
git worktree prune
git worktree list | grep -iE "combined-check|config-bug|refactor-base|wf_87c49086" && echo
"LEFTOVER" || echo "all sweep worktrees cleaned"
echo "=== open PRs authored by me (should be 0) ==="
gh pr list --state open --author '@me' --json number -q 'length'
[/external_agent_tool_call]
```

465. assistant (2026-07-02T06:11:33.276Z)

```
[external_agent_tool_result]
all sweep worktrees cleaned
=== open PRs authored by me (should be 0) ===
2
[/external_agent_tool_result]
```

466. assistant (2026-07-02T06:11:33.276Z)

Let me confirm those 2 remaining open PRs aren't from my sweep (all 8 of mine should be merged).

467. assistant (2026-07-02T06:11:33.276Z)

```
[external_agent_tool_call: Bash]
description: Verify remaining open PRs are not mine
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== still-open PRs ==="
gh pr list --state open --json number,title,headRefName -q '.[] | "#\(.number)  \(.headRefName)
?  \(.title)"'
echo "=== confirm my 8 are all MERGED ==="
for pr in 221 222 223 224 225 226 227 228; do echo -n "#$pr "; gh pr view $pr --json state -q
.state; done
[/external_agent_tool_call]
```

468. assistant (2026-07-02T06:11:33.276Z)

```
[external_agent_tool_result]
=== still-open PRs ===
#220  feat/long-rally-lens  ?  feat(eval): P7 long-rally (>5s) quality lens + reel knob (both
default-OFF)
#219  docs/desktop-app-plan  ?  docs(desktop): scope a cross-platform desktop app + the de-WSL
```

native-compute port
=== confirm my 8 are all MERGED ===
#221 MERGED
#222 MERGED
#223 MERGED
#224 MERGED
#225 MERGED
#226 MERGED
#227 MERGED
#228 MERGED
[/external_agent_tool_result]

469. assistant (2026-07-02T06:11:33.276Z)

Confirmed: all 8 of my PRs are **MERGED**. The 2 still-open (#219 docs, #220 long-rally feature) are other sessions' work ? #220 is awaiting your review of its numbers ? so I correctly left them alone. Let me update the checkpoint.

470. assistant (2026-07-02T06:11:33.276Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

471. assistant (2026-07-02T06:11:33.276Z)

[external_agent_tool_result]
10   # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12   **Checkpoint:** 2026-06-18 (? **REFACTORING SWEEP + BUG ? 8 PRs OPEN FOR OWNER REVIEW (#221-#228), all CI-green, NOT merged**). Owner: "pause dev, refactor the grown files, find bugs + critical tech debt, make PRs (I'll review), ensure quality doesn't regress." Ran a 2-phase discovery workflow (~18 agents): bug-hunt + tech-debt analysis over the grown files (mapped LOC + longest-functions: gemini.process_video=472L/71br worst, golden_fixtures=1113L, main.py=871L, database.py=738L). Produced a risk-rated worklist; the genuinely-risky/under-tested items were correctly DEFERRED (motion's random-heavy path, wasb GPU run(), main.py job-worker seam, golden_fixtures split, _execute_write dedup, rally_seg_eval reporting ? all "needs characterization tests first"). Shipped:
[/external_agent_tool_result]

472. assistant (2026-07-02T06:11:33.276Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

473. assistant (2026-07-02T06:11:33.276Z)

[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

474. assistant (2026-07-02T06:11:33.276Z)

All 8 merged and master is green. Here's the wrap-up.

Merged to `master` (now `e67add1`)

PR	Change

[#221](https://github.com/avidullu/badminton-highlight-indexer/pull/221)	**bug:** non-object `config.json` no longer crashes startup
[#222](https://github.com/avidullu/badminton-highlight-indexer/pull/222)	tracknet: dedup midpoint-split + extract `_active_frames`
[#223](https://github.com/avidullu/badminton-highlight-indexer/pull/223)	database: `_init_db` ? base-table + per-version migration helpers
[#224](https://github.com/avidullu/badminton-highlight-indexer/pull/224)	stitcher: extract `_trim_one_segment`
[#225](https://github.com/avidullu/badminton-highlight-indexer/pull/225)	gemini: the 472-line `process_video` ? 6 named helpers
[#226](https://github.com/avidullu/badminton-highlight-indexer/pull/226)	eval: `export_pdf` ? `ablation_pdf.py`
[#227](https://github.com/avidullu/badminton-highlight-indexer/pull/227)	yolo_hybrid: `process_video` ? phase helpers
[#228](https://github.com/avidullu/badminton-highlight-indexer/pull/228)	main.py: upload island ? `backend/api/uploads.py` router

How I honored "merge when all suites + regression tests are green"

- **Per-PR:** re-verified each was `CI=PASS` + `MERGEABLE` immediately before merging.
- **Aggregate (extra safety against combined regressions):** merged all 8 branches into one throwaway worktree ? **no conflicts** ? and ran the full gate: **857 passed / 7 skipped, ruff clean, mypy clean (107 files), coverage 80.94%**.
- **Post-merge:** the push-triggered **master CI run completed `success`** (full suite + ruff + mypy + golden-fixtures cross-OS). Branches deleted, worktrees pruned.

Left untouched (intentionally)

[#219](https://github.com/avidullu/badminton-highlight-indexer/pull/219) (desktop-app plan) and [#220](https://github.com/avidullu/badminton-highlight-indexer/pull/220) (long-rally lens) are from other sessions ? #220 is awaiting your review of its numbers ? so they're not mine to merge.

Two small follow-ups (your call)

1. **`docs/CODE_MAP.md`** still lists the upload endpoints under `backend.main`; #228 moved them to `backend/api/uploads.py`. A one-line doc update would keep the cold-start map accurate ? want me to push that?
2. The **11 deferred tech-debt items** (motion's random path, `wasb_infer.run()`, the main.py job-worker seam, `golden_fixtures` split, etc.) each need **characterization tests first**. I can do those as "add tests ? then refactor" PRs whenever you want the next sweep.

475. user (2026-07-02T06:11:33.276Z)

please go ahead with the code map and fixing the 11 tech debts with the same rigor and ensuring the quality doiesn;t regress

476. assistant (2026-07-02T06:11:33.277Z)

On it. These are the harder, deferred ones ? so the discipline is **characterization tests first** (pin current behavior, prove they pass on today's code, *then* refactor), the same combined-gate + CI rigor, and an honest "hold" for anything that turns out to be a risky redesign rather than a pure move.

Quick reality check on the 11 from the discovery analysis: a few were explicitly **"don't refactor"** (trajectory_hybrid is already decomposed; person/tracknet grouping needs a redesign, not a pure move) and two need **owner-side GPU/cv2 validation** (`wasb_infer.run()`, the main.py routes/CLI split). I'll do the genuinely-safe ones with tests-first rigor, and report honestly on the rest rather than force a regression.

Let me launch the tech-debt fleet (tests-first ? refactor ? full gate ? open PR), and fix CODE_MAP myself while it runs.

477. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_call: Workflow]

description: Tech-debt fleet: tests-first refactors, open PRs
[/external_agent_tool_call]

478. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_result]
Workflow launched in background. Task ID: wcldbim6x
Summary: Resolve deferred tech-debt items: characterization tests FIRST, then behavior-preserving refactor; open PRs
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_1d39e81c-851
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\techdebt-tests-first-fleet-wf_1d39e81c-851.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\techdebt-tests-first-fleet-wf_1d39e81c-851.js"} to iterate without resending the script.)
Run ID: wf_1d39e81c-851
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\workflows\scripts\techdebt-tests-first-fleet-wf_1d39e81c-851.js", resumeFromRunId: "wf_1d39e81c-851"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.
[/external_agent_tool_result]

479. assistant (2026-07-02T06:11:33.277Z)

The tech-debt fleet (7 agents) is running. Let me do the CODE_MAP fix myself in parallel. First, find what's now stale after the 8 merges.

480. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_call: Bash]
description: Create codemap worktree, find stale upload/pdf refs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git worktree add -b docs/code-map-uploads .claude/worktrees/codemap-fix origin/master 2>&1 | tail -1
cd .claude/worktrees/codemap-fix
echo "=== CODE_MAP references that the 8 merges made stale ==="
grep -nE "upload_init|upload_part|upload_complete|upload_abort|_uploads|chunked.upload|backend.main.*upload|export_pdf|ablation\.py.*pdf" docs/CODE_MAP.md | head -25
[/external_agent_tool_call]

481. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_result]
HEAD is now at e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router (#228)
=== CODE_MAP references that the 8 merges made stale ===
115: - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state (`::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits); per-user partitioning lands with PR-3. `::download_highlight` `@GET /api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95,

allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
[/external_agent_tool_result]

482. assistant (2026-07-02T06:11:33.277Z)

Let me see the section structure around that line to update the module attribution correctly.

483. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\codemap-fix\docs\CODE_MAP.md
[/external_agent_tool_call]

484. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_result]
104   ```
105   Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py` (manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression), `@scripts/nightly_regression.py` (nightly golden-corpus rally-quality tripwire, exit 1=regression/4=stale), `@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning drivers).
106
107   ---
108
109   ## 2. SUBSYSTEMS
110
111   ### 2.0 Orchestration & config
112   - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries may still use `print` for direct output.
113   - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
114   - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx (path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 + job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches + logs traceback). `::_execute_processing` = the former sync body (hash -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in `result["status"]`. `::_get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
115   - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state (`::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits); per-user partitioning lands with PR-3. `::download_highlight` `@GET /api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95,

allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
116   - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
117   - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path....
[/external_agent_tool_result]

485. assistant (2026-07-02T06:11:33.277Z)

The upload endpoints are documented under `@backend/main.py` (line 115); after #228 they live in
`backend/api/uploads.py` (download_highlight stays in main). Let me check for any other
staleness from the 8 merges, then make a focused, accurate edit.

486. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_call: Bash]
description: Check for other stale CODE_MAP refs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/codemap-fix
echo "=== other potentially-stale internal-symbol refs in CODE_MAP ==="
grep -nE
"_init_db|compile_highlights|export_pdf|_active_frames|process_video.*helper|backend/api"
docs/CODE_MAP.md | head
[/external_agent_tool_call]

487. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_result]
=== other potentially-stale internal-symbol refs in CODE_MAP ===
66: -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
112:- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static
UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
114: - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 +
job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches +
logs traceback). `::_execute_processing` = the former sync body (hash -> validate -> add_video
-> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never
raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-
raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result,
reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline
verdict lives in `result["status"]`. `::_get_executor` lazy module-global
`ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests
monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running
-> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST
/api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy
`import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-
path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
117: - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
190:- `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`/`end_padding` ctor defaults 0.5/1.0 but
every live call site (`/api/compile`, CLI trim, `scripts/run_process_and_stitch.py`) threads
`config.json stitching.{begin_padding,end_padding}` (typed default 0.5/0.5); temp clips in a
TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so

final concat can stream-copy ? change a preset and concat may break. ? optional **branding watermark** (`stitching.watermark`, default-on): `::_watermark_filter` builds a `-vf` (drawtext via `textfile=` + optional logo `movie/overlay`) applied to the per-clip re-encode only (concat stays `-c copy`); a failed watermark encode retries the clip WITHOUT it (never nukes the reel, and logs the ffmpeg stderr reason). Pass `watermark=` to `VideoCompiler(...)` (a `WatermarkConfig` model or dict; `None` = off). Default font = bundled Apache-2.0 `::_BUNDLED_FONT` (`backend/assets/fonts/RobotoSlab-Regular.ttf`) so it renders without fontconfig; `font_path` overrides.
216:- `@backend/infrastructure/database.py::Database` ? SQLite, 6 tables (`videos/play_segments/events/candidate_segments/jobs/user_models`), `PRAGMA user_version` migration ladder in `::_init_db` (currently =**5**: `version<1` creates `candidate_segments`;
...
[/external_agent_tool_result]

488. assistant (2026-07-02T06:11:33.277Z)

Only the upload endpoints (line 115) are stale ? `compile_highlights`/`_init_db` etc. are still valid (those refactors kept the public methods as orchestrators). Let me make three targeted, accurate edits.

489. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\codemap-fix\docs\CODE_MAP.md
[/external_agent_tool_call]

490. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\codemap-fix\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

491. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\codemap-fix\docs\CODE_MAP.md
[/external_agent_tool_call]

492. assistant (2026-07-02T06:11:33.277Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\codemap-fix\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

493. assistant (2026-07-02T06:11:33.277Z)

Let me check whether a doc-consistency test parses CODE_MAP (so I run the right gate for a docs PR):

494. assistant (2026-07-02T06:11:33.278Z)

[external_agent_tool_call: Bash]
description: Check for CODE_MAP doc-consistency test
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/codemap-fix
grep -rlnE "CODE_MAP|code_map" tests/ 2>/dev/null || echo "(no test references CODE_MAP)"
echo "=== verify the two edits read coherently ==="
sed -n '115p' docs/CODE_MAP.md | head -c 700; echo
[/external_agent_tool_call]

495. assistant (2026-07-02T06:11:33.278Z)

```
[external_agent_tool_result]
(no test references CODE_MAP)
=== verify the two edits read coherently ===
  - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::_upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST ...
[/external_agent_tool_result]
```

496. assistant (2026-07-02T06:11:33.278Z)

```
No test reads CODE_MAP ? pure docs change, zero code impact. Let me commit, push, and open the
PR (CI will still confirm green).
```

497. assistant (2026-07-02T06:11:33.278Z)

```
[external_agent_tool_call: Bash]
description: Commit, push, open CODE_MAP docs PR
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/codemap-fix
git add docs/CODE_MAP.md
git status --porcelain
git commit -F - <<'EOF'
docs(code-map): point UPLOAD endpoints at backend/api/uploads.py (post #228)

#228 extracted the chunked-upload island (upload_init/part/complete/abort +
_uploads/_uploads_lock + the request models) from backend/main.py into a new
backend/api/uploads.py APIRouter. CODE_MAP §2.0 still listed those symbols under
@backend/main.py. Re-point them at uploads.py, note the include_router wiring +
the lazy backend.main.global_config resolution (cycle-avoiding import), and clarify
that download_highlight was NOT moved (still in main.py). Docs-only; no code touched.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
git push -u origin docs/code-map-uploads 2>&1 | tail -2
gh pr create --base master --head docs/code-map-uploads \
  --title "docs(code-map): point UPLOAD endpoints at backend/api/uploads.py (post #228)" \
  --body "Follow-up to #228 ? the chunked-upload endpoints moved from \`backend/main.py\` to
\`backend/api/uploads.py\`, but \`docs/CODE_MAP.md\` §2.0 still listed them under main.py. Re-
points the symbols, notes the \`include_router\` wiring + lazy \`global_config\` resolution, and
clarifies \`download_highlight\` stayed in main.py. **Docs-only, no code touched** (no test
references CODE_MAP). Keeps the cold-start map honest.

? Generated with [Claude Code](https://claude.com/claude-code)" 2>&1 | tail -2
[/external_agent_tool_call]
```

498. assistant (2026-07-02T06:11:33.278Z)

```
[external_agent_tool_result]
M  docs/CODE_MAP.md
[docs/code-map-uploads 9ed1f52] docs(code-map): point UPLOAD endpoints at backend/api/uploads.py
(post #228)
 1 file changed, 1 insertion(+), 1 deletion(-)
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      docs/code-map-uploads -> docs/code-map-uploads
https://github.com/avidullu/badminton-highlight-indexer/pull/229
[/external_agent_tool_result]
```

499. assistant (2026-07-02T06:11:33.278Z)

CODE_MAP PR [#229](https://github.com/avidullu/badminton-highlight-indexer/pull/229) opened. Now let me collect the tech-debt fleet's results.

500. assistant (2026-07-02T06:11:33.278Z)

[external_agent_tool_call: TaskOutput]
input: {"task_id":"wcldbim6x","block":true,"timeout":600000}
[/external_agent_tool_call]

501. assistant (2026-07-02T06:11:33.278Z)

[external_agent_tool_result]
<retrieval_status>timeout</retrieval_status>

<task_id>wcldbim6x</task_id>

<task_type>local_workflow</task_type>

<status>running</status>
[/external_agent_tool_result]

502. assistant (2026-07-02T06:11:33.278Z)

Still running (tests-first refactors take longer). Let me keep waiting.

503. assistant (2026-07-02T06:11:33.278Z)

[external_agent_tool_call: TaskOutput]
input: {"task_id":"wcldbim6x","block":true,"timeout":600000}
[/external_agent_tool_call]

504. assistant (2026-07-02T06:11:33.278Z)

[external_agent_tool_result]
<retrieval_status>success</retrieval_status>

<task_id>wcldbim6x</task_id>

<task_type>local_workflow</task_type>

<status>completed</status>

<output>

{
  "summary": "Resolve deferred tech-debt items: characterization tests FIRST, then behavior-preserving refactor; open PRs",
  "agentCount": 7,
  "logs": [
    "opened 6/7: https://github.com/avidullu/badminton-highlight-indexer/pull/230
https://github.com/avidullu/badminton-highlight-indexer/pull/232
https://github.com/avidullu/badminton-highlight-indexer/pull/235
https://github.com/avidullu/badminton-highlight-indexer/pull/233
https://github.com/avidullu/badminton-highlight-indexer/pull/231
https://github.com/avidullu/badminton-highlight-indexer/pull/234 | not-opened: base-resolve-provider-dedup=held"
  ],
  "result": {
    "results": [
      {
        "slug": "motion-decompose",
        "branch": "refactor/motion-process-video-tests-then-helpers",
        "status": "pr-opened",
        "pr_url": "https://github.com/avidullu/badminton-highlight-indexer/pull/230",
        "char_tests": "Added tests/test_motion_segmenter.py (3 tests). It mocks the module-level

cv2 (backend.pipeline.segmenters.motion.cv2) ? the WHOLE module is faked, not just VideoCapture, because process_video calls cvtColor/GaussianBlur/absdiff/threshold/dilate/countNonZero directly; a scripted countNonZero drives a deterministic per-frame motion_ratio (mirroring test_yolo_hybrid's _make_cap_mock capture pattern). It SEEDS stdlib random via random.seed(0) before the call so the fabricated server/receiver/metadata/events are reproducible. Pins: (1) test_process_video_failure_on_unopenable_video -> Failure(message='Could not open video: bad.mp4', is_recoverable=False) with no DB writes; (2) test_process_video_no_motion_returns_empty_success -> Success(value=[]) when motion stays below the 0.08 threshold; (3) test_process_video_fabricates_one_segment_deterministically -> Success with exactly one segment dict {id:42, start_time?0.1667, end_time?8.1667, duration?8.0, server:'Deepak', receiver:'Suresh', metadata:{shot_count:5,intensity:'medium',avg_speed_kmh:51.7}}, the exact add_play_segment args, and 4 add_event calls in order (serve Deepak valid/high_serve; smash Suresh 195.8; smash Deepak 156.6; foul Suresh net_touch). All 3 PASSED against the UNCHANGED motion.py before refactoring, and PASSED again unchanged after.",
        "diffstat": "2 files changed, 251 insertions(+), 1 deletion(-). backend/pipeline/segmenters/motion.py | 30 (29 ins, 1 del); tests/test_motion_segmenter.py | 222 (new file).",
        "suite": "python run_all_tests.py -> PASS: 860 passed, 7 skipped in ~65s (and 860 passed / 7 skipped under the coverage run).",
        "ruff": "python -m ruff check . -> All checks passed! (exit 0)",
        "mypy": "python -m mypy backend training -> Success: no issues found in 107 source files (exit 0; only informational annotation-unchecked notes).",
        "coverage": "python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70 -> TOTAL 80.94% (Required 70% reached). Matches base ~80.9% ? no regression. motion.py stays omitted in .coveragerc (omit unchanged), so coverage accounting is unaffected by the new helpers.",
        "behavior_note": "Pure behaviour-preserving extraction. process_video became a thin orchestrator: it still opens the capture and returns the same Failure on !isOpened, then calls _detect_motion_segments(cap, max_frames) (the detection loop verbatim: sub-sample via grab(), motion-signal build, moving-average smooth, threshold->raw segments, dead-time merge, min-duration filter; releases cap; returns the (start,end) list) and _populate_db_with_events(video_id, final_segments, player_pool) (the legacy fabrication half verbatim: random.sample server/receiver, fabricated metadata, serve/0-3 smashes/termination events, DB add_play_segment + add_event, builds per-segment dicts), then wraps the result in Success(value=...). No change to logic, thresholds, co...
[/external_agent_tool_result]

505. assistant (2026-07-02T06:11:33.278Z)

The tech-debt fleet finished: **6 PRs opened, 1 correctly HELD**. The held one (`base-resolve-provider-dedup`) is exactly the rigor working ? the agent found the two `_resolve_provider` bodies *aren't* byte-identical (5 real differences) so a "pure lift" would silently change behavior; it stopped instead of forcing it. Good.

Let me independently verify CI + commit-cleanliness on the 6 + the docs PR, then scrutinize the riskiest (#234 job-worker: it touched `mypy.ini` and reworked the monkeypatch seam).

506. assistant (2026-07-02T06:11:33.278Z)

[external_agent_tool_call: Bash]
description: Verify CI + commit count for the 7 PRs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git fetch origin 2>&1 | tail -1
echo "=== per-PR: commits-off-master + CI rollup + coverage-ish ==="
for pr in 229 230 231 232 233 234 235; do
  br=$(gh pr view $pr --json headRefName -q .headRefName 2>/dev/null)
  n=$(git rev-list --count origin/master..origin/$br 2>/dev/null)
  state=$(gh pr view $pr --json statusCheckRollup -q '[.statusCheckRollup[].conclusion] | (map(select(.=="FAILURE"))|length) as $f | (map(select(.==null or .==""))|length) as $p | if $f>0 then "FAIL" elif $p>0 then "PENDING" else "PASS" end' 2>/dev/null)
  echo "PR #$pr ($br): commits=$n CI=$state"
done
[/external_agent_tool_call]

507. assistant (2026-07-02T06:11:33.278Z)

```
[external_agent_tool_result]
=== per-PR: commits-off-master + CI rollup + coverage-ish ===
PR #229 (docs/code-map-uploads): commits=1  CI=PASS
PR #230 (refactor/motion-process-video-tests-then-helpers): commits=1  CI=PASS
PR #231 (refactor/rally-seg-eval-build-preset): commits=1  CI=PASS
PR #232 (refactor/db-execute-write-helper): commits=1  CI=PASS
PR #233 (refactor/golden-fixtures-real-split): commits=1  CI=PASS
PR #234 (refactor/main-job-worker-module): commits=1  CI=PASS
PR #235 (refactor/eval-load-videos-shared): commits=1  CI=PENDING
[/external_agent_tool_result]
```

508. assistant (2026-07-02T06:11:33.278Z)

6/7 CI PASS (incl. the complex #234), all 1 clean commit; #235 still running. Let me scrutinize #234's `mypy.ini` change and the monkeypatch-seam (the deferral hazard) before trusting it.

509. assistant (2026-07-02T06:11:33.278Z)

```
[external_agent_tool_call: Bash]
description: Scrutinize #234 mypy.ini + seam indirection
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
echo "=== #234 mypy.ini change (should be a quarantine entry mirroring backend.main) ==="
git diff origin/master..origin/refactor/main-job-worker-module -- mypy.ini
echo "=== #234 main.py re-export shim (how worker symbols stay addressable on backend.main) ==="
git diff origin/master..origin/refactor/main-job-worker-module -- backend/main.py | grep -E
"^\+" | grep -iE "job_worker|import|JobWaiting|_drain|_arm_wake|_get_executor|_MAX_DRAIN|noqa" |
head -20
echo "=== job_worker.py: do cross-fn seam calls route through backend.main (so monkeypatch
intercepts)? ==="
git show origin/refactor/main-job-worker-module:backend/api/job_worker.py | grep -nE "import
backend.main|_main\.|def _drain_due_jobs|def
_arm_wake_timer|_arm_wake_timer\(|_drain_due_jobs\(|_get_executor\(" | head -20
[/external_agent_tool_call]
```

510. assistant (2026-07-02T06:11:33.278Z)

```
[external_agent_tool_result]
=== #234 mypy.ini change (should be a quarantine entry mirroring backend.main) ===
diff --git a/mypy.ini b/mypy.ini
index 9698460..57e63fd 100644
--- a/mypy.ini
+++ b/mypy.ini
@@ -14,6 +14,12 @@ explicit_package_bases = True
 [mypy-backend.main]
 ignore_errors = True

+# Job-worker drain/wake loop extracted from backend.main (same Optional module-global
+# debt: resolves db_instance/global_config THROUGH backend.main at call-time). Shares
+# backend.main's quarantine until the deferred app-factory/lifespan refactor lands.
+[mypy-backend.api.job_worker]
+ignore_errors = True
+
 # Legacy demo segmenter (fabricated metadata; coverage-omitted too).
 [mypy-backend.pipeline.segmenters.motion]
 ignore_errors = True
=== #234 main.py re-export shim (how worker symbols stay addressable on backend.main) ===
+# now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
+# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
+# `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams THROUGH
+# intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
+from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable on
backend.main)
```

```
+    JobWaiting,
+    _arm_wake_timer,
+    _drain_due_jobs,
+    _get_executor,
+    _MAX_DRAIN_WAKE_SEC,
+    _schedule_next_drain_if_waiting,
=== job_worker.py: do cross-fn seam calls route through backend.main (so monkeypatch
intercepts)? ===
11:`backend.main` module object at call-time (`import backend.main as _main`), never as a
33:def _get_executor() -> ThreadPoolExecutor:
63:    import backend.main as _main
64:    return _main.ProcessRequest(
81:    import backend.main as _main
82:    db_instance = _main.db_instance
84:    req = _main._job_to_request(job)
86:        result = _main._execute_processing(
91:    except _main.JobWaiting as w:
100:def _drain_due_jobs() -> int:
109:    import backend.main as _main
112:        job = _main.db_instance.claim_due_job()
116:        _main._run_claimed_job(job)
120:    _main._schedule_next_drain_if_waiting()
128:    import backend.main as _main
130:        delay = _main.db_instance.seconds_until_next_due()
137:    _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
140:def _arm_wake_timer(delay_sec: float) -> None:
144:    import backend.main as _main
145:    timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
[/external_agent_tool_result]
```

511. assistant (2026-07-02T06:11:33.278Z)

#234 is correct: the `mypy.ini` change is a minimal quarantine entry mirroring the existing `backend.main` one (same already-excused Optional-global debt), and every cross-function call routes through `backend.main` (`_main._arm_wake_timer(...)`) so existing `monkeypatch.setattr(m, ...)` still intercepts ? the leaked-timer hazard is correctly avoided. Let me wait for #235's CI, then run the combined-state gate before merging.

512. assistant (2026-07-02T06:11:33.278Z)

```
[external_agent_tool_call: Bash]
description: Watch #235 CI
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
gh pr checks 235 --watch --interval 15 2>&1 | tail -8
echo "=== #235 final ===" && gh pr view 235 --json mergeStateStatus -q .mergeStateStatus
[/external_agent_tool_call]
```

513. assistant (2026-07-02T06:11:33.278Z)

```
[external_agent_tool_result]
Syntax sweep + import smoke (no GPU stack)      pass    20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27753949047/job/82111019403
Full test suite (offline, mocked)       pass    2m14s   https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27753949047/job/82111019447
Golden fixtures cross-OS (macos-latest) pass    18s     https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27753949047/job/82111019365
Golden fixtures cross-OS (ubuntu-latest)        pass    20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27753949047/job/82111019357
Golden fixtures cross-OS (windows-latest)       pass    54s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27753949047/job/82111019370
Mypy (lenient baseline) pass    31s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27753949047/job/82111019412
Ruff lint       pass    12s     https://github.com/avidullu/badminton-highlight-
```

```
indexer/actions/runs/27753949047/job/82111019360
Syntax sweep + import smoke (no GPU stack)      pass    20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27753949047/job/82111019403
=== #235 final ===
CLEAN
[/external_agent_tool_result]
```

514. assistant (2026-07-02T06:11:33.279Z)

All 7 are CI-green. Now the binding combined-state gate ? merge all 7 together and run the full suite:

515. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_call: Bash]
description: Merge all 7 tech-debt+docs branches together
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git worktree add --detach .claude/worktrees/combined-td origin/master 2>&1 | tail -1
cd .claude/worktrees/combined-td
git checkout -b combined-td-verify 2>&1 | tail -1
for br in docs/code-map-uploads refactor/motion-process-video-tests-then-helpers refactor/rally-
seg-eval-build-preset refactor/db-execute-write-helper refactor/golden-fixtures-real-split
refactor/main-job-worker-module refactor/eval-load-videos-shared; do
  out=$(git merge --no-edit "origin/$br" 2>&1)
  if echo "$out" | grep -qiE "conflict|error"; then echo "CONFLICT on $br:"; echo "$out" | tail
-3; else echo "merged OK: $br"; fi
done
echo "=== combined diffstat ===" && git diff --stat origin/master..HEAD | tail -2
[/external_agent_tool_call]
```

516. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_result]
HEAD is now at e67add1 refactor(api): extract chunked-upload island from main into
backend.api.uploads router (#228)
Switched to a new branch 'combined-td-verify'
merged OK: docs/code-map-uploads
merged OK: refactor/motion-process-video-tests-then-helpers
merged OK: refactor/rally-seg-eval-build-preset
merged OK: refactor/db-execute-write-helper
merged OK: refactor/golden-fixtures-real-split
merged OK: refactor/main-job-worker-module
merged OK: refactor/eval-load-videos-shared
=== combined diffstat ===
 tests/test_rally_seg_eval_report.py   | 306 ++++++++++++++++++++++++++++++
 19 files changed, 1657 insertions(+), 650 deletions(-)
[/external_agent_tool_result]
```

517. assistant (2026-07-02T06:11:33.279Z)

All 7 merged cleanly. Now the full gate on the combined state:

518. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_call: Bash]
description: Combined-state full gate for the 7 PRs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/combined-td
echo "=== ruff ===" && python -m ruff check . 2>&1 | tail -1
echo "=== mypy ===" && python -m mypy backend training 2>&1 | tail -1
echo "=== full suite + coverage ===" && python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70 2>&1 | tail -5
[/external_agent_tool_call]
```

519. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_result]
=== ruff ===
All checks passed!
=== mypy ===
Success: no issues found in 109 source files
=== full suite + coverage ===
Required test coverage of 70% reached. Total coverage: 82.68%
889 passed, 7 skipped, 7 warnings in 85.17s (0:01:25)
================================================================================
 RESULT: PASS - all tests green
================================================================================
[/external_agent_tool_result]
```

520. assistant (2026-07-02T06:11:33.279Z)

Combined state green ? ruff clean, mypy clean (109 files), **889 passed**, coverage **82.68%** (UP from 80.94% base ? the characterization tests net-improved coverage). Aggregate doesn't regress. Merging all 7 (squash):

521. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_call: Bash]
description: Squash-merge all 7 PRs
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git worktree remove .claude/worktrees/combined-td --force 2>&1 | tail -1; git branch -D
combined-td-verify 2>&1 | tail -1
echo "=== merging all 7 (squash, delete branch) ==="
for pr in 229 230 231 232 233 234 235; do
  gh pr merge $pr --squash --delete-branch >/dev/null 2>&1
  echo "PR #$pr -> $(gh pr view $pr --json state -q .state 2>/dev/null)"
done
[/external_agent_tool_call]
```

522. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_result]
error: failed to delete 'C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/combined-td': Permission denied
Deleted branch combined-td-verify (was 9ab95b7).
=== merging all 7 (squash, delete branch) ===
PR #229 -> MERGED
PR #230 -> MERGED
PR #231 -> MERGED
PR #232 -> MERGED
PR #233 -> MERGED
PR #234 -> MERGED
PR #235 -> MERGED
[/external_agent_tool_result]
```

523. assistant (2026-07-02T06:11:33.279Z)

All 7 MERGED. Let me confirm master and watch the final master-branch CI run.

524. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_call: Bash]
description: Confirm master + watch master CI
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
git fetch origin 2>&1 | tail -1
echo "=== master now ===" && git log --oneline -8 origin/master
run_id=$(gh run list --branch master --limit 1 --json databaseId -q '.[0].databaseId')
echo "=== watching master CI run $run_id ==="
gh run watch "$run_id" --interval 20 --exit-status 2>&1 | tail -4
gh run view "$run_id" --json conclusion -q '"master CI conclusion=\(.conclusion)"'
```

[/external_agent_tool_call]

525. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_result]
   e67add1..5f555cf  master     -> origin/master
=== master now ===
5f555cf refactor(eval): extract one shared golden-feature loader behind the three load_videos
(#235)
5409fc4 refactor(api): extract job-worker drain/wake loop from main into backend.api.job_worker
(#234)
5c46ebc refactor(eval): split real-fixture subsystem into golden_real_fixtures (pure move)
(#233)
97fa187 refactor(db): route bool-returning single-statement writers through _execute_write
(#232)
ae10447 refactor(eval): extract _build_preset(args, side) from rally_seg_eval.main() (#231)
1e324ea refactor(motion): extract detection + DB-populate halves from process_video (#230)
53b9cef docs(code-map): point UPLOAD endpoints at backend/api/uploads.py (post #228) (#229)
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
=== watching master CI run 27754238078 ===
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
master CI conclusion=success
[/external_agent_tool_result]
```

526. assistant (2026-07-02T06:11:33.279Z)

Master CI: **success**. Let me prune leftover worktrees and record the checkpoint.

527. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_call: Bash]
description: Prune worktrees, confirm clean state
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
for d in combined-td codemap-fix; do [ -d ".claude/worktrees/$d" ] && rm -rf
".claude/worktrees/$d" 2>&1 | tail -1; done
git worktree prune
echo "=== my open PRs (should be 0) ===" && gh pr list --state open --json number,headRefName -q
'.[] | select(.headRefName|test("refactor/|fix/|docs/code-map")) | "#\(.number)
\(.headRefName)"' || true
echo "=== leftover sweep worktrees? ===" && git worktree list | grep -iE
"combined|codemap|refactor|config-bug|wf_" || echo "(clean)"
echo "=== held branch still on origin? (base-resolve-provider had no PR) ===" && git ls-remote
--heads origin 'refactor/base-resolve-provider*' | wc -l
[/external_agent_tool_call]
```

528. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_result]
=== my open PRs (should be 0) ===
=== leftover sweep worktrees? ===
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-1
3057ada [refactor/motion-process-video-tests-then-helpers]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3
f7b8a78 [refactor/db-execute-write-helper]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-4
5445dbc [refactor/eval-load-videos-shared]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-5
727a421 [refactor/golden-fixtures-real-split]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-6
0a014ba [refactor/rally-seg-eval-build-preset]
```

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-7
174cef6 [refactor/main-job-worker-module]
=== held branch still on origin? (base-resolve-provider had no PR) ===
0
[/external_agent_tool_result]
```

529. assistant (2026-07-02T06:11:33.279Z)

```
Fleet worktrees persisted (held the merged branches). Let me prune them and their local
branches.
```

530. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_call: Bash]
description: Remove fleet worktrees and local branches
command: cd /c/Users/avidu/Projects/badminton-highlight-indexer
for w in 1 3 4 5 6 7; do git worktree remove ".claude/worktrees/wf_1d39e81c-851-$w" --force 2>&1
| tail -1; done
git worktree prune
git branch -D refactor/motion-process-video-tests-then-helpers refactor/db-execute-write-helper
refactor/eval-load-videos-shared refactor/golden-fixtures-real-split refactor/rally-seg-eval-
build-preset refactor/main-job-worker-module refactor/base-resolve-provider-dedup docs/code-map-
uploads 2>&1 | tail -3
echo "=== remaining sweep worktrees? ===" && git worktree list | grep -iE
"wf_1d39e81c|combined|codemap|refactor" || echo "(all clean)"
[/external_agent_tool_call]
```

531. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_result]
Deleted branch refactor/main-job-worker-module (was 174cef6).
Deleted branch refactor/base-resolve-provider-dedup (was e67add1).
Deleted branch docs/code-map-uploads (was 9ed1f52).
=== remaining sweep worktrees? ===
(all clean)
[/external_agent_tool_result]
```

532. assistant (2026-07-02T06:11:33.279Z)

```
[external_agent_tool_result]
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
[/external_agent_tool_result]
```

533. assistant (2026-07-02T06:11:33.279Z)

```
All clean. Let me record the checkpoint.
```

534. assistant (2026-07-02T06:11:33.280Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md
[/external_agent_tool_call]
```

535. assistant (2026-07-02T06:11:33.280Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md
[/external_agent_tool_call]
```

536. assistant (2026-07-02T06:11:33.280Z)

[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

537. assistant (2026-07-02T06:11:33.280Z)

Done ? the CODE_MAP fix and the deferred tech-debt sweep are merged to master (`5f555cf`), with the characterization-tests-first rigor throughout.

Merged (CODE_MAP + 6 refactors)

PR	What	Tests-first
[#229](https://github.com/avidullu/badminton-highlight-indexer/pull/229)	CODE_MAP: upload endpoints re-pointed to `backend/api/uploads.py`	docs-only
[#230](https://github.com/avidullu/badminton-highlight-indexer/pull/230)	motion: split `process_video` into detect + db-populate halves	new char tests mock cv2 + seed `random` to pin the fabricated segment/events (the untested 204-line path)
[#231](https://github.com/avidullu/badminton-highlight-indexer/pull/231)	rally_seg_eval: extract `_build_preset(args, side)`	11 char tests for the 0%-coverage report/main() paths ? module **0%?98%**
[#232](https://github.com/avidullu/badminton-highlight-indexer/pull/232)	database: route the 11 bool writers through `_execute_write`	6 failure-branch tests pin each **exact log message** ? byte-identical
[#233](https://github.com/avidullu/badminton-highlight-indexer/pull/233)	golden_fixtures: split real-fixture subsystem out (?296 lines)	lazy PEP-562 re-export (handles both import orders) + 7 char tests
[#234](https://github.com/avidullu/badminton-highlight-indexer/pull/234)	main.py: job-worker loop ? `backend/api/job_worker.py`	3 seam tests prove `monkeypatch` still intercepts (no leaked timer)
[#235](https://github.com/avidullu/badminton-highlight-indexer/pull/235)	eval: one shared `load_golden_videos` behind the 3 loaders	char test pins the 3-way audio-policy difference

How quality was protected

- **Tests-first discipline:** every refactor added characterization tests that pin today's behavior, proven to pass on the *unchanged* code before any refactor, then again after.
- **Independently verified:** each PR = 1 clean commit + green GitHub CI. I personally spot-checked the riskiest (#234: the `mypy.ini` quarantine entry mirrors the existing `backend.main` one, and the seam routes through `backend.main` so monkeypatches still intercept).
- **Combined-state gate:** merged all 7 together ? full suite **889 passed**, ruff + mypy (109 files) clean, **coverage 82.68% ? up from 80.94%** (the char tests net-*added* coverage). Master push-CI: **success**. Worktrees/branches cleaned.

1 held for your call

`base-resolve-provider` dedup ? the agent found gemini's and yolo_hybrid's `_resolve_provider` are **not** identical (5 real differences: signature, `sport_profile` capture-vs-discard, `model_name` source, an extra log line, return shape). Folding them to a base method would **change behavior**, not move code ? so it correctly stopped. Doable only as a deliberate redesign (with char tests around both segmenters' early-returns + the relocated log line) **if you want it**.

4 reported as not-a-pure-refactor (not forced)

- **trajectory_hybrid** ? already decomposed (hook-based base class); nothing to do.
- **person_detector ? tracknet grouping dedup** ? different tuple shapes ? a redesign with real regression surface, not a move.
- **`wasb_infer.run()`** ? GPU/WSL-only; needs an owner-side torch-mocked test + a real run to refactor safely.
- **main.py routes-vs-CLI split** ? high blast radius; needs cv2-backed tests first.

So: **7 of the 11** are resolved or correctly closed-out (3 of those 11 were "don't-do" by design), 1 awaits your decision, and the 3 GPU/redesign ones I've scoped for when you want them. Want me to take on the held dedup as an explicit redesign, or set up the characterization tests

for the `wasb_infer.run()` / main-routes splits next?